S32K3XX
PINS & CLOCKS - RTD

Automotive Systems & Applications Engineering Team
GPIS-BL, Automotive Processing

SECURE CONNECTIONS
FOR A SMARTER WORLD

PUBLIC

AGENDA

SIUL2

1. SIUL2 Features Overview
2. S32DS Pins Configuration Tool
3. Siul2_Dio and Siul2_Port API Functions
4. Siul2_Icu Configuration
5. Siul2_Icu API Functions

Clocks

1. S32K3 Clocking Overview
2. Clock Configuration Tool
3. Clock API Functions

SIUL2 Features Overview

SECURE CONNECTIONS
FOR A SMARTER WORLD

SIUL2 FEATURES

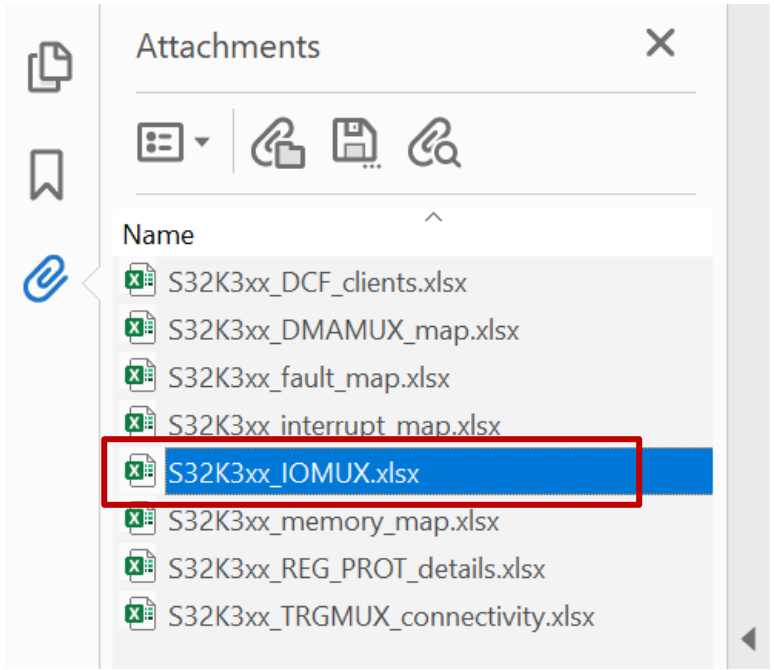
- Number of GPIO PINs

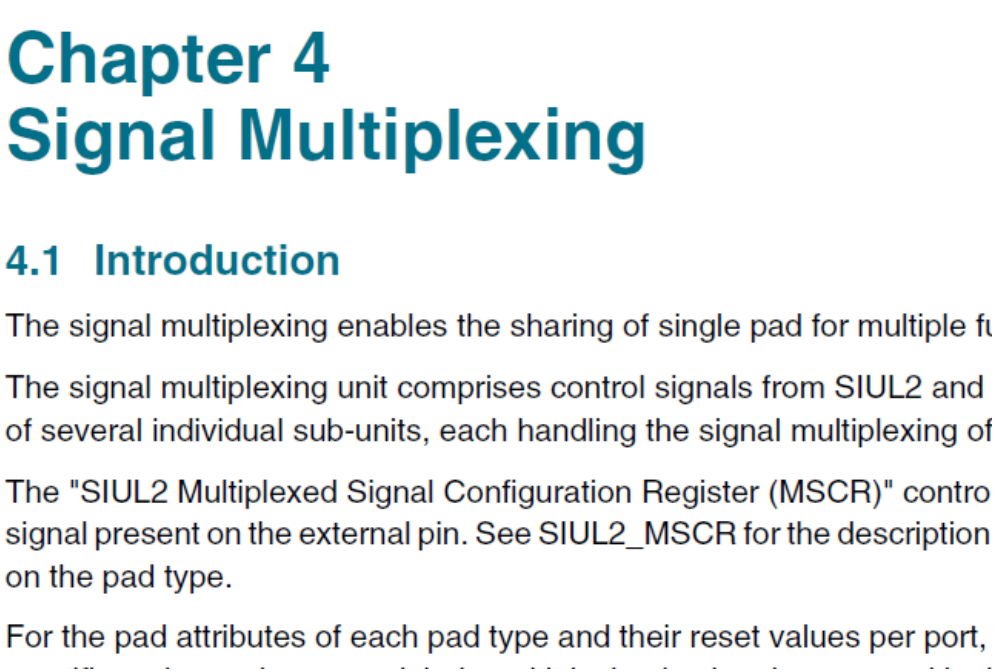
Package	Port	GPIO pins	GPI pins	Power & GND pins	XOSC pins	Debug and reset pins
BGA 257	PTA~PTG	211	2	37	2	5
MaxQFP 172	PTA~PTE	136	2	27	2	5

- One PORT (PTA~PTG) includes max 32 pins which can output or input in parallel. (16 pins as a group)
- Interrupts and DMA requests:
  - 32 IRQ pins – every 8 pins share one interrupt vector.
  - Rising edge or falling edge, with digital glitch filter.
  - 16 pins are linked to DMA requests.
- Two power domains: VDD_HV_A and VDD_HV_B
  - It's possible to apply 3.3 V and 5 V on different S32K3 pins at the same time.
- IO Pad Type

IO Pad Type	Max Frequency	High drive-strength	Slew-rate control
GPIO-Standard	10MHz	Not supported	Not supported
GPIO-Standard plus	25Mhz	Supported	Not supported
GPIO-Medium	50MHz	Supported	Supported
GPIO-Fast	210MHz	Supported	Supported

- For the attributes of each pad, refer to the tab “IO Signal Table” the *S32K3xx_IOMUX.xlsx* attached to the RM.





Chapter 4
Signal Multiplexing

4.1 Introduction

The signal multiplexing enables the sharing of single pad for multiple fu

The signal multiplexing unit comprises control signals from SIUL2 and of several individual sub-units, each handling the signal multiplexing of

The "SIUL2 Multiplexed Signal Configuration Register (MSCR)" contro signal present on the external pin. See SIUL2_MSCR for the description on the pad type.

For the pad attributes of each pad type and their reset values per port,

SIUL2 - IO SIGNAL TABLE (REFER TO ATTACHMENT IN REF MANUAL)

Po	CR	SSS	Function	Module	Description	Direction	Pad Type	S32K344_172mqfp	S32K344_257bga	I/O Power Domain
PTA0	SIUL_MSCR0	0000_0000	GPIO[0]	SIUL		I/O	GPIO-STANDARD	137	A13	VDD_HV_A
		0000_0001	LPSPI4_PCS2	LPSPI4	Peripheral Chip Select 2	O				
		0000_0010	eMIOS_0_CH[17]_Y	eMIOS_0	eMIOS Channel	O				
		0000_0011	LCU0_OUT4	LCU0	LCU Output	O				
		0000_0100	FXIO_D2	FXIO	FlexIO Bi-directional Shift/timer I/O	O				
		0000_0101	eMIOS_1_CH[0]_X	eMIOS_1	eMIOS Channel	O				
		0000_0110	LPSPI0_PCS7	LPSPI0	Peripheral Chip Select 7	O				
		0000_0111	TRGMUX_OUT3	TRGMUX	Trigger Mux Output	O				
	-	-	ADC0_S8	ADC0	ADC Standard Input	I				
	-	-	CMP1_IN0	CMP1	Comparator Input Signal	I				
	SIUL_IMCR528	0000_0001	EIRQ[0]	SIUL	External Interrupt	I				
	SIUL_IMCR577	0000_0010	eMIOS_0_CH[17]_Y	eMIOS_0	eMIOS Channel	I				
	SIUL_IMCR592	0000_0011	eMIOS_1_CH[0]_X	eMIOS_1	eMIOS Channel	I				
	SIUL_IMCR666	0000_0010	FXIO_D2	FXIO	FlexIO Bi-directional Shift/timer I/O	I				
	SIUL_IMCR740	0000_0001	LPSPI0_PCS7	LPSPI0	Peripheral Chip Select 7	I				
	SIUL_IMCR769	0000_0001	LPSPI4_PCS2	LPSPI4	Peripheral Chip Select 2	I				
	SIUL_IMCR872	0000_0001	LPUART0_CTS	LPUART0	Clear To Send (bar)	I				

- **SIUL_MSCR** for selecting output function
- **SIUL_IMCR** for selecting input function
- **Pad Type** indicates the max output frequency and drive strength
- **I/O Power Domain** determines this pin's voltage level

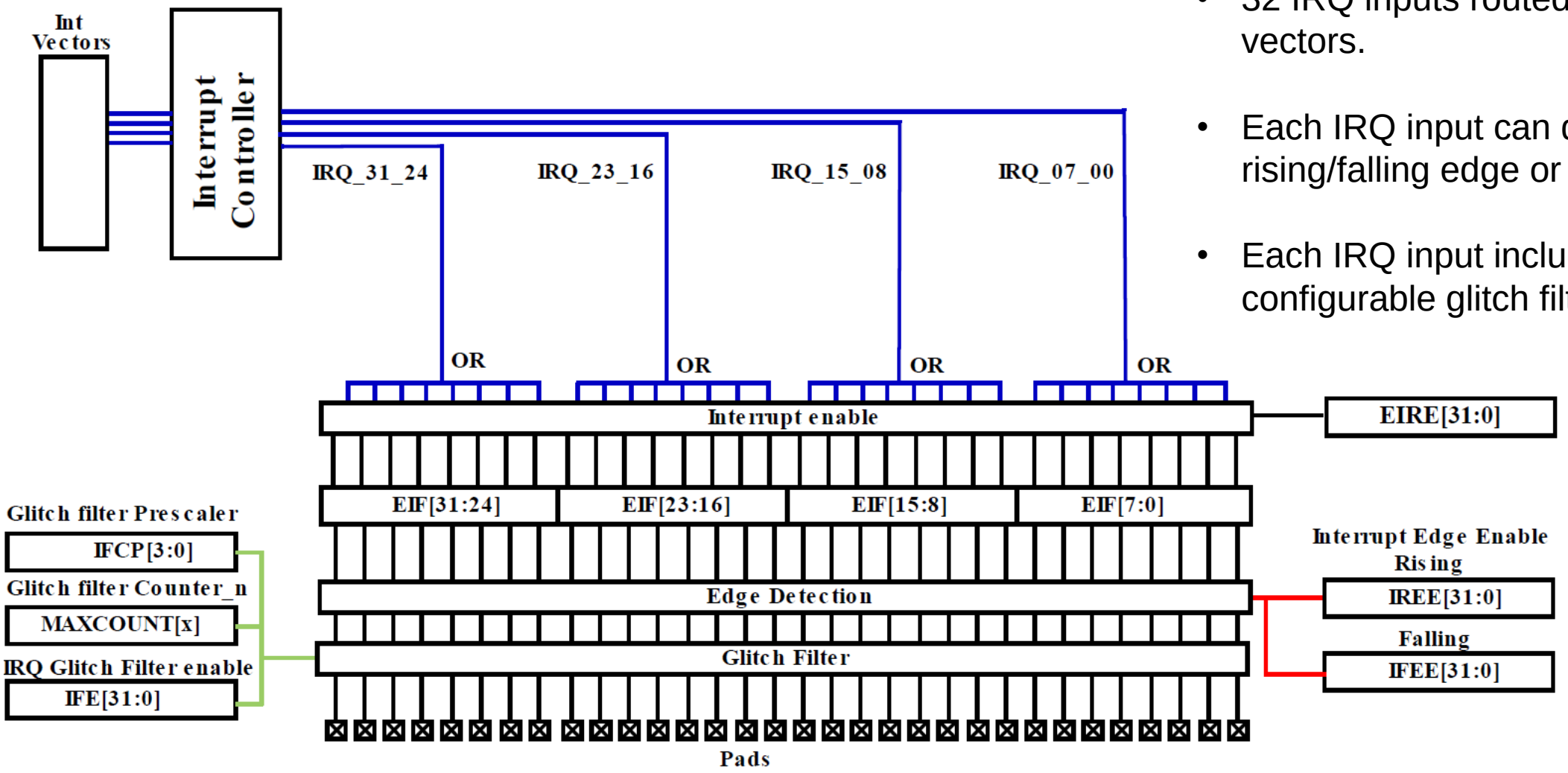
- The SIUL2 provides control and configurability of the functions and electrical characteristics that appear on external device pins.

- S32K3xx PORT assignments are shown below

Port	SIUL2_MSCRn index
PortA[0-31]	MSCR0 - MSCR31
PortB[0-31]	MSCR32 - MSCR63
PortC[0-31]	MSCR64 - MSCR95
PortD[0-31]	MSCR96 - MSCR127
PortE[0-31]	MSCR128 - MSCR159
PortF[0-31]	MSCR160 - MSCR191
PortG[0-27]	MSCR192 - MSCR219

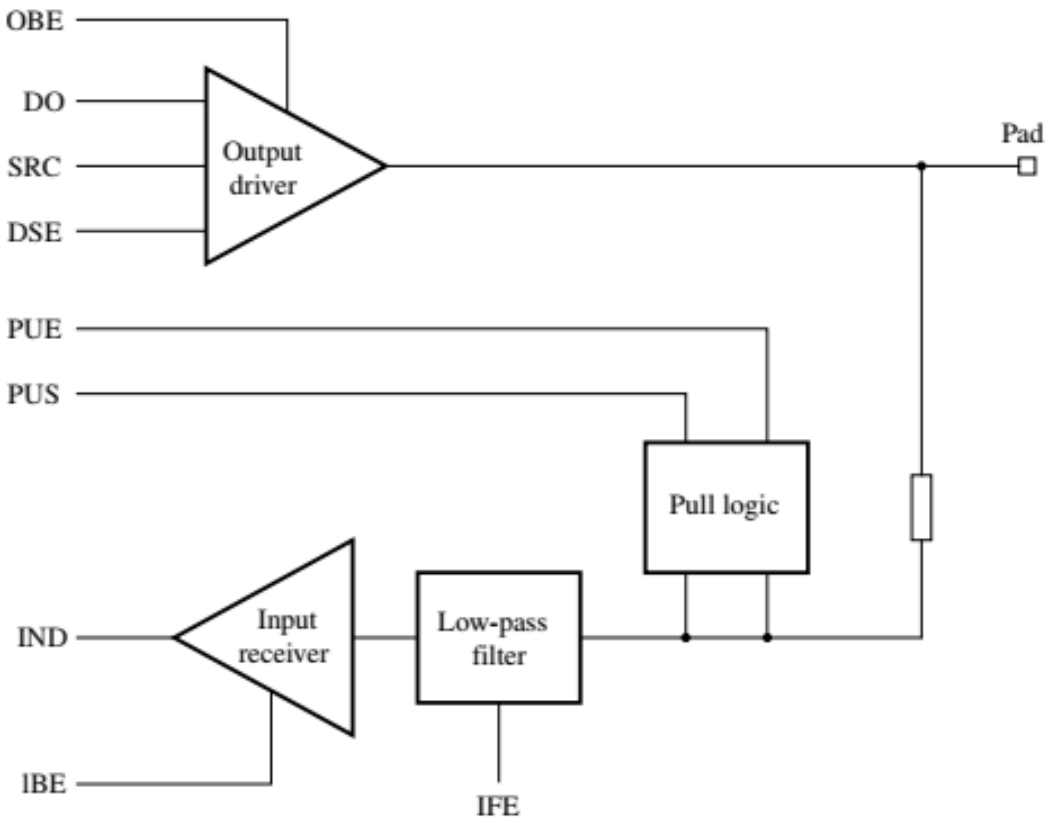
MSCR: Multiplexed Signal Control Register

EXTERNAL INTERRUPT REQUEST



- 32 IRQ inputs routed to 4 Int vectors.
- Each IRQ input can detect rising/falling edge or both edges.
- Each IRQ input includes configurable glitch filter.

GPIO PAD INTERNAL BLOCK DIAGRAM



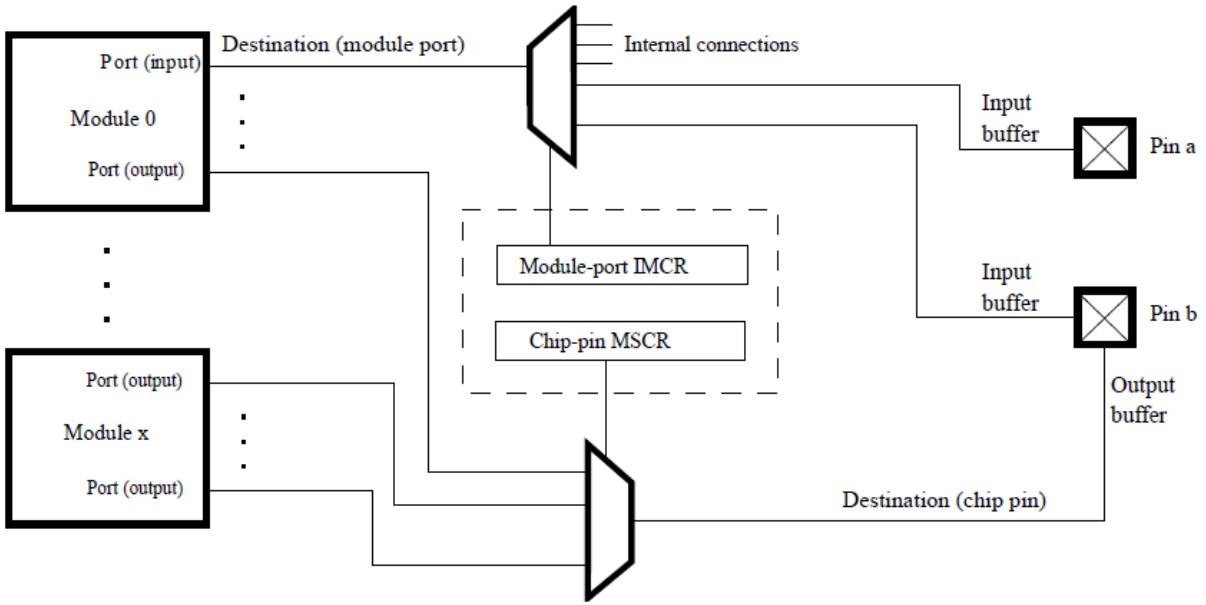
Pad as Output:

- OBE – output buffer enable
- SRC – slew rate control
- DSE – driver strength enable
- MSCR_SSS – output function select

Pad as Input:

- IBE – input buffer enable
- IFE – input filter enable
- PUE – pull-up/pull-down enable
- PUS – pull-up/pull-down select
- IMCR_SSS - input function select

- For input selection, there is another dedicated register: **IMCR**, Where the **SSS** field, must be configured according to the *S32K3xx_IOMUX.xlsx.*



- "CR" numbers 0 to 511 correspond to the MSCR register instances.
- "CR" numbers 512 to 1023 correspond to the IMCR register instances.

SW EXAMPLE: SETTING PTB0 AND PTB1 AS CAN0 RX AND TX

378	PTB0	SIUL_MSCR32	0000_0000	GPIO[32]	SIUL
388	PTB0	-	-	WKPU[7]	WKPU
389	PTB0	SIUL_IMCR512	0000_0011	CAN0_RX	CAN0
390	PTB0	SIUL_IMCR535	0000_0001	EIRQ[8]	SIUL
391	PTB0	SIUL_IMCR563	0000_0100	eMIOS_0_CH[3]_G	eMIOS_0
392	PTB0	SIUL_IMCR598	0000_0001	eMIOS_1_CH[6]_H	eMIOS_1
393	PTB0	SIUL_IMCR678	0000_0011	FXIO_D14	FXIO

```
// CAN0_RX (PTB0)
SIUL2->IMCR[512-512] = SIUL2_IMCR_SSS(0b011); // CAN0_RX
SIUL2->MSCR[32]      = SIUL2_MSCR_IBE_MASK; // Input Buffer
```

397	PTB1	SIUL_MSCR33	0000_0000	GPIO[33]	SIUL
398	PTB1		0000_0001	LPI2C0_SCLS	LPI2C0
399	PTB1		0000_0010	LPUART0_TX	LPUART0
400	PTB1		0000_0011	LPSPI0_SOUT	LPSPI0
401	PTB1		0000_0100	eMIOS_0_CH[7]_G	eMIOS_0
402	PTB1		0000_0101	CAN0_TX	CAN0

```
// CAN0_TX (PTB1)
SIUL2->MSCR[33] = 0b101 // CAN0_TX
                | SIUL2_MSCR_OBE_MASK; // Output Buffer
```

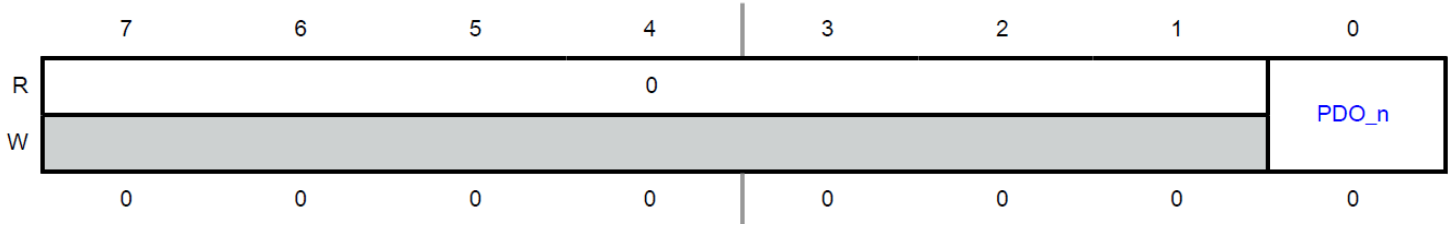
"CR" numbers (512 to 1023) correspond to the **IMCR** register instances.

IMCR512 – 512 = **IMCR0**

GENERAL PURPOSE INPUT AND OUTPUT PADS (GPIO)

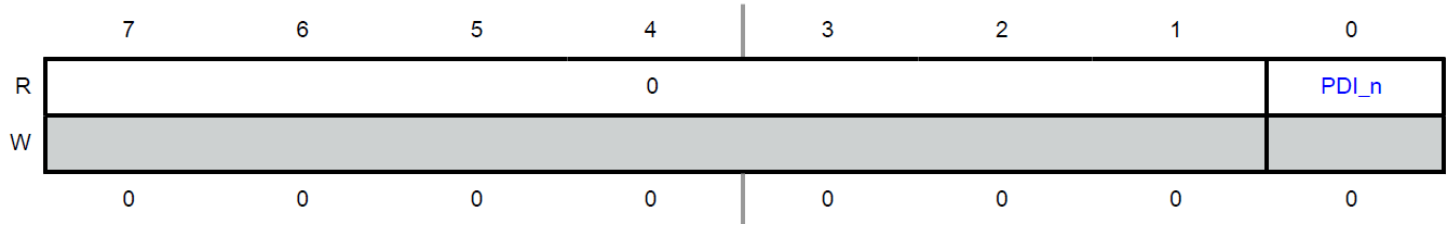
- SIUL2 has separate data input and data output registers for all pads.
- Data output registers support both read and write operations.
- Data input registers support read access only.
- When a pad is configured to use one of its alternate functions, the data input values reflect the respective value of the pad and write operations do not affect the pad state.

GPDO0 - GPDO219



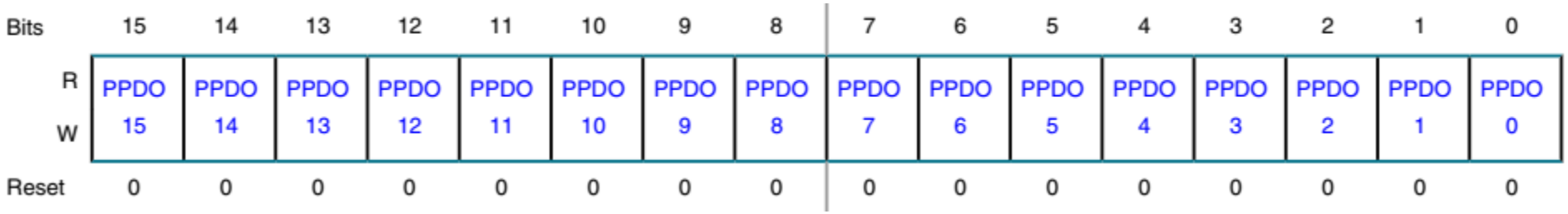
void Siul2_Dio_Ip_WriteChannel(uint8 u8Siul2Instance, Siul2_Dio_Ip_PinsChannelType pin, Siul2_Dio_Ip_PinsLevelType value)

GPDI0 - GPDI219



Siul2_Dio_Ip_PinsLevelType Siul2_Dio_Ip_ReadChannel(uint8 u8Siul2Instance, Siul2_Dio_Ip_PinsChannelType pin)

- PGPDOn register:
  - PGPDO*n* registers can set the values of all output pins assigned to a chip port with a single 16-bit register write.
  - For example, writing to PGPDO0 can change GPDO0~GPDO15 in a single write.



Bits	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	PPDO	PPDO	PPDO	PPDO	PPDO	PPDO	PPDO	PPDO	PPDO	PPDO	PPDO	PPDO	PPDO	PPDO	PPDO	PPDO
W	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

PGPDO bit mapping:

PGPDO0_PPDO15 bit => GPDO0 (PTD_0)
PGPDO0_PPDO14 bit => GPDO1 (PTD_1)
……
PGPDO0_PPDO0 bit => GPDO15 (PTD_15)

PGPDO1_PPDO15 bit => GPDO16 (PTD_16)
PGPDO1_PPDO14 bit => GPDO17 (PTD_17)
……
PGPDO1_PPDO0 bit => GPDO31 (PTD_31)

API functions accessing PGPDO/PGPDI registers

```
Siul2_Dio_Ip_WritePin
Siul2_Dio_Ip_WritePins
Siul2_Dio_Ip_SetPins
Siul2_Dio_Ip_ClearPins
Siul2_Dio_Ip_TogglePins
Siul2_Dio_Ip_ReadPin
Siul2_Dio_Ip_ReadPins
```

S32DS
Pin Configuration Tool

SECURE CONNECTIONS
FOR A SMARTER WORLD

START THE PINS CONFIGURATION TOOL

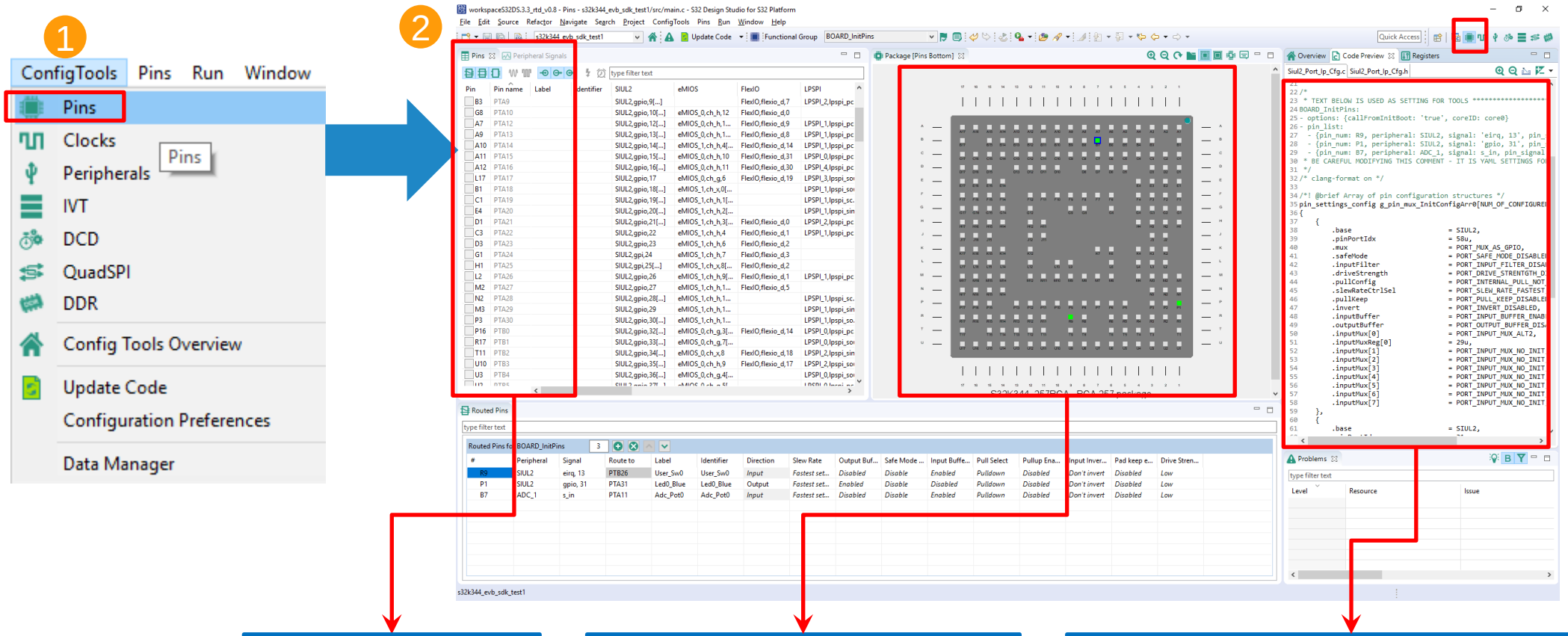
- The Pin Configuration Tool is a useful tool for the customer to easily and quickly enable the pins required for the application.

- Each pin has a set of options to configure depending on the peripheral that is going to be used in the project

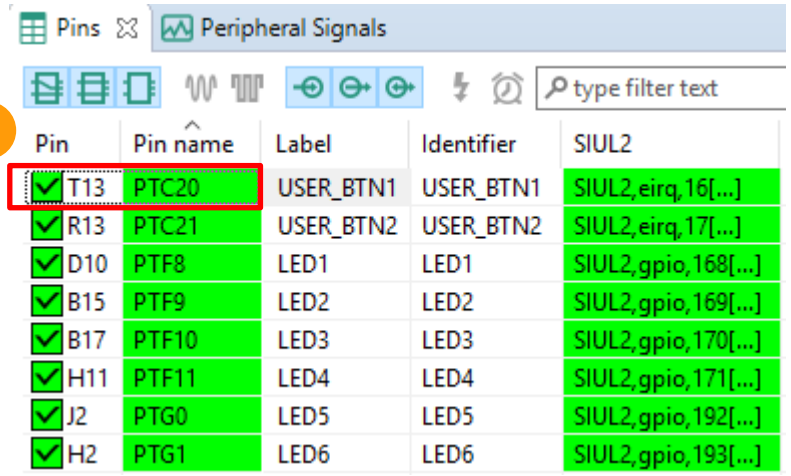
Some configuration examples are shown in the following slides but notice that the number of pins and their specific configuration depends on the customer's application.

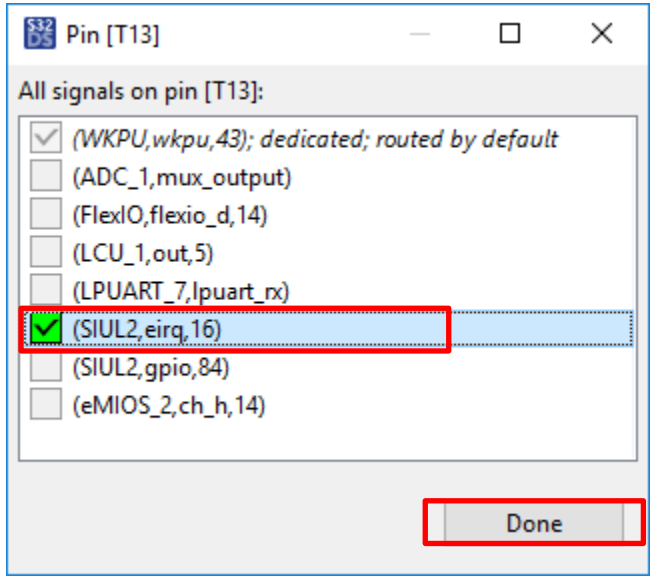
START THE PINS CONFIGURATION TOOL

- Double click the “ .mex” file to start the pins configuration tool.
- Or click menu ConfigTools => Pins, to start the pins configuration tool.
- Use Pins Configuration Tool to configure the function/property for each pin.

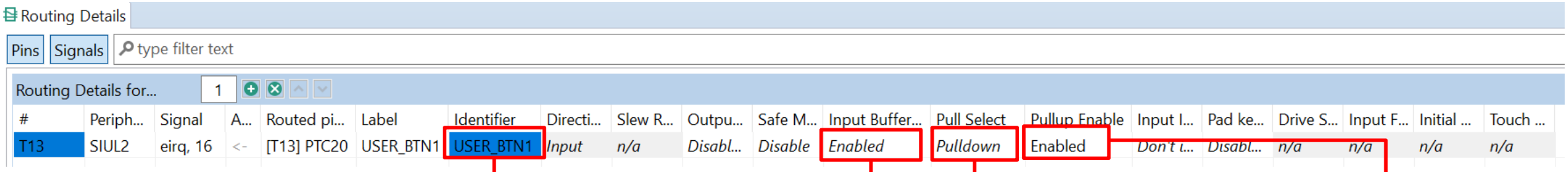


- Configure PTC20 as EIRQ16.





3



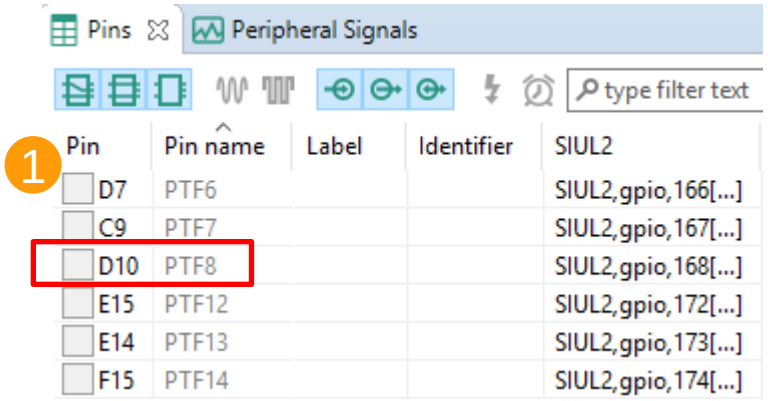
It is recommended to use the **Identifier** field to give a meaningful ID to each pin.
Pin Id can be used in Siul2_Dio API functions.

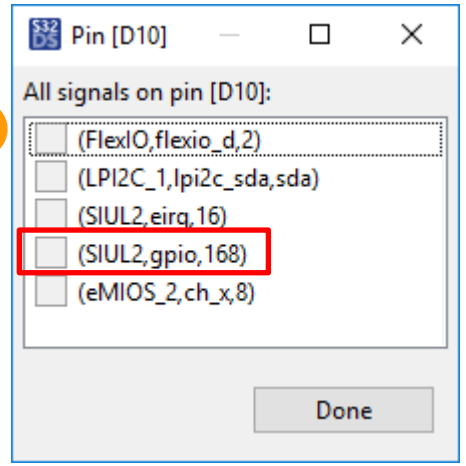
Input buffer enable

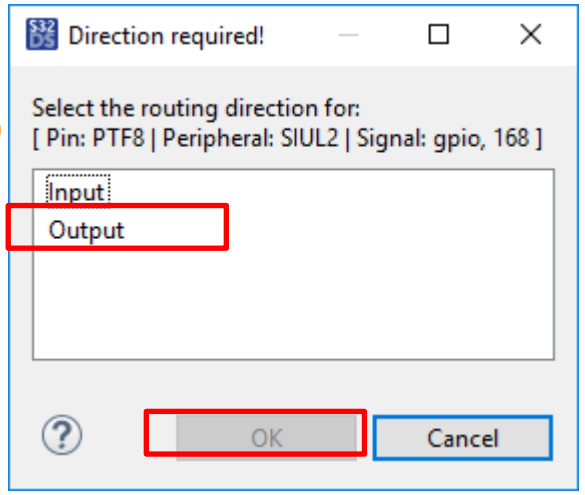
Pull up/down select

Pull up/down enable

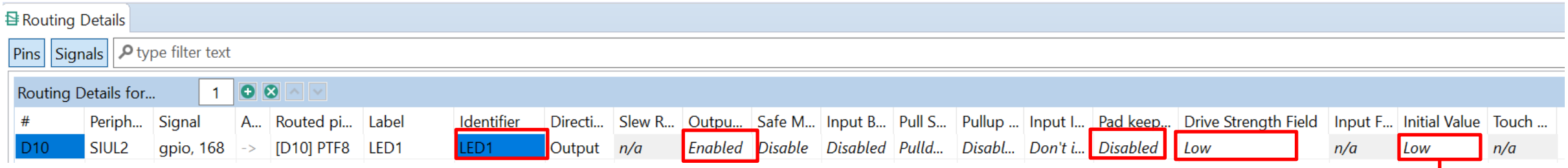
- Configure PTF8 as SIUL GPDO





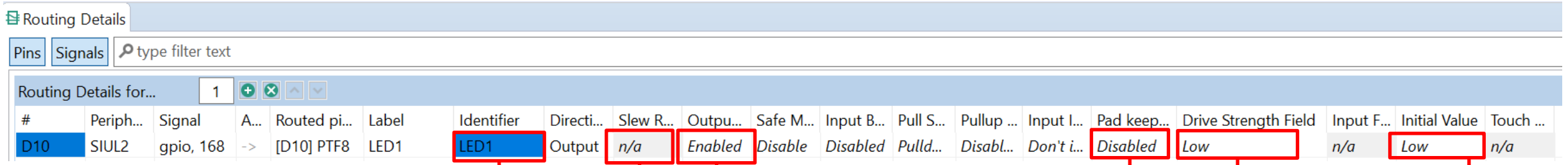


4



The output initial value should be configured properly to avoid output glitch during MCU startup.

CONFIGURE AN OUTPUT PIN (CONTINUED)



MSCR_SRC

MSCR_OBE

MSCR_PKE

MSCR_DSE

It is recommended to use the **Identifier** field to give a meaningful ID to each pin.

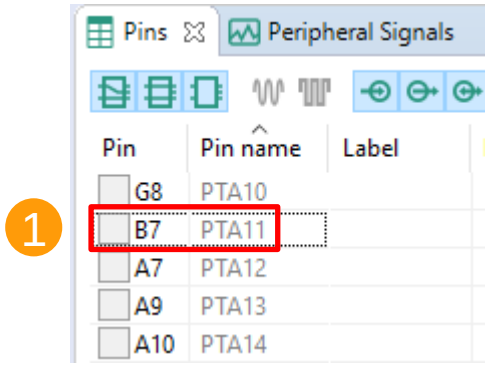
The pin “Identifier” is used to generate the port and pin MACRO defines in the file “Siul2_Port_Ip_Cfg.h”;
The MACRO defines can be used in some Siul2 DIO APIs:

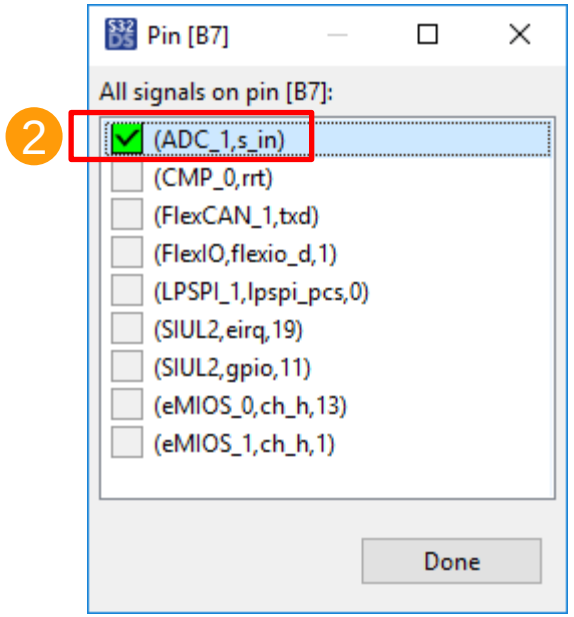
- Siul2_Dio_Ip_WritePins / Siul2_Dio_Ip_TogglePins

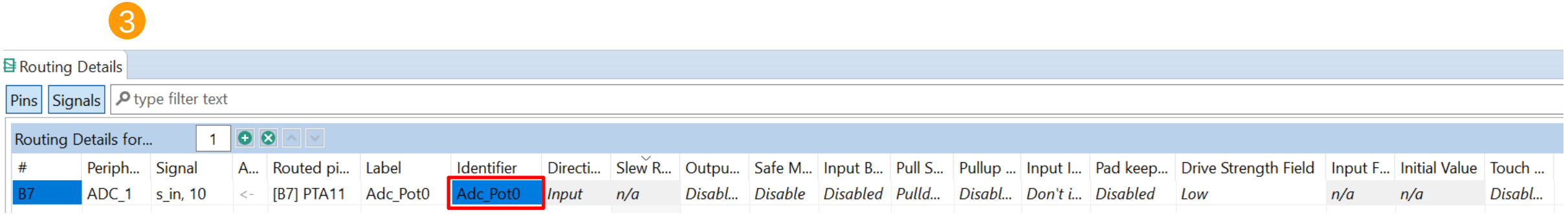
The output initial value should be configured properly to avoid output glitch during MCU startup.

CONFIGURE ADC INPUT PIN

- Configure PTA11 as ADC1_S10 input.

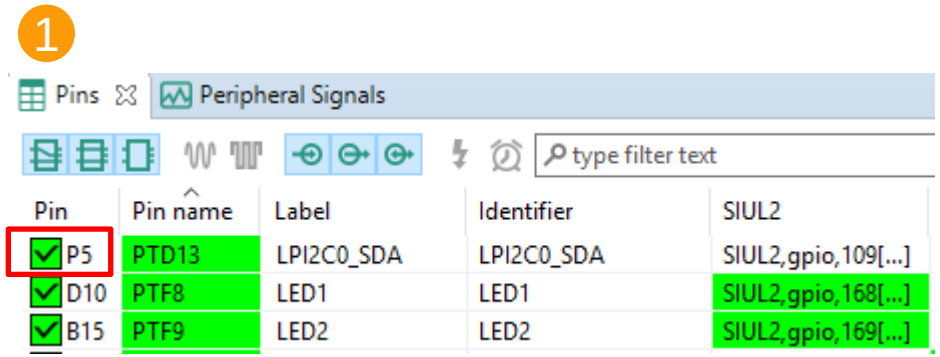




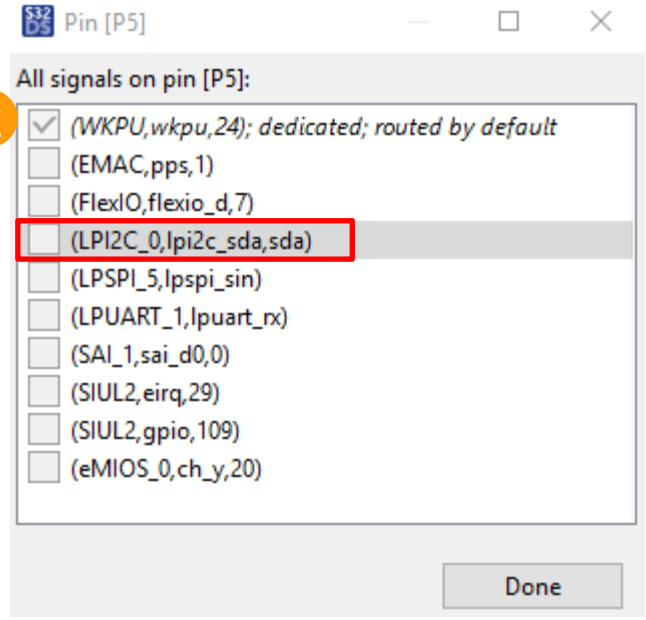


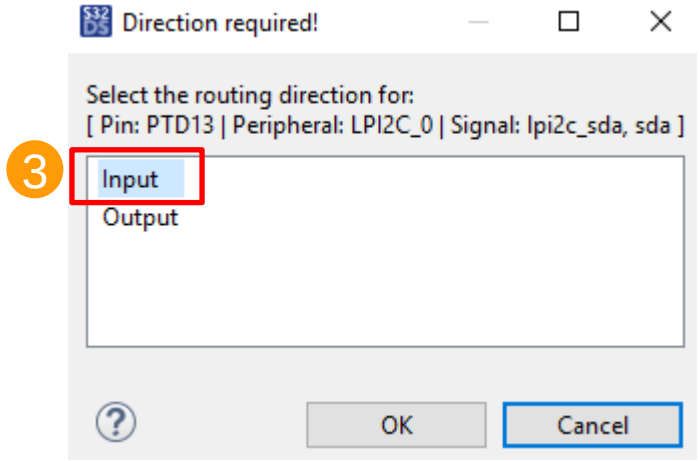
CONFIGURE I2C SDA PIN AS INPUT AND OUTPUT

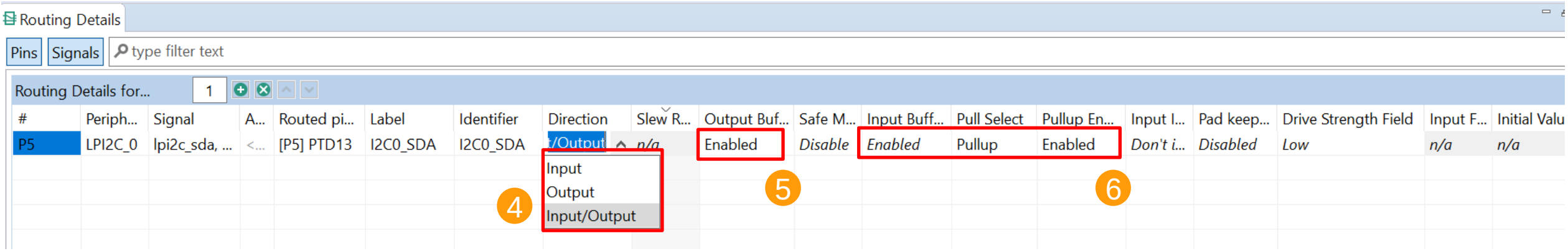
- Configure PTD13 as LPI2C0_SDA input/output.



3) Configure the pin as LPI2C SDA input;
4) Set the Direction as "**Input/Output**"
5) Set the Output Buffer as enable;

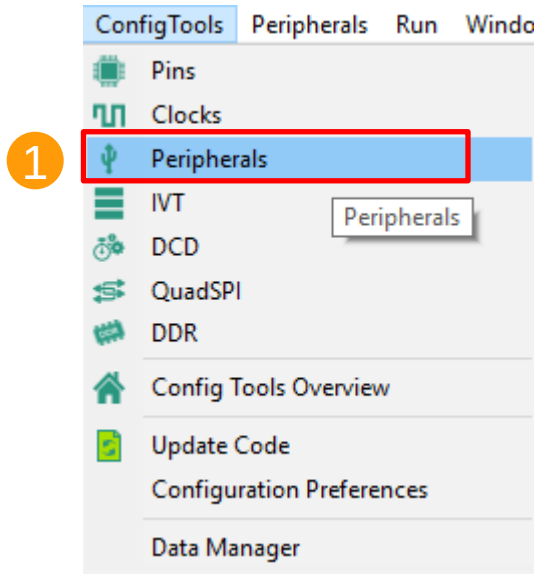


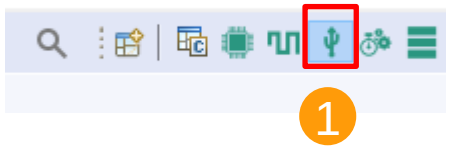


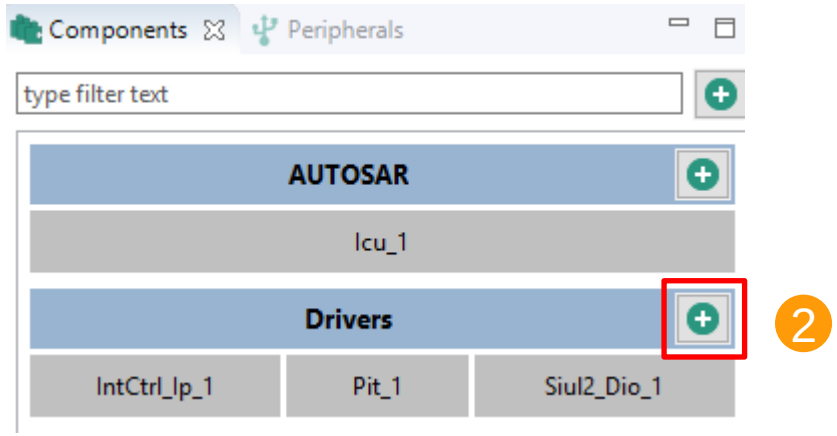


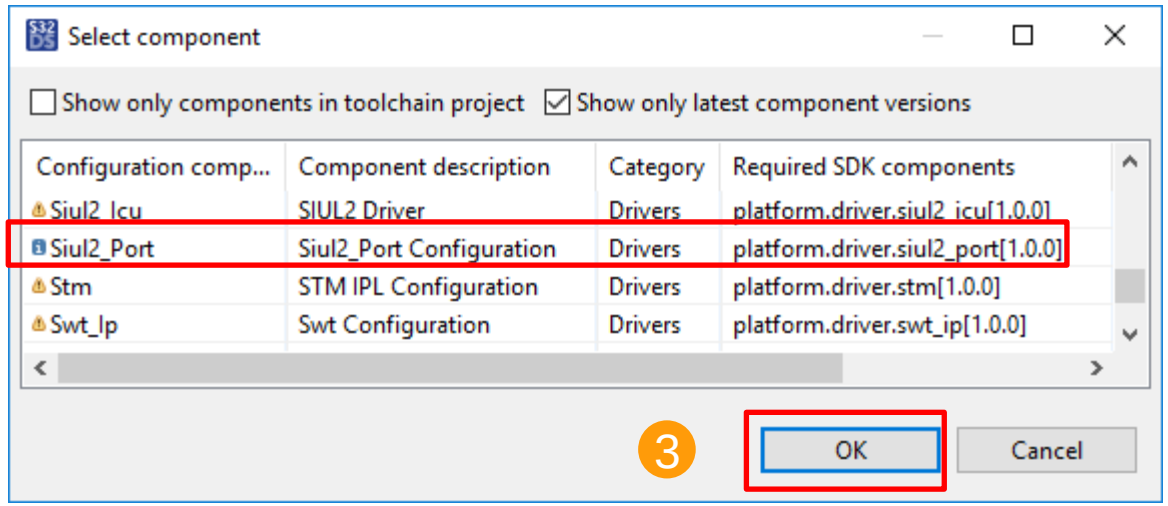
ADD SIUL2_PORT MODULE

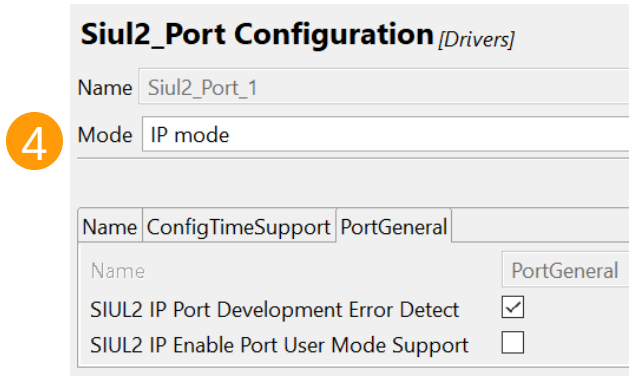
- After configuring all the desired pins. Go to the Peripherals view and add the Siul2_Port driver.



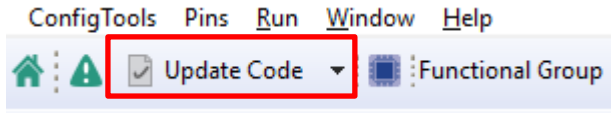




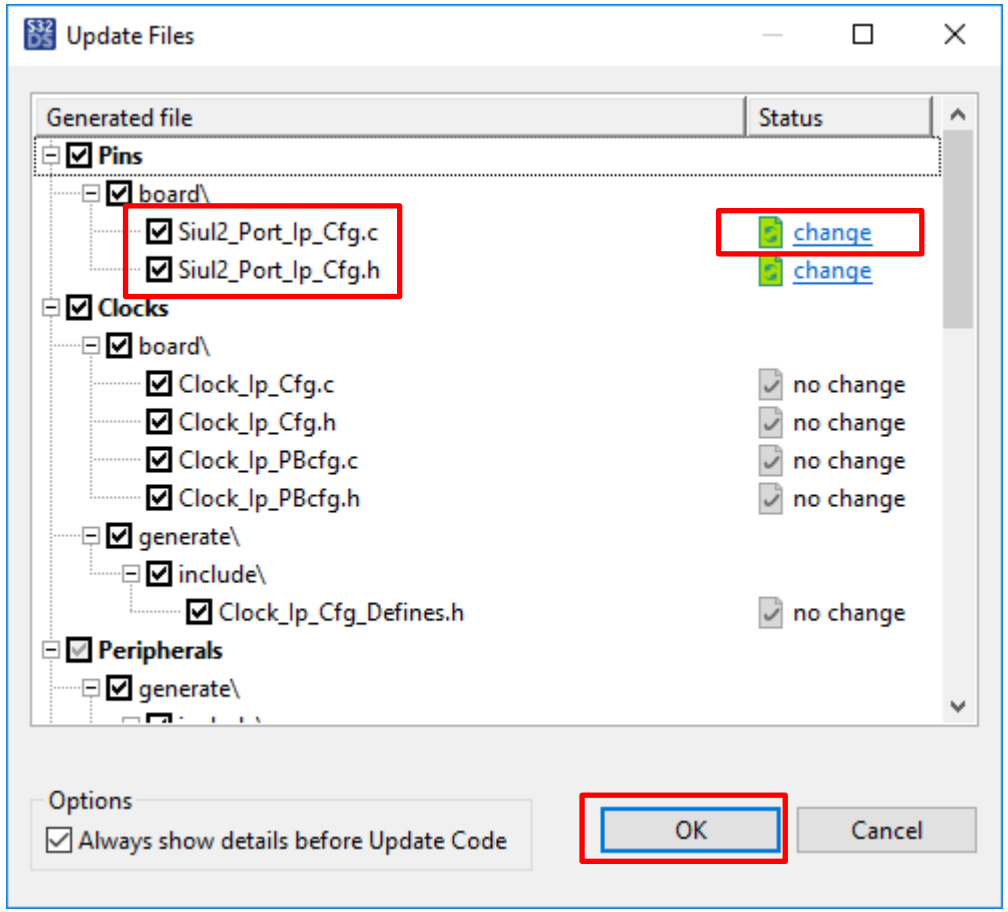




GENERATE CODE FOR PINS CONFIGURATION



Click “Update Code” to generate code for Pins Configuration



Click “change” to view the update of the newly generated code.

Select or unselect the files for applying or not applying the updates.

S32DS CT GENERATED FILES FOR SIUL2_PORT

- The following files are Siul2_Port related driver source codes.
  - RTD/include/Siul2_Port_Ip_Types.h
  - RTD/include/Siul2_Port_Ip.h
  - RTD/src/Siul2_Port_Ip.c

- The following files include pins configuration structures.
  - board/Siul2_Port_Ip_Cfg.h
  - board/Siul2_Port_Ip_Cfg.c

Siul2_PORT
API Functions

SECURE CONNECTIONS
FOR A SMARTER WORLD

"Siul2_Port_Ip.h" must be included in your source code.
Available API Functions for Pins Configure can be found in "Siul2_Port_Ip.h". Some are explained below:

```
Siul2_Port_Ip_PortStatusType Siul2_Port_Ip_Init(uint32 pinCount,
                                                const Siul2_Port_Ip_PinSettingsConfig config[])
```

Initialize the pins with the given structure generated by S32DS configuration tool.

```
void Siul2_Port_Ip_SetPullSel(Siul2_Port_Ip_PortType * const base,
                              uint16 pin,
                              Siul2_Port_Ip_PortPullConfig pullConfig)
```

Enable the pull-up/pull-down settings of the given pin.

```
void Siul2_Port_Ip_SetInputBuffer(Siul2_Port_Ip_PortType * const base,
                                  uint16 pin,
                                  boolean enable,
                                  uint32 inputMuxReg,
                                  Siul2_Port_Ip_PortInputMux inputMux)
```

Configure a pin as input function with given option.

```
void Siul2_Port_Ip_SetOutputBuffer(Siul2_Port_Ip_PortType * const base,
                                   uint16 pin,
                                   boolean enable,
                                   Siul2_Port_Ip_PortMux mux)
```

Configure a pin as output function with given option.

The customer can use the functions included within the Siul2_Port driver. Some examples are shown on the next slide.

SIUL2_PORT EXAMPLE CODE

```
#include "Mcal.h"
#include "Siul2_Port_Ip.h"
```

Include the necessary header files.

```
Siul2_Port_Ip_Init(NUM_OF_CONFIGURED_PINS0,
g_pin_mux_InitConfigArr0);
```

Initialize all pins configurations. “g_pin_mux_InitConfigArr0” is defined in the generated code.

```
Siul2_Port_Ip_SetOutputBuffer(PORTG_L_HALF, 2,
TRUE, PORT_MUX_AS_GPIO);
```

Configure PTG2 as GPIO output.

```
Siul2_Port_Ip_SetInputBuffer(PORTD_H_HALF, 1,
TRUE, 701-512, PORT_INPUT_MUX_ALT2);
```

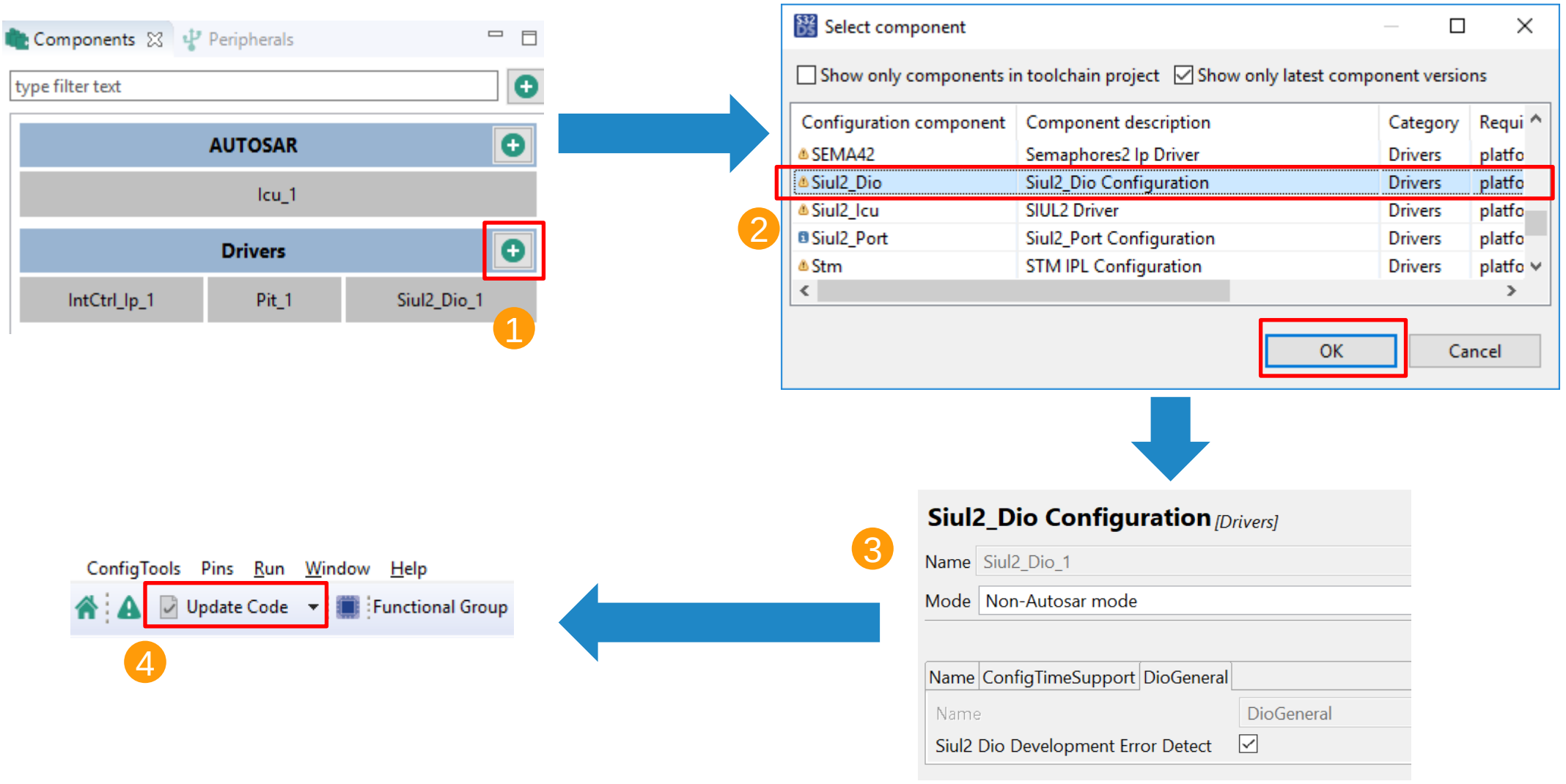
Configure PTD17 as LPUART2 Rx input.

Siul2_Dio
API Functions

SECURE CONNECTIONS
FOR A SMARTER WORLD

ADD SIUL2_DIO MODULE

- The Siul2_Dio driver has the necessary functions for digital input/output pins



S32DS CT GENERATED FILES FOR SIUL2_DIO

- The following files are Siul2_Dio related driver source codes.
  - RTD/include/Siul2_Dio_Ip.h
  - RTD/src/Siul2_Dio_Ip.c

- The following files include pins configuration structures.
  - generate/include/Siul2_Dio_Ip_Cfg.h

"Siul2_Dio_Ip.h" must be included in your source code
Available API Functions for Pins Configure can be found in "Siul2_Dio_Ip.h". Some are explained below:

```
void Siul2_Dio_Ip_WritePin(Siul2_Dio_Ip_GpioType * const base,
                           Siul2_Dio_Ip_PinsChannelType pin,
                           Siul2_Dio_Ip_PinsLevelType value);
```

Write value 0 or 1 to the specified pin.

```
void Siul2_Dio_Ip_WritePins(Siul2_Dio_Ip_GpioType * const base,
                            Siul2_Dio_Ip_PinsChannelType pins)
```

Write a 16-bit value to a 16-pin parallel output port.

SIUL2_DIO API FUNCTIONS

```
void Siul2_Dio_Ip_SetPins(Siul2_Dio_Ip_GpioType * const base,
                          Siul2_Dio_Ip_PinsChannelType pins)
void Siul2_Dio_Ip_ClearPins(Siul2_Dio_Ip_GpioType * const base,
                            Siul2_Dio_Ip_PinsChannelType pins)
void Siul2_Dio_Ip_TogglePins(Siul2_Dio_Ip_GpioType * const base,
                             Siul2_Dio_Ip_PinsChannelType pins)
```

Set, clear or toggle the specified pins within a given GPIO port.

```
Siul2_Dio_Ip_PinsChannelType Siul2_Dio_Ip_ReadPins(const Siul2_Dio_Ip_GpioType *const base)
```

Read the 16-bit value of a 16-pin wide parallel port.

```
Siul2_Dio_Ip_PinsLevelType Siul2_Dio_Ip_ReadPin(const Siul2_Dio_Ip_GpioType * const base,
Siul2_Dio_Ip_PinsChannelType pin)
```

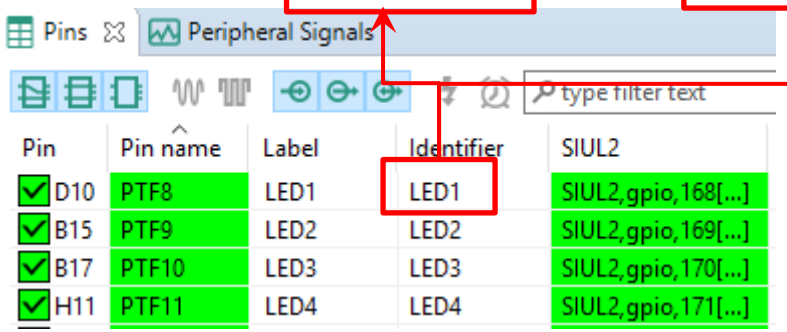
Read the 16-bit parallel port value to get the state of a specified single pin. (Read a PGPDI register)

The customer can use the functions included within the Siul2_Dio driver. Some examples are shown on the next slide.

```
#include "Mcal.h"
#include "Siul2_Port_Ip.h"
#include "Siul2_Dio_Ip.h"
```

Include the necessary header files.

```
Siul2_Dio_Ip_TogglePins(LED1_PORT, (1<<LED1_PIN));
```



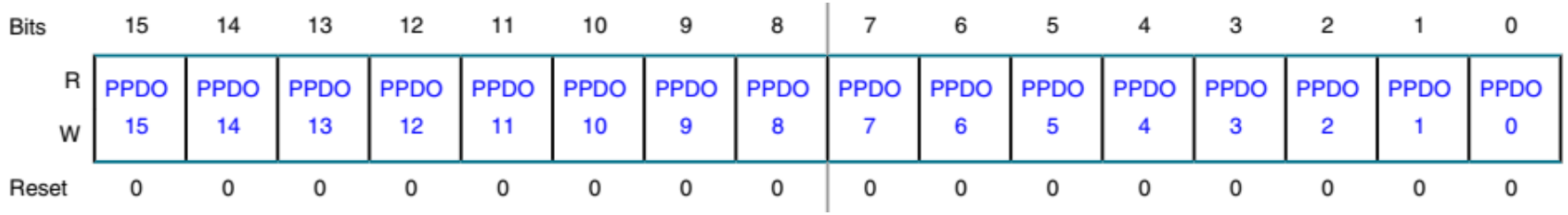
Toggle the pin “LED1” which is defined in the pin's configuration tool.
LED1_PORT and LED1_PIN macro definition is generated in “Siul2_Port_Ip_Cfg.h”

```
Siul2_Dio_Ip_WritePin(LED1_PORT, LED1_PIN, 1);
```

Write PGPDO register to let LED1_PIN output high.

```
Siul2_Dio_Ip_ReadPin(USER_BTN1_PORT, USER_BTN1_PIN);
```

Read PGPDI register to return USER_BTN1_PIN input buffer status.

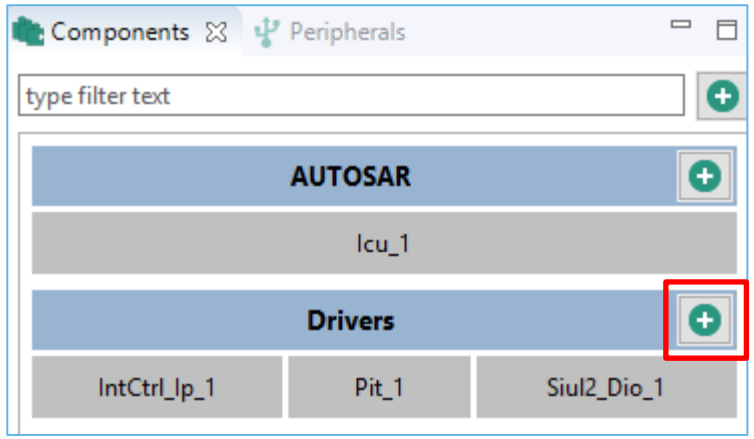


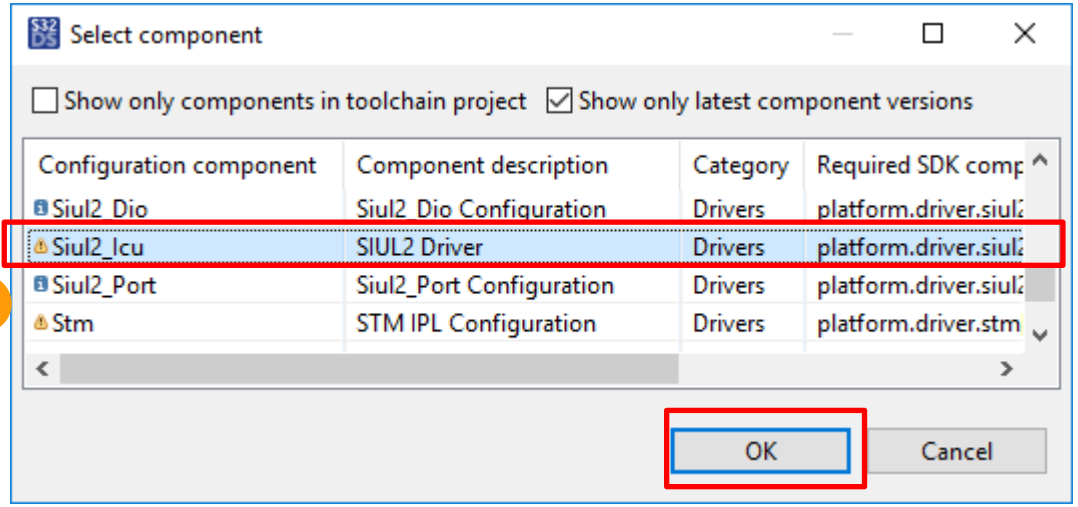
Parallel GPDO register

Siul2_Icu Configuration

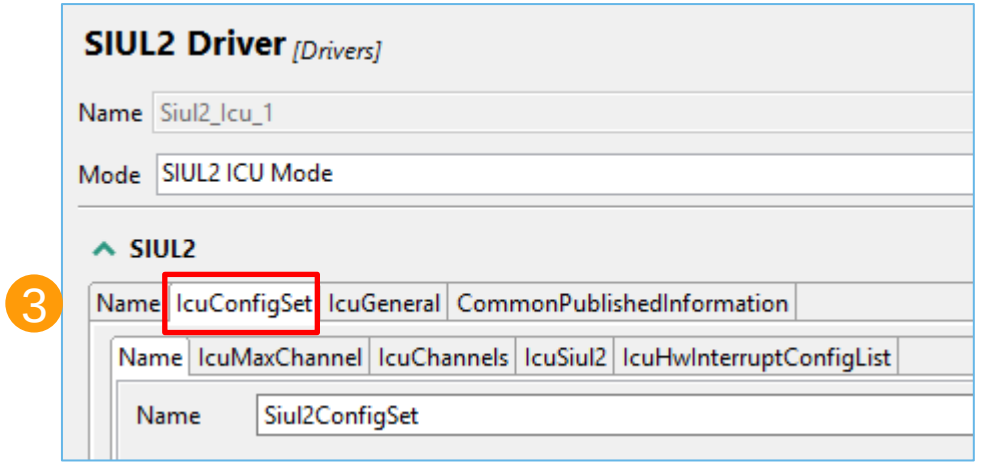
SECURE CONNECTIONS
FOR A SMARTER WORLD

ADD SIUL2_ICU MODULE

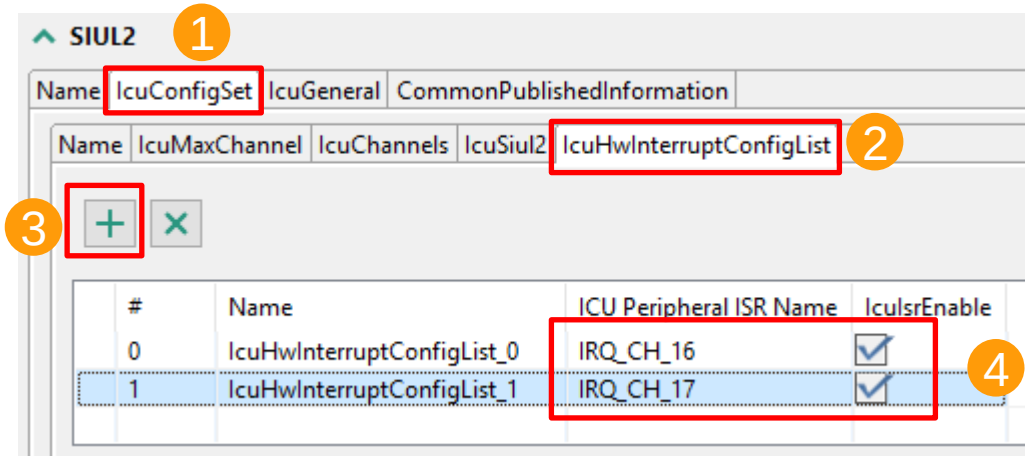




- The Siul2_Icu driver has the necessary functions to capture rising/falling edges and configure their interrupts.



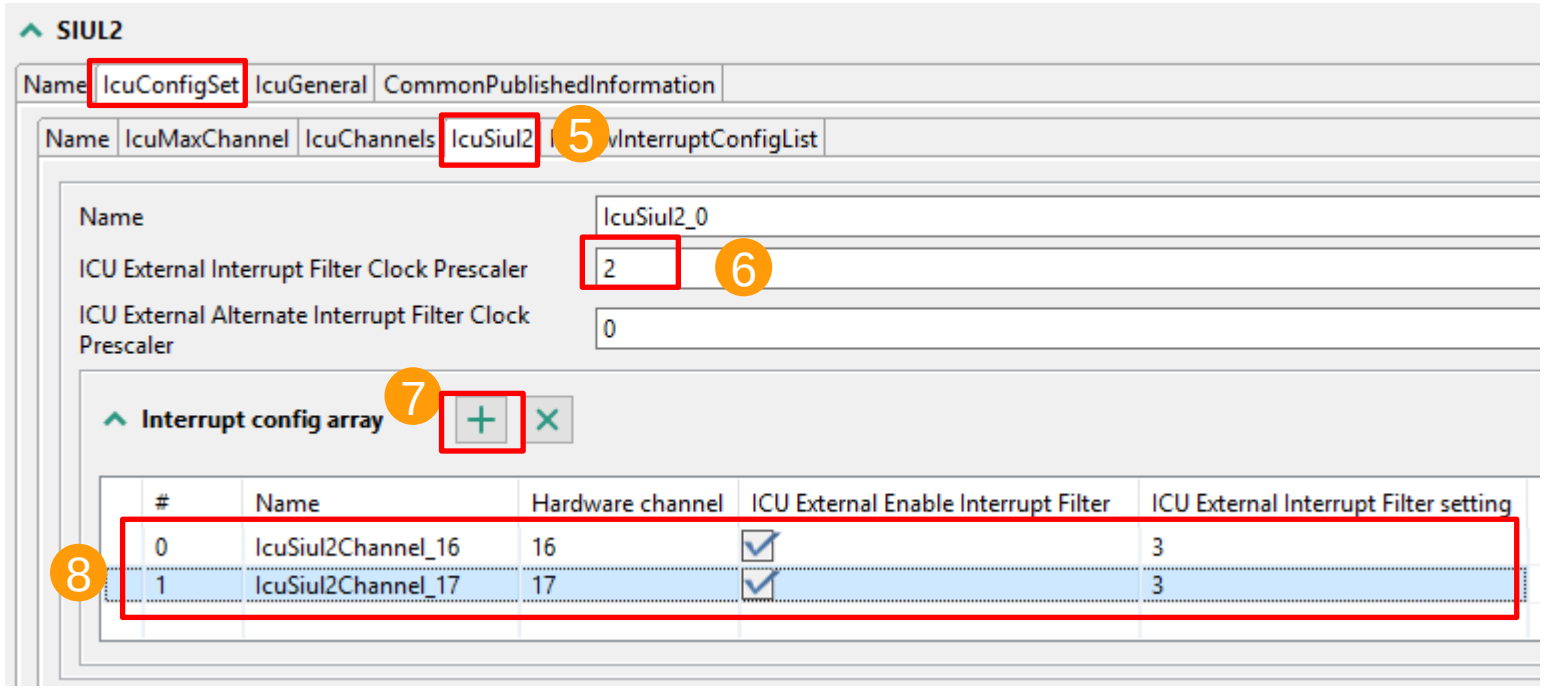
Add the desired Siul2 IRQ channel for detecting rising/falling edges.
Enable this IRQ channel interrupt



Configure each IRQ channel glitch filter:

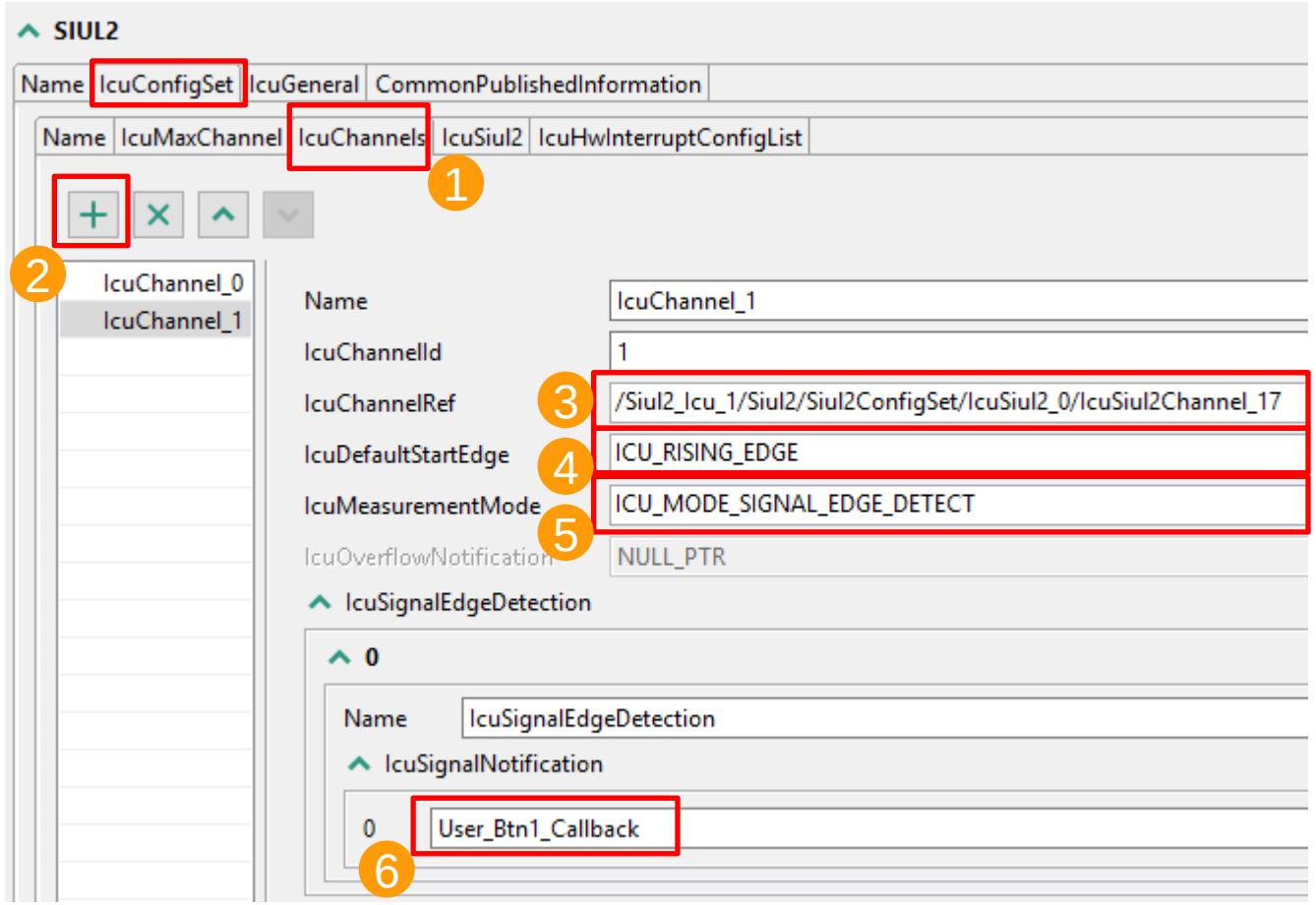
- Set the filter clock prescaler
- Enable the glitch filter
- Set the filter counter

Interrupt Filter setting => IFMCRn registers
Interrupt Filter Clock Prescaler => IFCPR register
(Filter clock source is FIRC 48MHz)



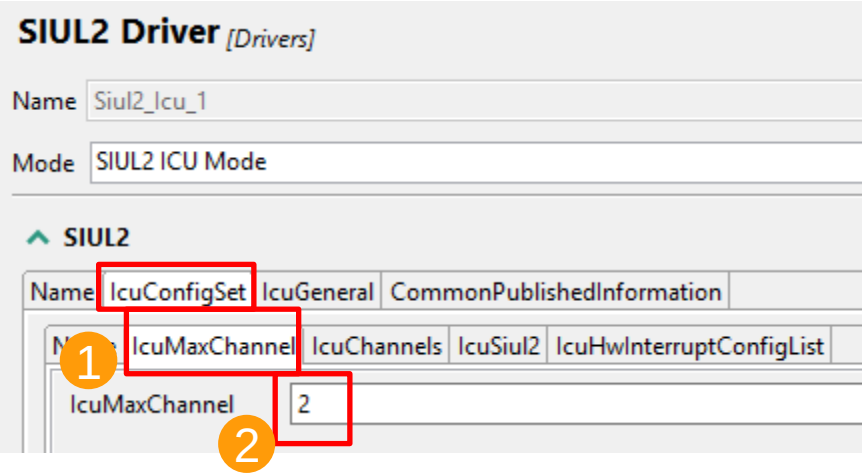
Add ICU channel and configure the property:

1. Select the tab “IcuChannels”
2. Clock “plus” button to add an icu channel
3. IcuChannelRef – must select from the existing enabled IRQ channels.
4. Select rising edge, falling edge, or both edges.
5. IcuMeasurementMode - must be “signal edge detect” mode.
6. Icu SignalNotification – set the callback function name which will be called when desired edge detected.

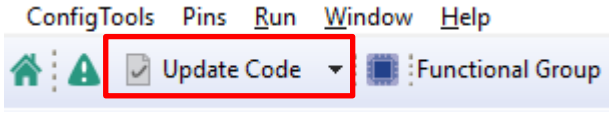


Make the same process for the IcuChannel_0 with the Channel_16

Set the number of used Siul2_Icu channels.



Click “Update Code” to generate code for Pins Configuration



S32DS CT GENERATED FILES FOR SIUL2_ICU

- The following files are Siul2_Icu related driver source codes.
  - RTD/include/Siul2_Icu_Ip.h
  - RTD/include/Siul2_Icu_Ip_Irq.h
  - RTD/include/Siul2_Icu_Ip_Types.h
  - RTD/src/Siul2_Icu_Ip.c
  - RTD/src/Siul2_Icu_Ip_Irq.c

- The following files include Siul2_Icu initialization structures.
  - generate/include/Siul2_Icu_Ip_Cfg.h
  - generate/include/Siul2_Icu_Ip_Defines.h
  - generate/include/Siul2_Icu_Ip_SA_BOARD_InitPeripherals_PBcfg.h
  - generate/src/Siul2_Icu_Ip_SA_BOARD_InitPeripherals_PBcfg.c

Siul2_Icu
API Functions

SECURE CONNECTIONS
FOR A SMARTER WORLD

"Siul2_Icu_Ip.h" must be included in your source code
Available API Functions for Pins Configure can be found in "Siul2_Icu_Ip.h". Some are explained below:

```
Siul2_Icu_Ip_StatusType Siul2_Icu_Ip_Init(uint8 instance, const Siul2_Icu_Ip_ConfigType*
userConfig);
```

Initialize all Siul2 IRQ channels in the configuration tool.
The initialization structure "*pConfig" contains all used channels' configurations.

```
void Siul2_Icu_Ip_SetActivationCondition(uint8 instance, uint8 hwChannel,
Siul2_Icu_Ip_EdgeType edge)
```

Set the edge type to detect.

```
void Siul2_Icu_Ip_EnableInterrupt(uint8 instance, uint8 hwChannel);
```

Enable the interrupt for the specified SIUL2 IRQ channel.

The customer can use the functions included within the Siul2_Icu driver. Some examples are shown on the next slide.

SIUL2_ICU EXAMPLE CODE

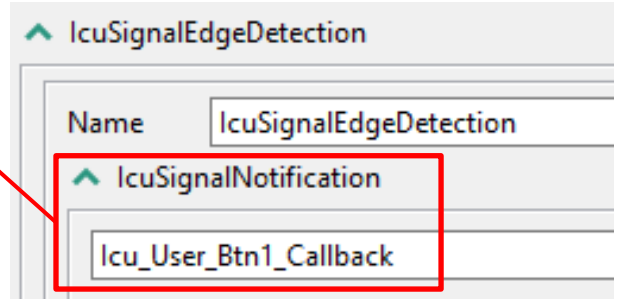
```
#include "Mcal.h"
#include "Siul2_Port_Ip.h"
#include "Siul2_Dio_Ip.h"
#include "Clock_Ip.h"
#include "Intctrl_Ip.h"
#include "Siul2_Icu_Ip.h"

ISR(SIUL2_EXT_IRQ_16_23_ISR);
```

Include the necessary header files. If enabling an interrupt in the Siul2_Icu driver, the **IntCtrl driver** should be added in the Peripherals view as well.

SIUL2_EXT_IRQ_16_23_ISR is the interrupt handler defined in RTD driver.
It clears the SIUL2 interrupts flags and calls the callback functions for each channel

```
void Icu_User_Btn1_Callback(void){
  Siul2_Dio_Ip_TogglePins(LED1_PORT, (1<<LED1_PIN));
  return;
}

void Icu_User_Btn2_Callback(void){
  Siul2_Dio_Ip_TogglePins(LED2_PORT, (1<<LED2_PIN));
  return;
}
```

Notification function for each Siul2 IRQ channel should be defined as the same function name in Siul2_Icu configuration.



NOTE: To avoid errors, write the same name for the callback in the ICU controller and the application.

SIUL2_ICU EXAMPLE CODE (CONTINUED)

```
/* SIUL2 IRQ configuration */
Siul2_Icu_Ip_Init(SIUL2_ICU_IP_INSTANCE,
&Siul2_Icu_Ip_0_Config_PB_BOARD_InitPeripherals);

/* Enable SIUL2_EIRQ16 */
Siul2_Icu_Ip_EnableInterrupt (SIUL2_ICU_IP_INSTANCE,
(*Siul2_Icu_Ip_0_Config_PB_BOARD_InitPeripherals.pChannelsConfig)[0].hwChannel);

/* Enable SIUL2_EIRQ17 */
Siul2_Icu_Ip_EnableInterrupt (SIUL2_ICU_IP_INSTANCE,
(*Siul2_Icu_Ip_0_Config_PB_BOARD_InitPeripherals.pChannelsConfig)[1].hwChannel);

/* Install IRQ handler for SIUL2 EIRQ16~23 */
IntCtrl_Ip_InstallHandler(SIUL_2_IRQn, SIUL2_EXT_IRQ_16_23_ISR, NULL_PTR);

/* Enable SIUL2 EIRQ16~13 in NVIC */
IntCtrl_Ip_EnableIrq(SIUL_2_IRQn);
```

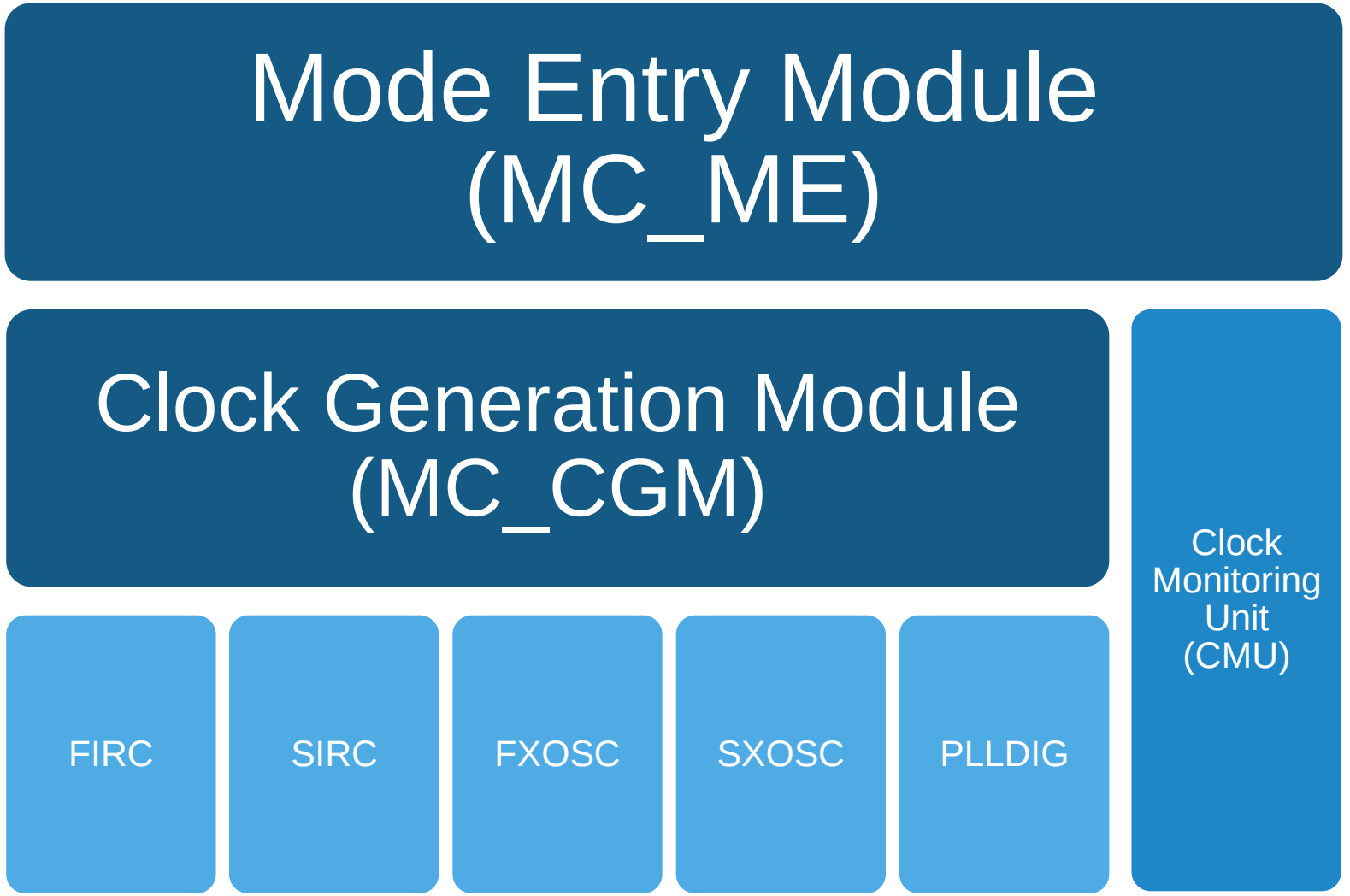
Initialize all Siul2_Icu channels. Siul2 IRQ is still disabled here. The initialization structure is generated by CT.

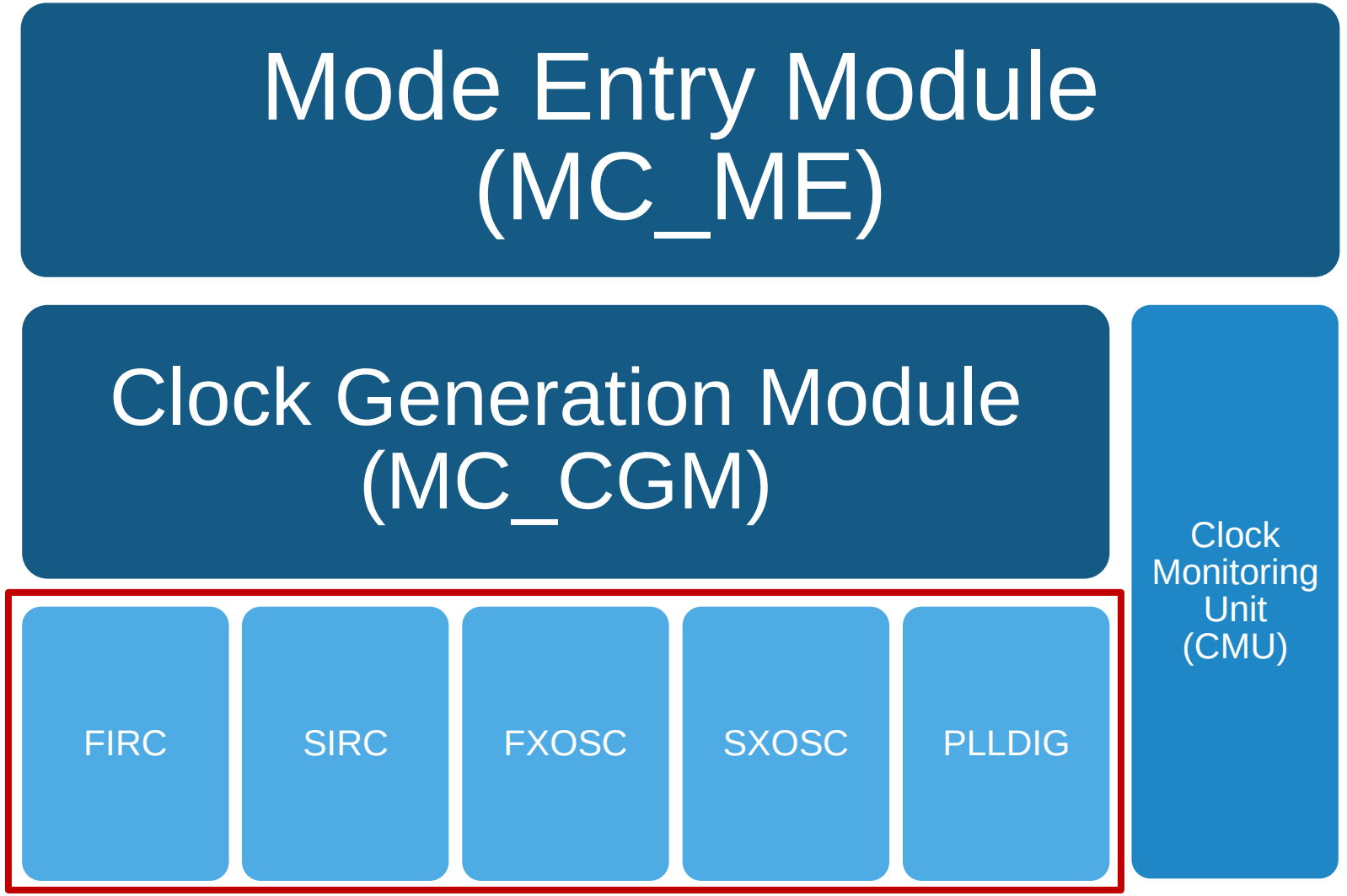
Enable Siul2 IRQ interrupt for the specified channels.

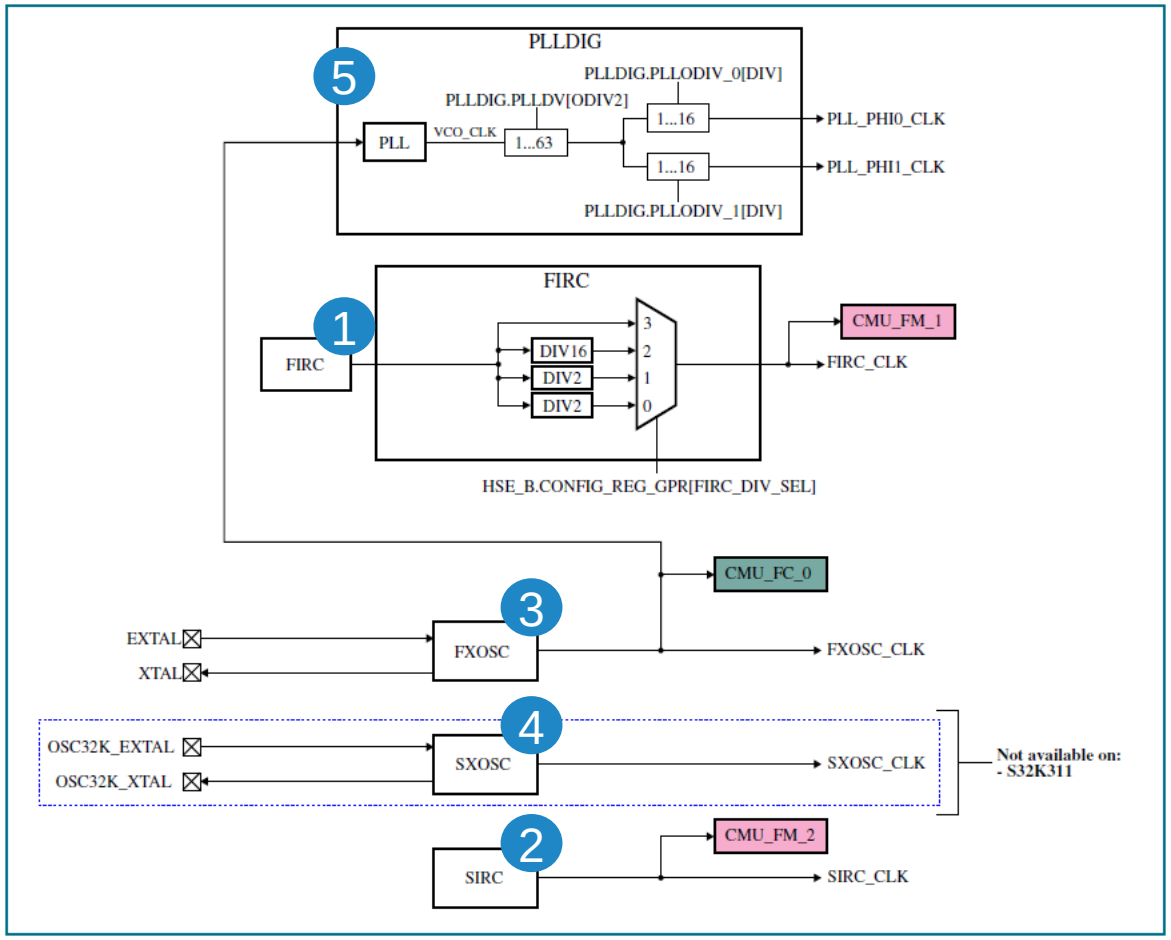
Install IRQ handler and enable corresponding interrupt in NVIC.

Clocking Overview

SECURE CONNECTIONS
FOR A SMARTER WORLD


Mode Entry Module (MC_ME)
Clock Generation Module (MC_CGM)
FIRC
SIRC
FXOSC
SXOSC
PLLDIG
Clock Monitoring Unit (CMU)


Mode Entry Module
(MC_ME)
Clock Generation Module
(MC_CGM)
FIRC
SIRC
FXOSC
SXOSC
PLLDIG
Clock
Monitoring
Unit
(CMU)



FIRC reference
FXOSC reference

CLOCK SOURCES

1. Fast Internal RC oscillator (FIRC) – Default.
2. Slow Internal RC oscillator (SIRC)
3. Fast External Crystal oscillator (FXOSC)
4. Slow External Crystal oscillator (SXOSC)
5. Phase-Locked Loop (PLL)

CLOCK MONITORING UNITS

1. Frequency Check (CMU_FC).
2. Frequency Meter (CMU_FM).

Clock	Uses	Default	STANDBY
FIRC (48 MHz)	✓ Boot clock (on power up & after any reset event and standby exit). ✓ SIUL2 filter clock. ✓ Safe clock for FCCU and FOSU.	ON	Optional
SIRC (32 KHz)	✓ SWT & POR_WDG clock source. ✓ Safe clock along with FIRC.	ON	Optional
SXOSC (32.768 KHz)	✓ SXOSC is not affected by functional reset. ✓ RTC source for operation across functional reset.	OFF	Optional
FXOSC (8-40 MHz)	✓ Reference clock source for PLL. ✓ Communication modules (FlexCAN, EMAC, QuadSPI, and so on).	OFF	Optional
PLL (48-320 MHz)	✓ Optional system clock source (in high performance applications). ✓ Communication modules (FlexCAN, EMAC, QuadSPI, and so on).	OFF	Not supported

FIRC & SIRC

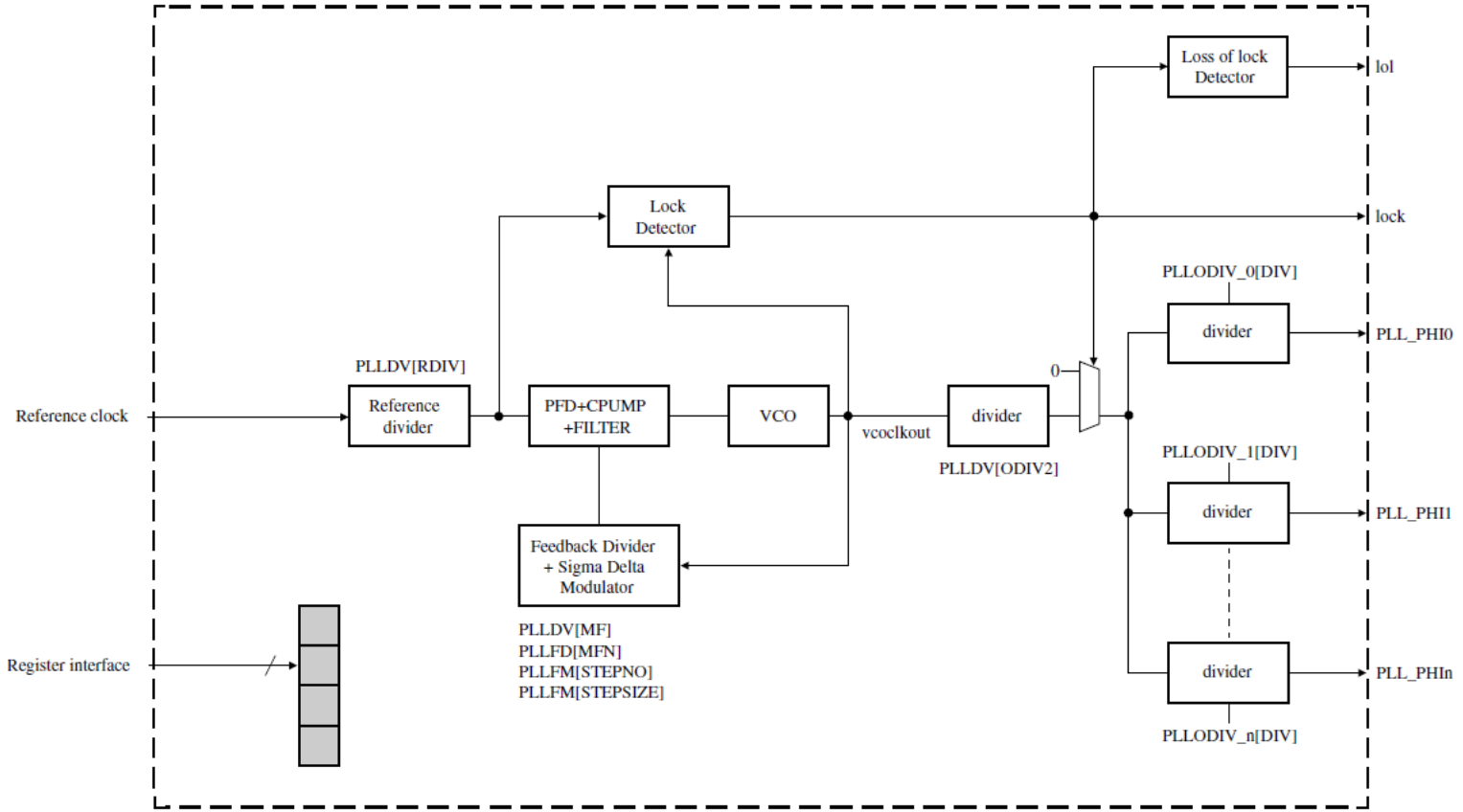
- Always enabled in RUN mode.
- Status registers provide the current operating state
  a. ON and stable
  b. OFF or not stable
- Safety clocks for safety relevant applications for modules responsible to detect and react to incorrect chip operation.

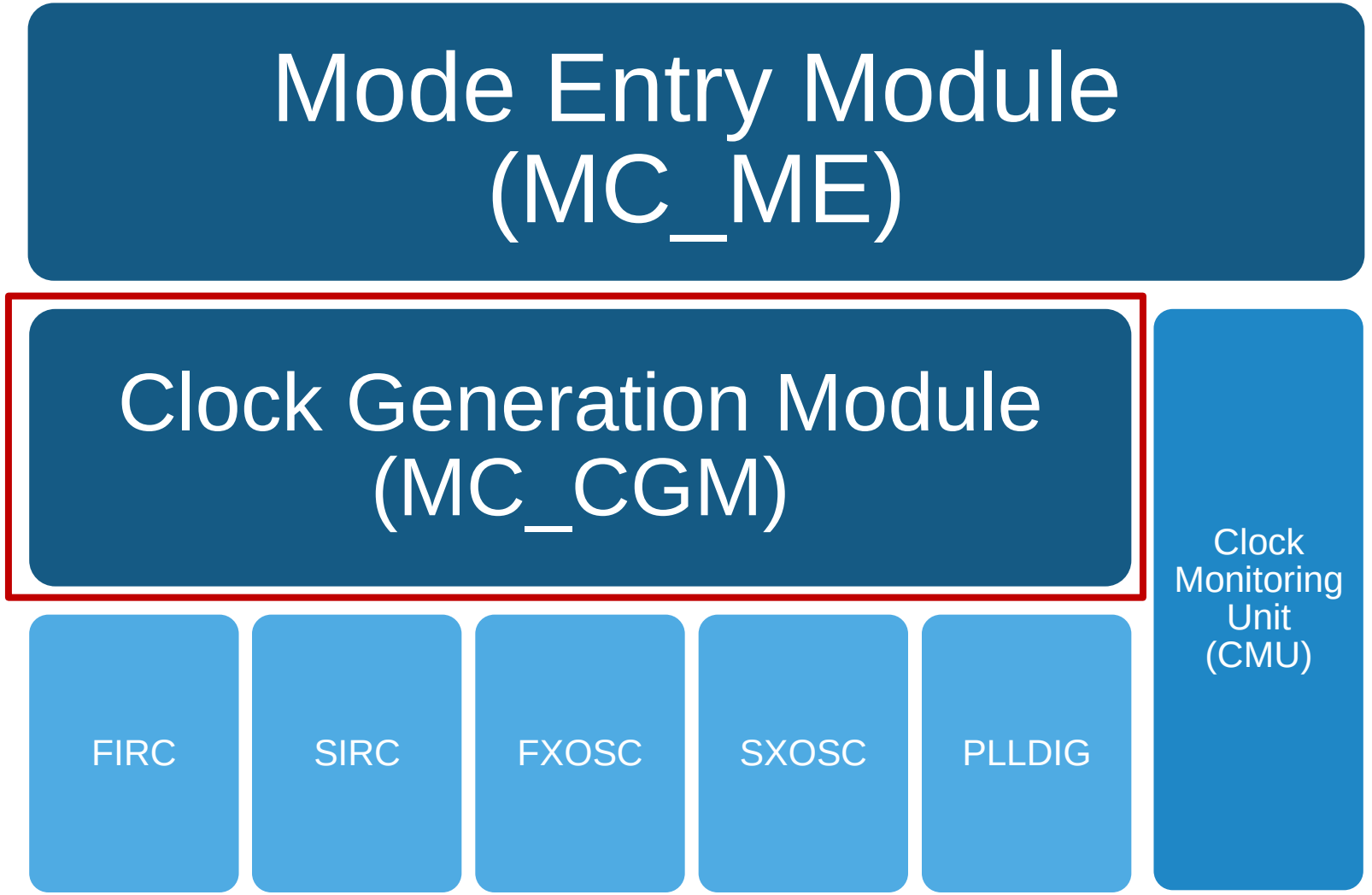
FXOSC & SXOSC

- Clock gating by the MC_ME.
- Status register showing current state
  a. ON and providing a stable clock.
  b. OFF or its output clock is not stable.
- Support
  a. Crystal input mode
  b. Bypass mode
- Configurable stabilization counter value.
- **FXOSC**: Configurable amplifier transconductance.

User interface to control the PLL system.

- Programmable frequency modulation.
- Multiple integer dividers on PLL outputs.
- Lock detection circuitry reports when the PLL achieves frequency lock.
- Continuous monitoring of lock status to report Loss of Lock (LOL) condition.

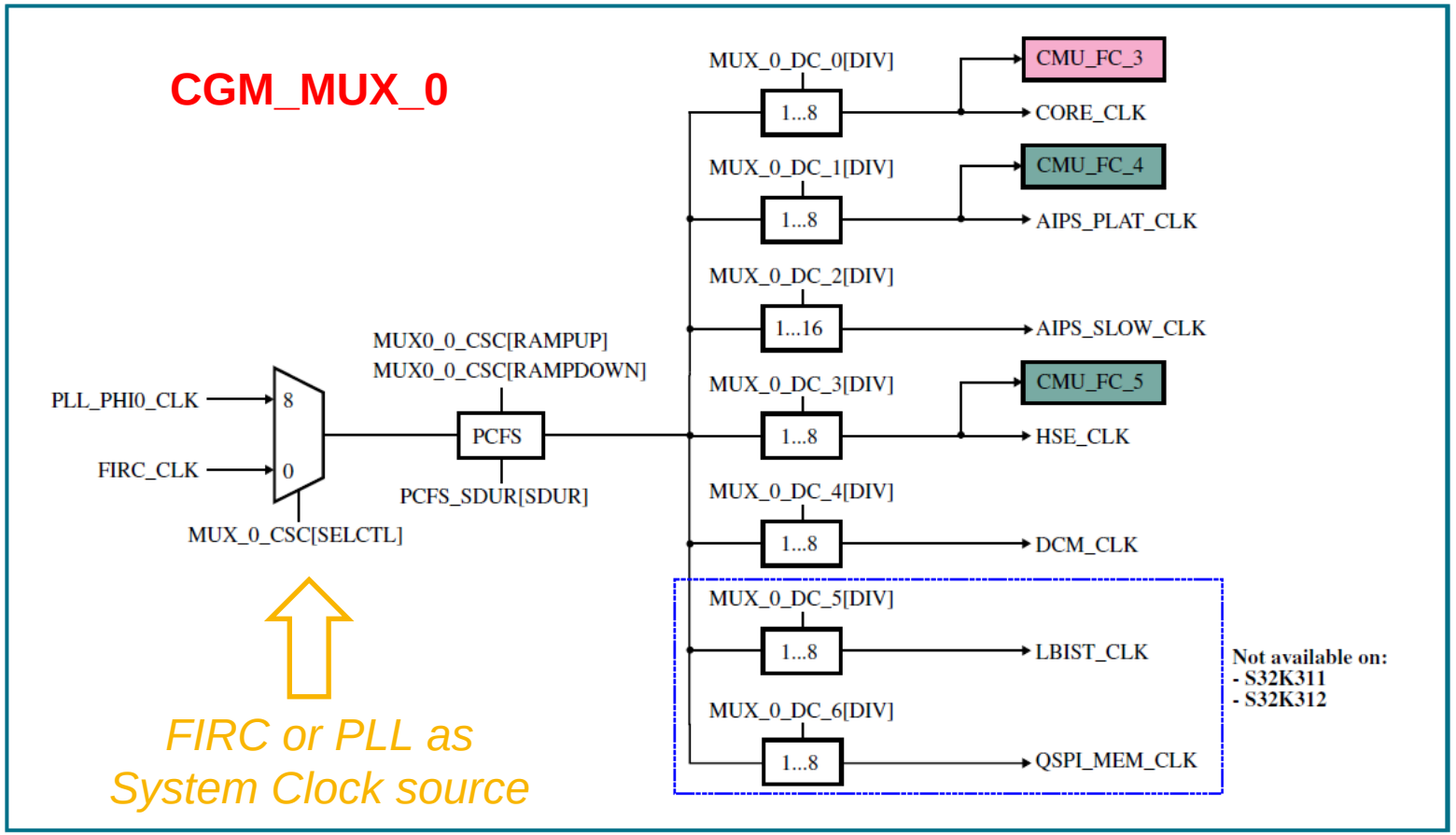



Mode Entry Module (MC_ME)
Clock Generation Module (MC_CGM)
FIRC
SIRC
FXOSC
SXOSC
PLLDIG
Clock Monitoring Unit (CMU)

Generates reference clocks for all the chip blocks.

Features:

- SW-configurable multiplexers and dividers for on-chip resources.
- Progressive Clock Frequency Switching (PCFS) to minimize the impact of a sudden power consumption change when switching clock sources.
- Common trigger to update all clock dividers within a MUX at the same time
- Glitch-less clock transitions when changing the clock selection at clock multiplexers.

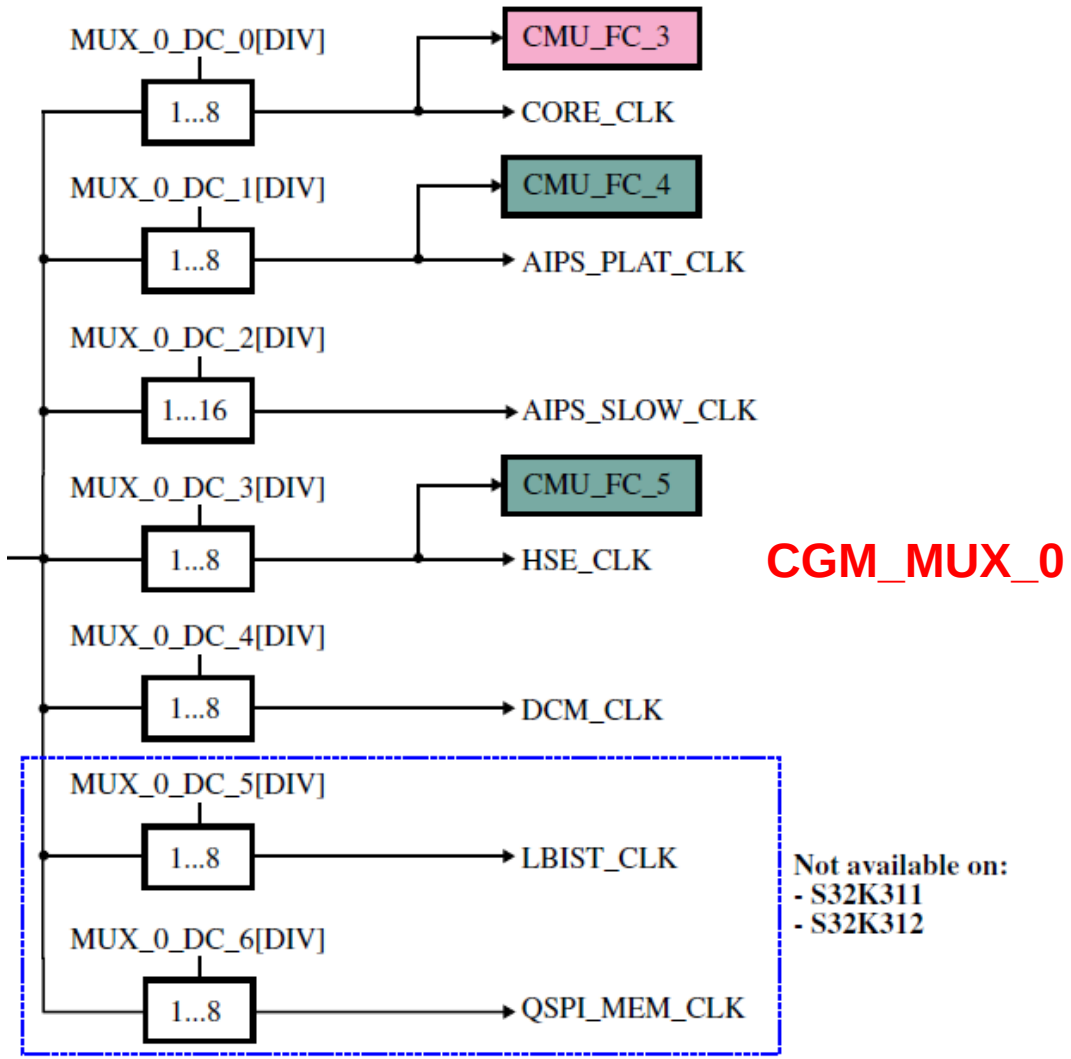


There are 12 clock multiplexers on S32K344: clock **MUX 0 ~ MUX 11**

FIRC reference

FXOSC reference

SYSTEM CLOCKING (CGM_MUX_0)

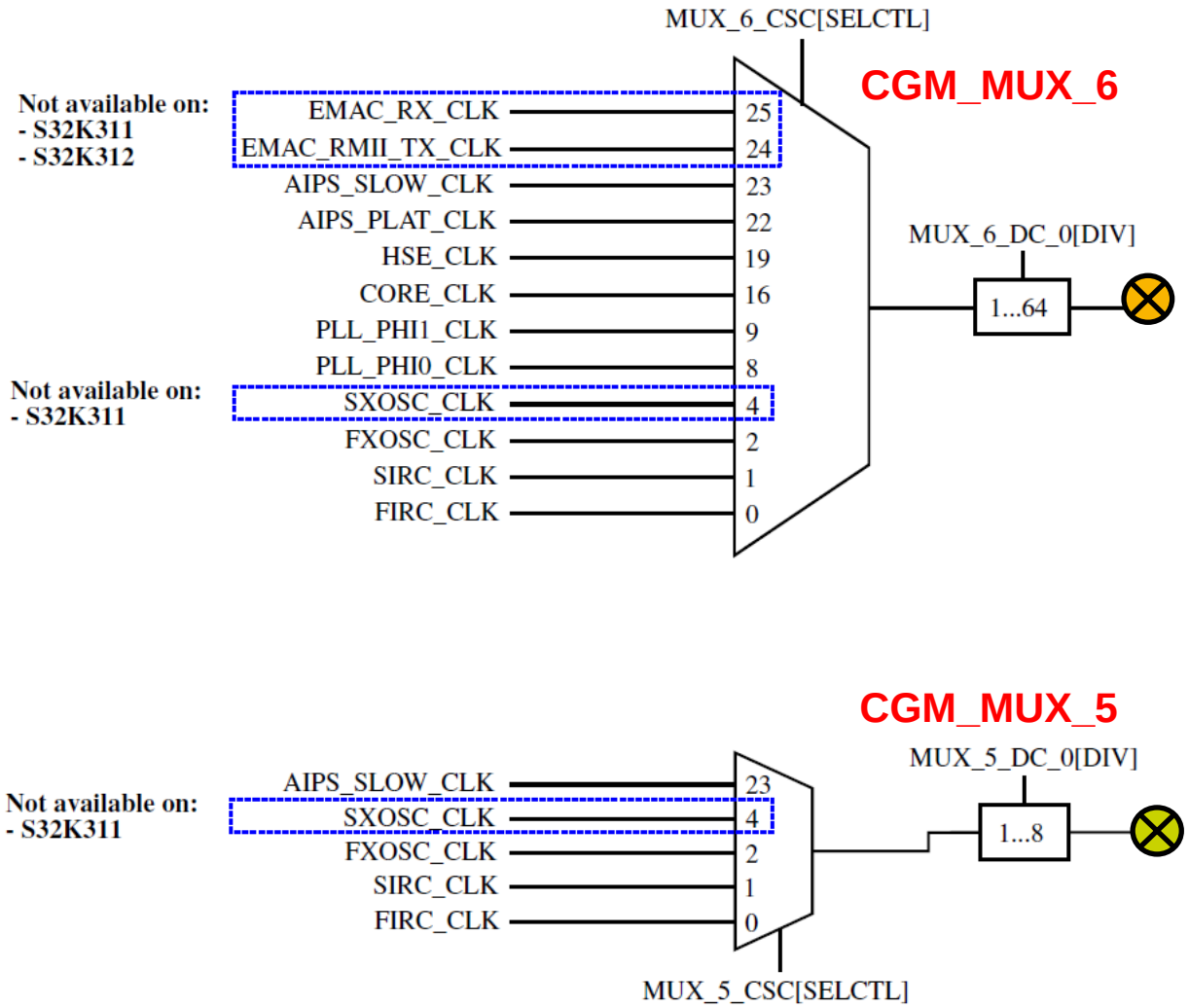


CGM_MUX_0

Node	Clock Source for…
CORE_CLK	Application cores, AXBS, SRAM, PFLASH and AIPS0 (high speed peripherals).
AIPS_PLAT_CLK	Medium speed peripherals (e.g. FlexCAN, LPUART, FlexIO).
AIPS_SLOW_CLK	Slow speed peripherals (e.g. LPI2C, LPSPI1-5, SIUL2).
HSE_CLK	HSE CM0+
DCM_CLK	Device Configuration Module
LBIST_CLK	STCU2 for LBIST operations.
QSPI_MEM_CLK	QSPI_RAM and QSPI_TX blocks.

All clock dividers in MUX_0 are enabled by default after reset.

CLKOUT OVERVIEW



The chip supports two CLKOUT pins for allowing the view of some internal clocks.

- **CLKOUT_RUN (CGM_MUX_6)**

Available during Run mode only.

PTB5, PTD10, PTD14

- **CLKOUT_STANDBY (CGM_MUX_5)**

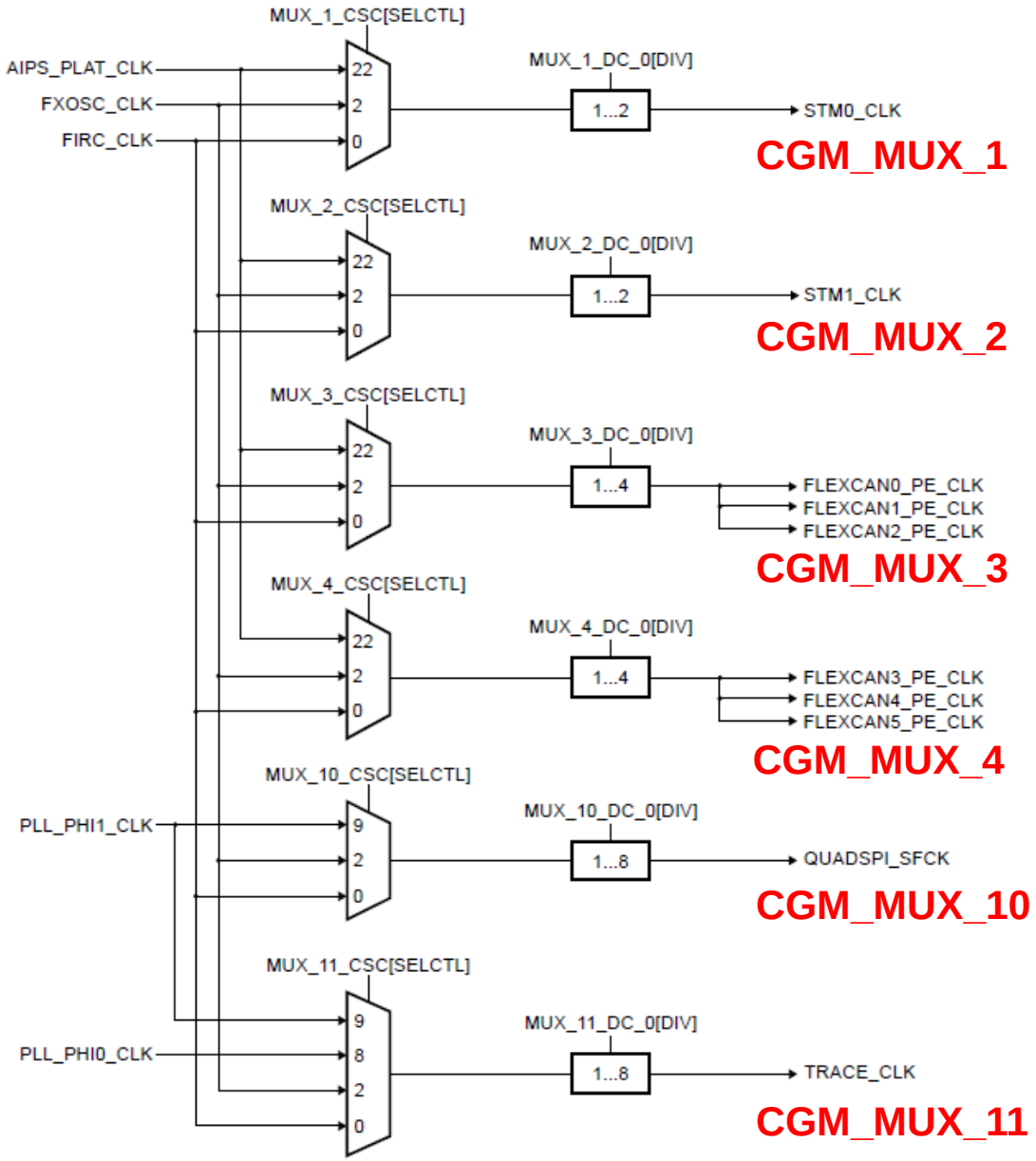
Available during both Run and Standby modes.

(Its registers are latched while the chip enters Standby mode and are reset in Standby exit sequence. Therefore, the CLKOUT_STANDBY signal needs to be reconfigured on Standby mode exit).

PTA12, PTE10

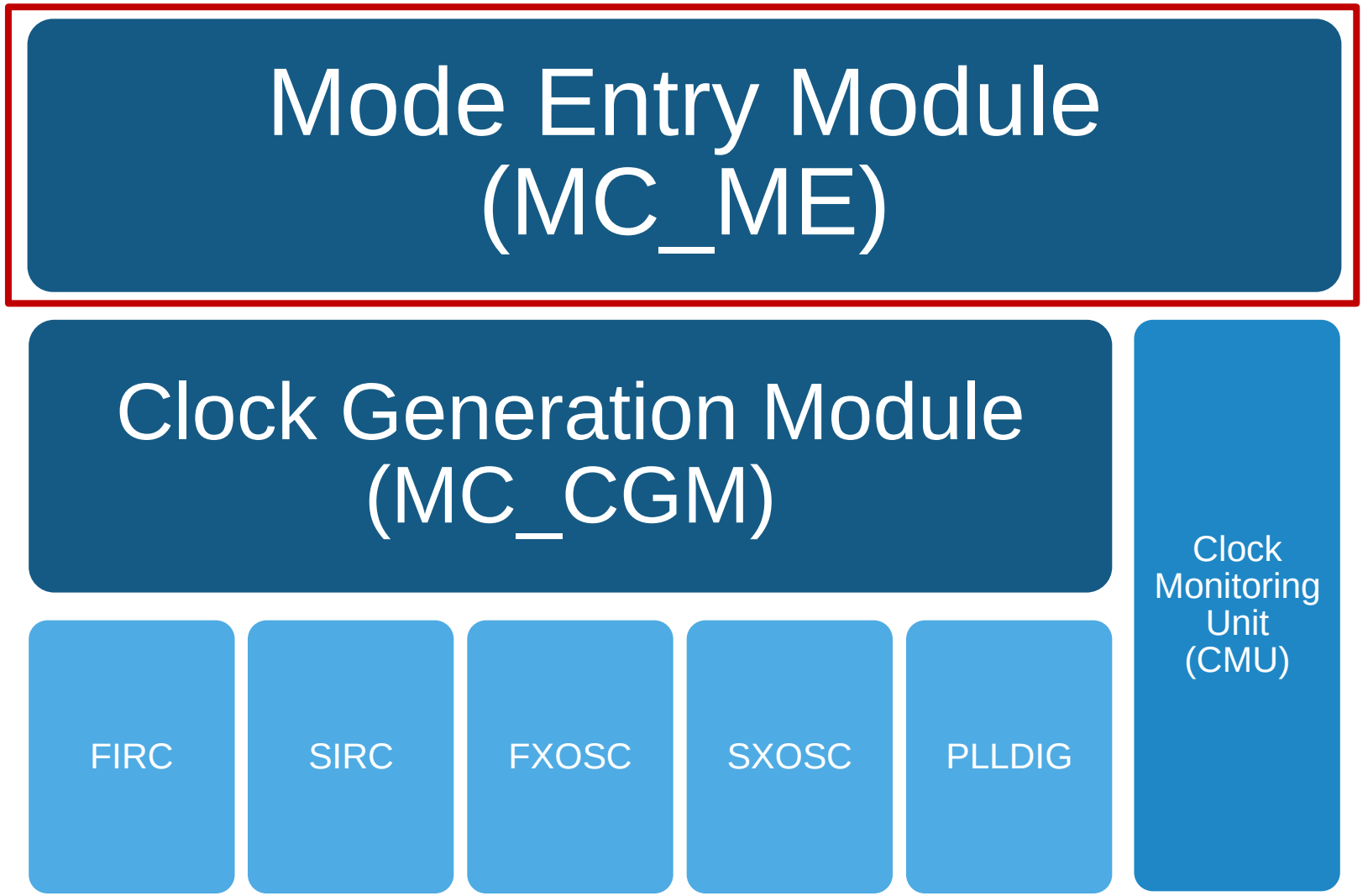
Clock dividers in MUX_5 and MUX_6 are enabled by default.

OTHER CLOCKS…



Node	Clock source for…
STMx_CLK	Both the operating and register interface clocks for the System Timer Modules (STMx).
FLEXCANx_PE_CLK	FlexCANx protocol engine clocks.
QUADSPI_SFCK	QuadSPI serial flash clock.
TRACE_CLK	Trace subsystem blocks.

Clock dividers in these clock MUX are disabled by default.


Mode Entry Module (MC_ME)
Clock Generation Module (MC_CGM)
FIRC
SIRC
FXOSC
SXOSC
PLLDIG
Clock Monitoring Unit (CMU)

Provides core and peripheral clock gating.

Each IP is assigned to a

- Partition (**PRTN**) and a
- Collection of Functional Blocks (**COFB**).

And has its own clock enabler (**CLKEN**).

Table 185. MC_ME Partition Peripheral mapping and Clock Control

AIPS	Peripheral description	MC_ME COFB control register	MC_ME peripheral control register	MC_ME peripheral slot no. in partition	MC_ME COFB present	MC_ME CLKEN present	MC_ME Default CLKIN	On Platform	IPS SLOT NUMBER
0	Reserved	PRTN0_COFB0_CLKEN[REQ0]	0	0	0	0	0	YES	0
0	Reserved	PRTN0_COFB0_CLKEN[REQ1]	1	1	0	0	0	YES	1
0	Reserved	PRTN0_COFB0_CLKEN[REQ2]	2	2	0	0	0	YES	2
0	Reserved	PRTN0_COFB0_CLKEN[REQ3]	3	3	0	0	0	YES	3
0	Reserved	PRTN0_COFB0_CLKEN[REQ4]	4	4	0	0	0	YES	4
0	Reserved	PRTN0_COFB0_CLKEN[REQ5]	5	5	0	0	0	YES	5
0	Reserved	PRTN0_COFB0_CLKEN[REQ6]	6	6	0	0	0	YES	6
0	Reserved	PRTN0_COFB0_CLKEN[REQ7]	7	7	0	0	0	YES	7

S32K3xx_RM

The **MC_ME** module supports a mechanism for starting the pre-configured hardware processes, through a simple key-writing sequence.

Before using a peripheral, turn on its clock.

Step	Description	Register
0	Consider the previous peripheral requirements/sequence.	
1	Program the corresponding clock enabler.	MC_ME . PRTNx_COFBy_CLKEN [**REQz**]
2	Program the corresponding process update.	MC_ME . PRTNx_PUPD [**PCUD**]
3	Enter the key, then the inverted key.	MC_ME . CTL_KEY
4	Check if the HW process has finished.	MC_ME . PRTNx_PUPD [**PCUD**]

SW EXAMPLE: ENABLING PIT0'S CLOCK

0	Programmable Interrupt Timer 0	PRTN0_COFB1_CLKEN[REQ44]	44	44	1	1	1	NO	12
0	Programmable Interrupt Timer 1	PRTN0_COFB1_CLKEN[REQ45]	45	45	1	1	0	NO	13

S32K3xx_RM

```
// Enabling clock for the PIT0 (REQ44 in COFB1 of PRTN0)
MC_ME->PRTN0_COFB1_CLKEN |= MC_ME_PRTN0_COFB1_CLKEN_REQ44_MASK;
MC_ME->PRTN0_PUPD        |= MC_ME_PRTN0_PUPD_PCUD_MASK;
MC_ME->CTL_KEY = 0x5AF0; // Enter key
MC_ME->CTL_KEY = 0xA50F; // Enter inverted key

// Wait the HW process to finish
while(MC_ME->PRTN0_PUPD & MC_ME_PRTN0_PUPD_PCUD_MASK){}
```

It is possible to enable/disable multiple peripherals/cores in the same HW process.

The application core clocks are gated by individual MC_ME core clock enable bits.
For HSE core, there is no clock control.

Step	Description	Register
0	Consider the previous core requirements/sequence.	
1	(Init) Write the application core boot address	MC_ME . PRTN0_COREx_ADDR
	(Shut down) Read the application core WFI status	MC_ME . PRTN0_COREx_STAT[**WFI**]
2	Program the corresponding core clock enabler.	MC_ME . PRTN0_COREx_PCONF [**CCE**]
3	Program the corresponding process update.	MC_ME . PTRN0_COREx_PUPD [**CCUPD**]
4	Enter the key, then the inverted key.	MC_ME . CTL_KEY
5	Check if the HW process has finished.	MC_ME . PTRN0_COREx_PUPD [**CCUPD**]

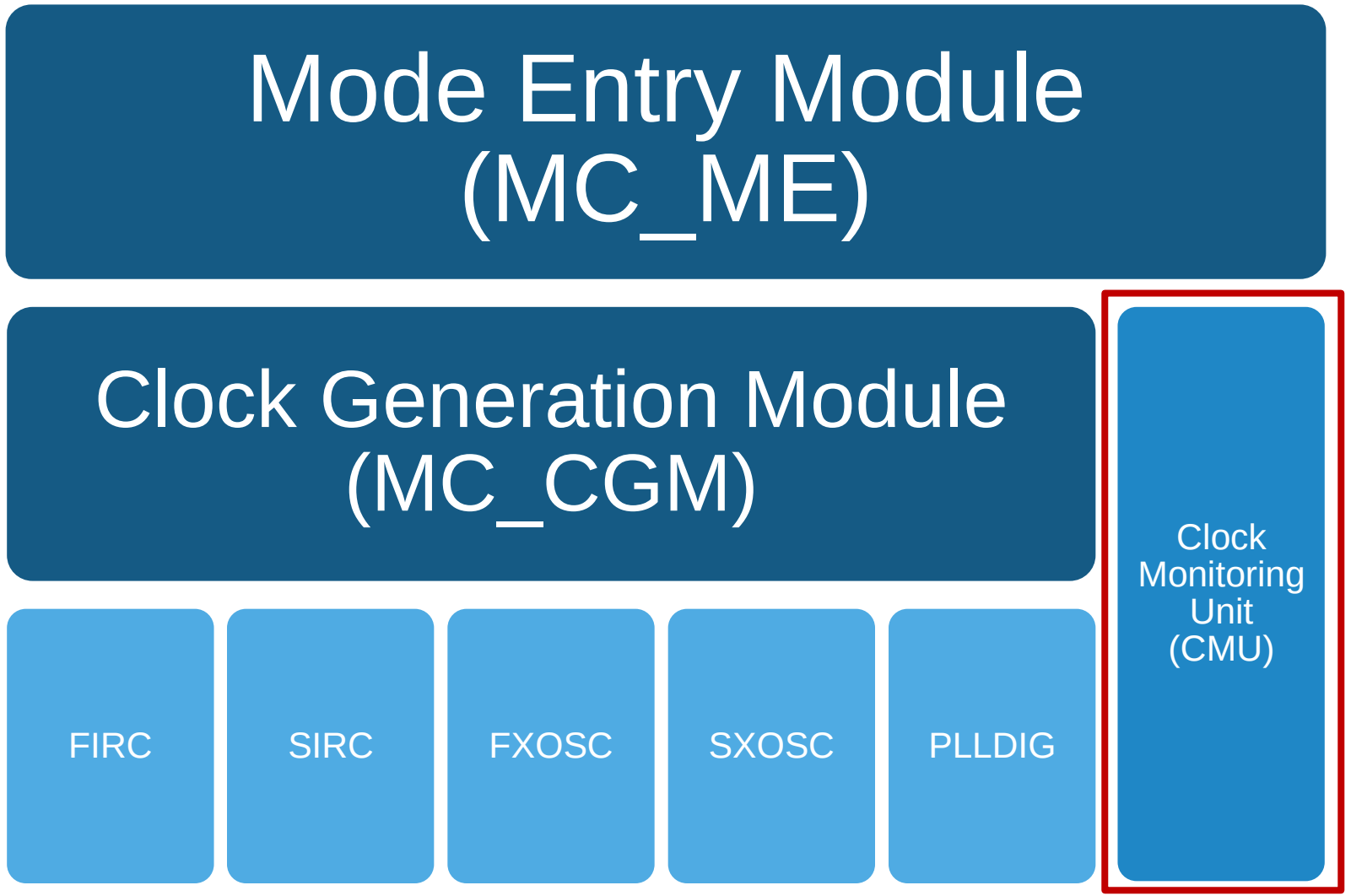
SW EXAMPLE: DISABLING CM7_1'S CLOCK

Serial No.	Core	MC_ME partition	MC_ME Clock Enable Register Bit
1	CM7_0 (Cortex M7 0)	0	MC_ME.PRTN0_CORE0_PCONF[CCE]
2	CM7_1 (Cortex M7 1)	0	MC_ME.PRTN0_CORE1_PCONF[CCE]

S32K3xx_RM

```
// Wait for CM7_1 to execute WFI
while(!(MC_ME->PRTN0_CORE1_STAT & MC_ME_PRTN0_CORE1_STAT_WFI_MASK)) { }

// Disabling clock for CORE1 (CM7_1)
MC_ME->PRTN0_CORE1_PCONF &= MC_ME_PRTN0_CORE1_PCONF_CCE_MASK;
MC_ME->PRTN0_CORE1_PUPD  |= MC_ME_PRTN0_CORE1_PUPD_CCUPD_MASK;
MC_ME->CTL_KEY = 0x5AF0; // Enter key
MC_ME->CTL_KEY = 0xA50F; // Enter inverted key

// Wait the HW process to finish
while(MC_ME->PRTN0_CORE1_PUPD & MC_ME_PRTN0_CORE1_PUPD_CCUPD_MASK) {}
```

Assumptions:
CM7_0 is running this code,
Device is not in Lockstep,
CM7_1 has already executed WFI.


Mode Entry Module (MC_ME)
Clock Generation Module (MC_CGM)
FIRC
SIRC
FXOSC
SXOSC
PLLDIG
Clock Monitoring Unit (CMU)

- This block consist of six monitors reporting malfunctions in the clocking system.

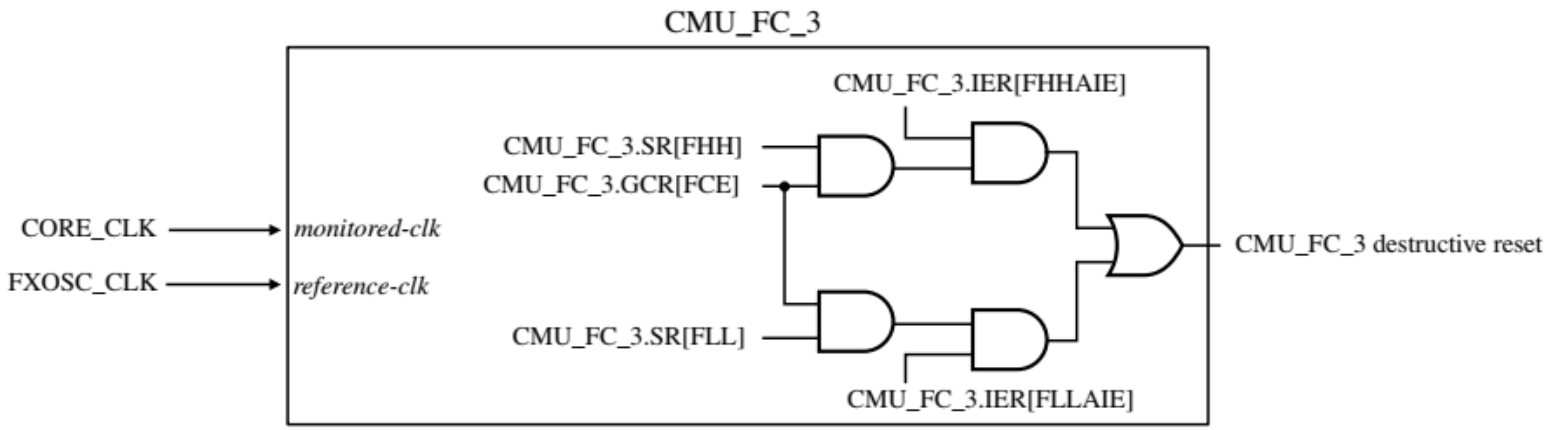
CMU	Type (Check / Meter)	Monitored or metered clock	Failure reaction
0	**FC** (Precision over and under frequency).	FXOSC_CLK	Destructive RST or Interrupt
1	**FM** (Freq. measurement periodically triggered by SW).	FIRC_CLK	Interrupt
2	**FM** (Freq. measurement periodically triggered by SW).	SIRC_CLK	Interrupt
3	**FC** (Precision over and under frequency).	CORE_CLK	Destructive RST
4	**FC** (Precision over and under frequency).	AIPS_PLAT_CLK	Destructive RST
5	**FC** (Precision over and under frequency).	HSE_CLK	Destructive RST

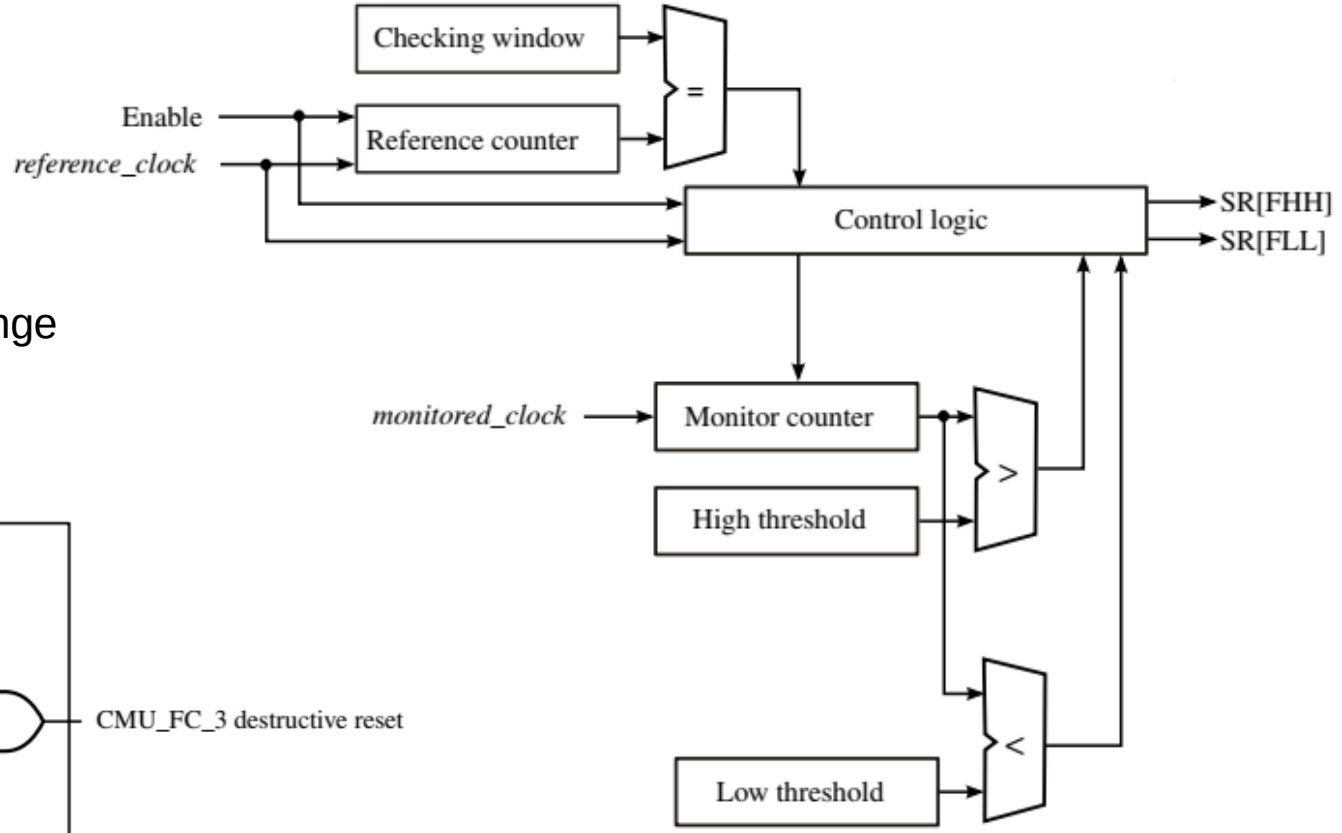
FIRC reference
FXOSC reference

CMU LOGIC EXAMPLE

Take CMU_FC_3 as an example:

- It uses FXOSC as reference clock to monitor the CORE_CLK
- Assume FXOSC is 16MHz, CORE_CLK is 160MHz
  - Monitored clock is 10 times of the reference clock
- Set the checking window duration to 160
- Set the High/Low threshold for monitored clock
  - HFREF = 1700; LFREF = 1500;
- With above settings, the CORE_CLK going out of the range 150MHz~170MHz will cause a destructive reset.





Clock Configuration Tool

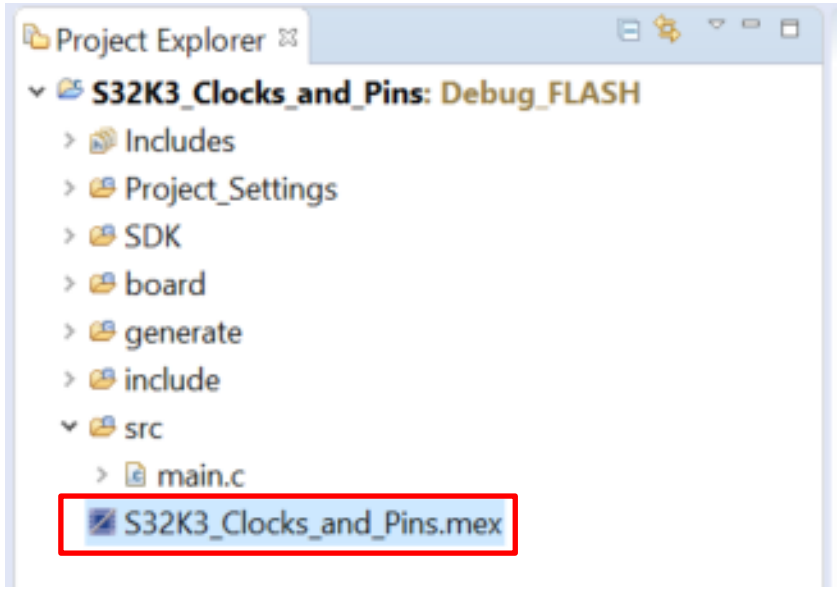
SECURE CONNECTIONS
FOR A SMARTER WORLD

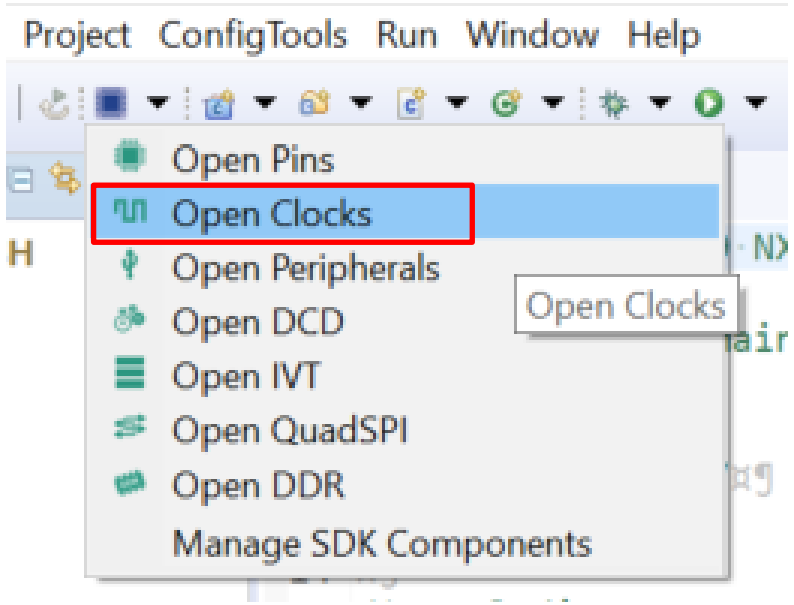
SYSTEM CLOCKING CONFIGURATIONS EXAMPLES

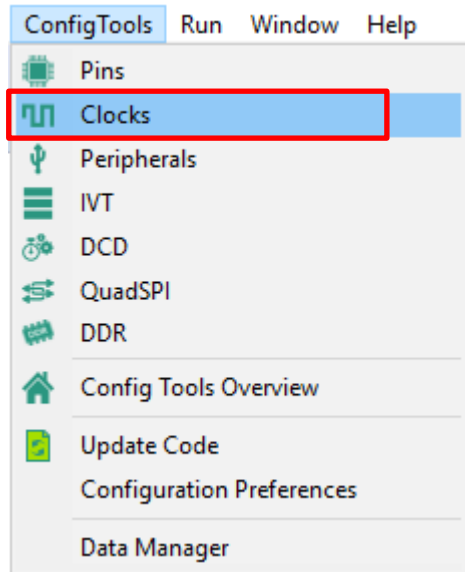
Clocking Options	Clock Option A	Clock Option B	Clock Option C	Clock Option D	Clock Option E	Clock Option E2	Clock Option F
	High-Performance Mode	**Reduced Speed Mode**	**Boot Standby Mode**	**FIRC divider bypassed**	**Low Speed RUN**	**Very low speed Run**	**Core and AXBS at same speed Mode**
System Clock Source (SYS_CLK)	PLL_PHI0_CLK	PLL_PHI0_CLK	FIRC(/2)	FIRC (/1)	FIRC (/2)	FIRC (/8)	PLL_PHI0_CLK
PLL VCO frequency	960 MHz	480 MHz	NA	NA	NA	NA	480 MHz
PLL_PHI1_CLK	160 MHz (VCO/6)	160 MHz (VCO/3)	NA	NA	NA	NA	K344: 240 MHz (VCO/2) K342: 160 MHz (VCO/3)
PLL_PHI0_CLK	160 MHz (VCO/3)	120 MHz (VCO/4)	NA	NA	NA	NA	160 MHz (VCO/3)
CORE_CLK (Application cores, AXBS, SRAM, AIPS0, Flash Controller Port Clock, QSPI Memory Clock, Fast Speed Peripherals Clock)	160 MHz(SYS_CLK)	120 MHz(SYS_CLK)	24 MHz (FIRC/2)	48 MHz(FIRC/1)	3 MHz(FIRC/2/8)	750 KHz (FIRC/8/8)	80 MHz(SYS_CLK/2)
AIPS_PLAT_CLK (Medium speed peripheral clock	80 MHz	60 MHz	24MHz	48 MHz	3 MHz	750 KHz	80 MHz
AIPS_SLOW_CLK (Slow speed peripheral clock)	40 MHz	30 MHz	12MHz	24 MHz	1.5 MHz	375 KHz	40 MHz
HSE_CLK	80 MHz	K34x: 60 MHz K31x: 60 MHz or 120 MHz	24MHz	48 MHz	3 MHz	750 KHz	80 MHz
LBIST_CLK	40 MHz	30 MHz	NA	NA	NA	NA	40 MHz
QSPI_SFCK	80 MHz	K344: 120 MHz (QSPI only) 80 MHz (QSPI + ENET RMII) K342: 80 MHz	NA	NA	NA	NA	K344: 120 MHz (QSPI only) 80 MHz (QSPI + ENET RMII)

LAUNCH CLOCKS CONFIG TOOL

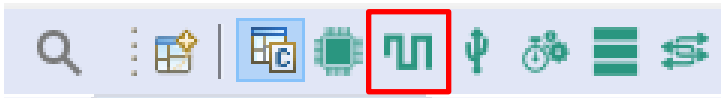
- Expand your project and double click on the *<Project name>.mex* file.
- **Or** click on the “Open S32 Configuration” on the top bar and select “**Open Clocks**”.





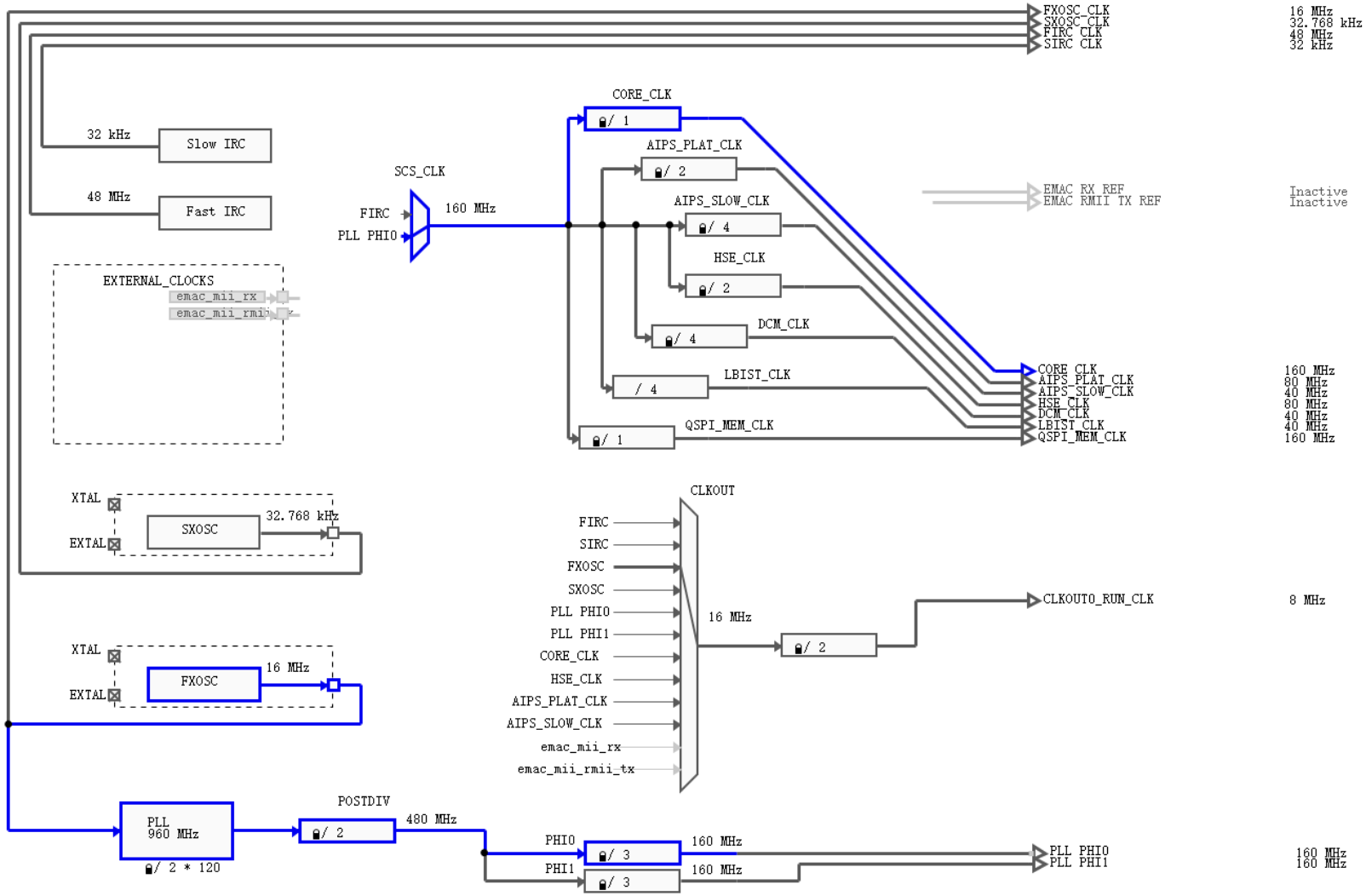


- Make sure, you are in the **Clocks tab** (top right bar).

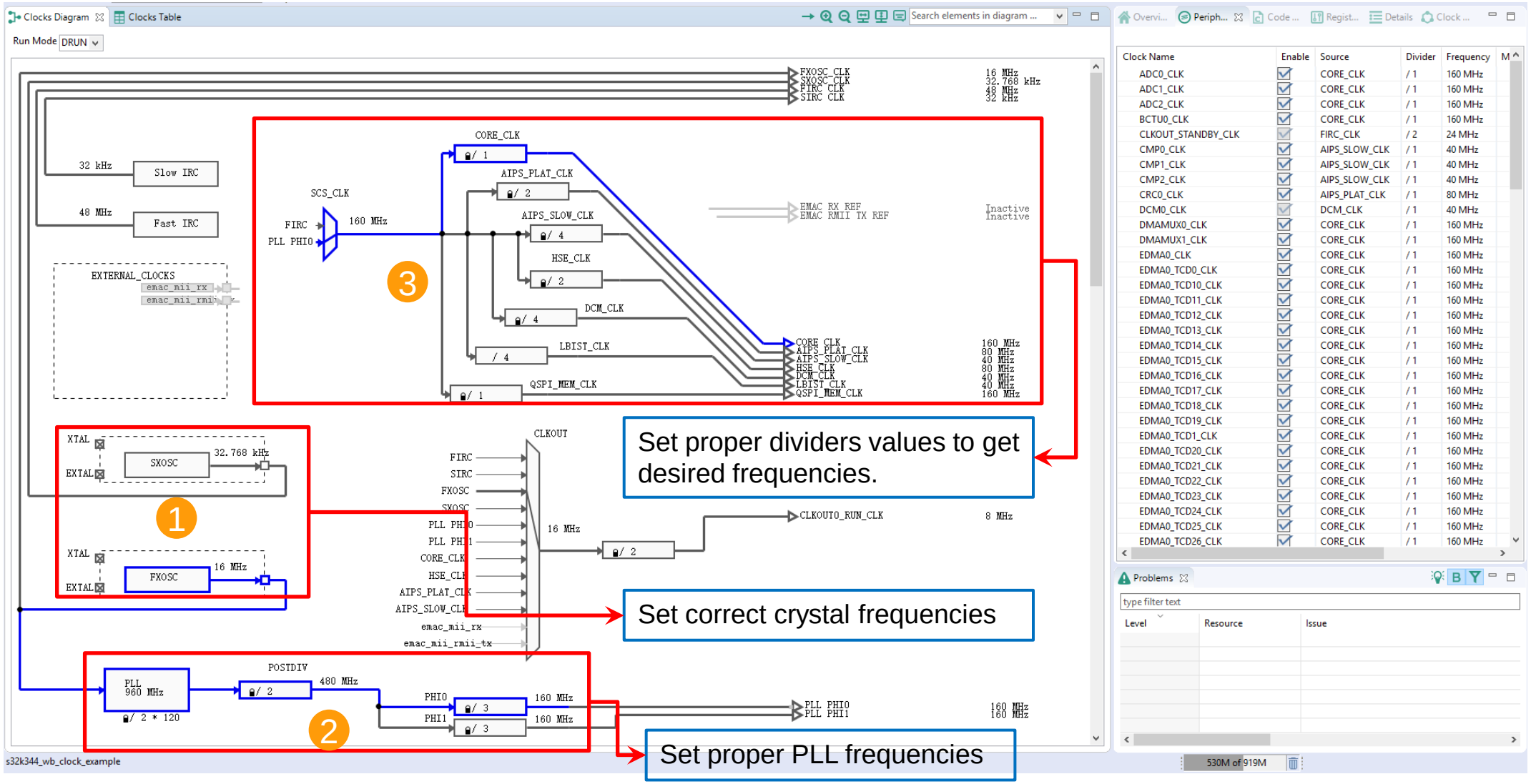


- In the **Clocks Diagram** view, you will see the complete Clocking tree of your device.

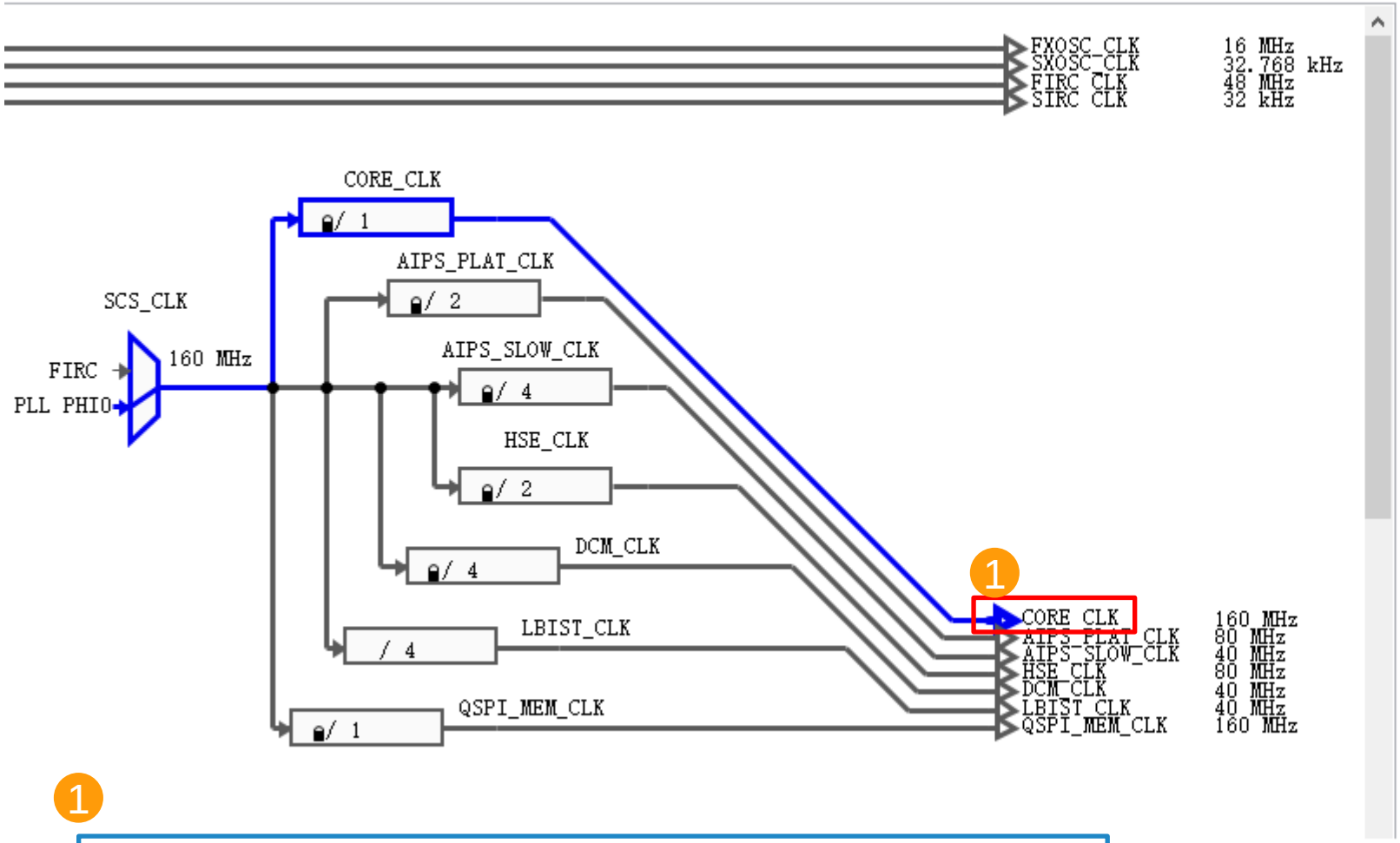
- The default settings is the recommended clock configuration for full performance operation. Proper code can be generated without any modifications.

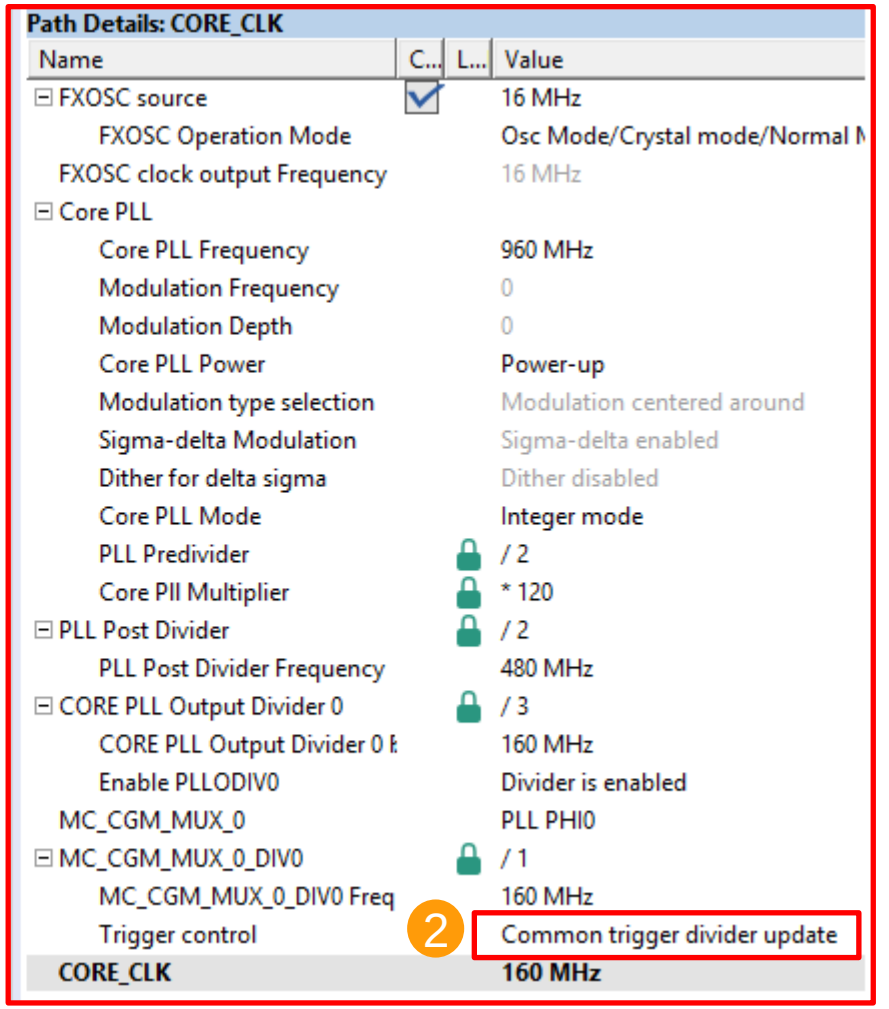


CONFIGURE CLOCKS FOR FULL PERFORMANCE MODE (DEFAULT SETTINGS IN S32DS CT)



CHECK A CLOCK PATH DETAILS





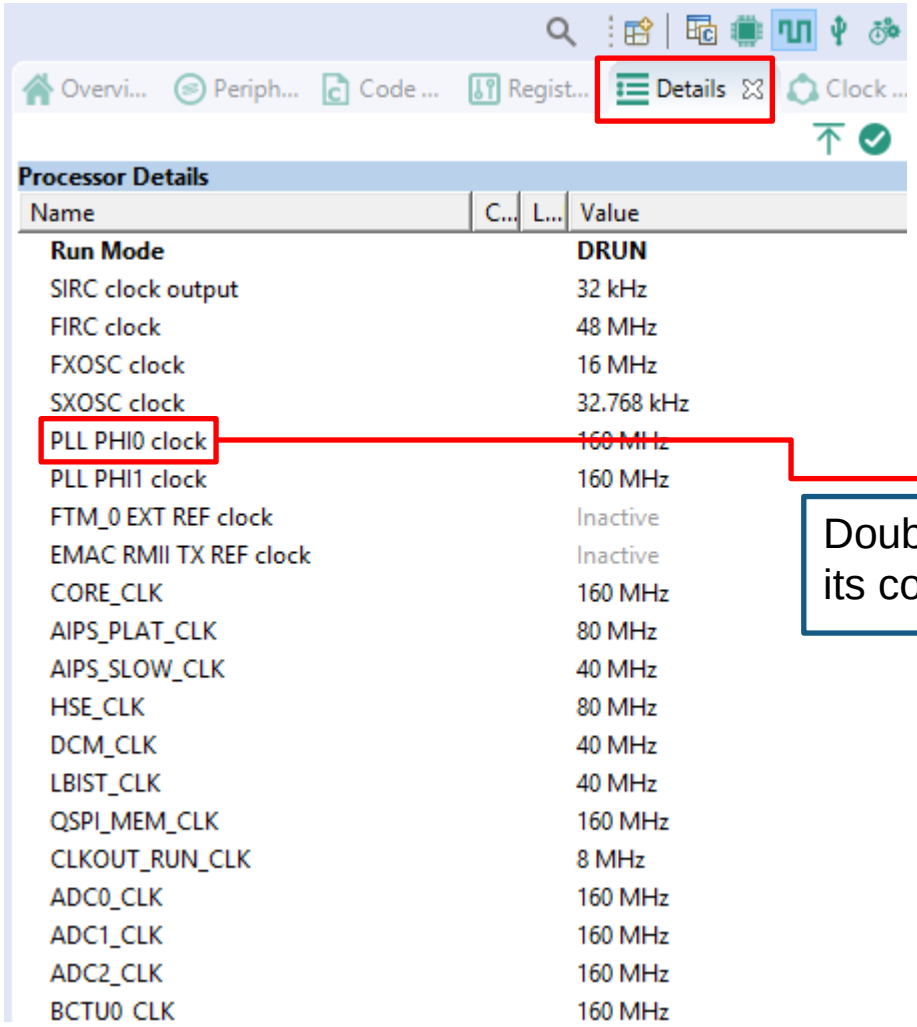
1

Click the “CORE_CLK” to check its clock path details.

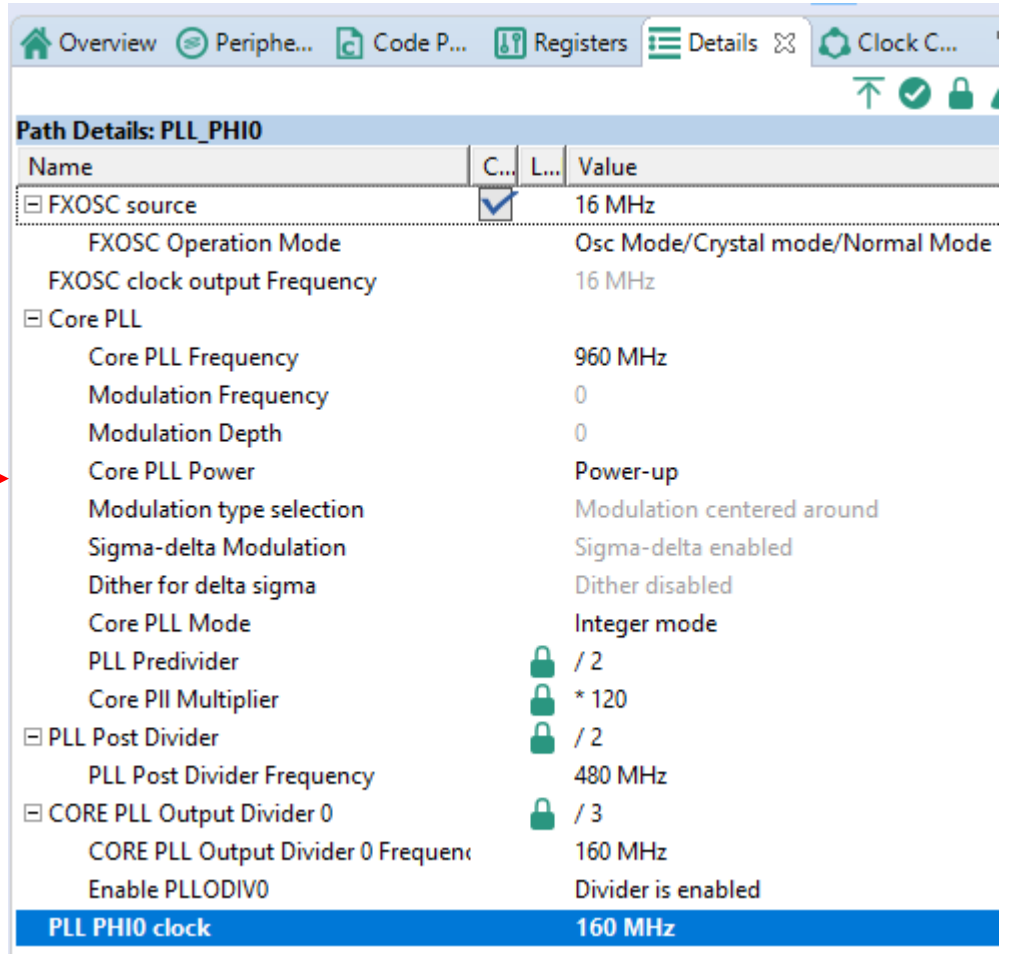
2

“Common trigger” should be enabled to assure system clock switching can be done properly.

CLOCK DETAILS VIEW AND CLOCK DETAILS MODIFICATION

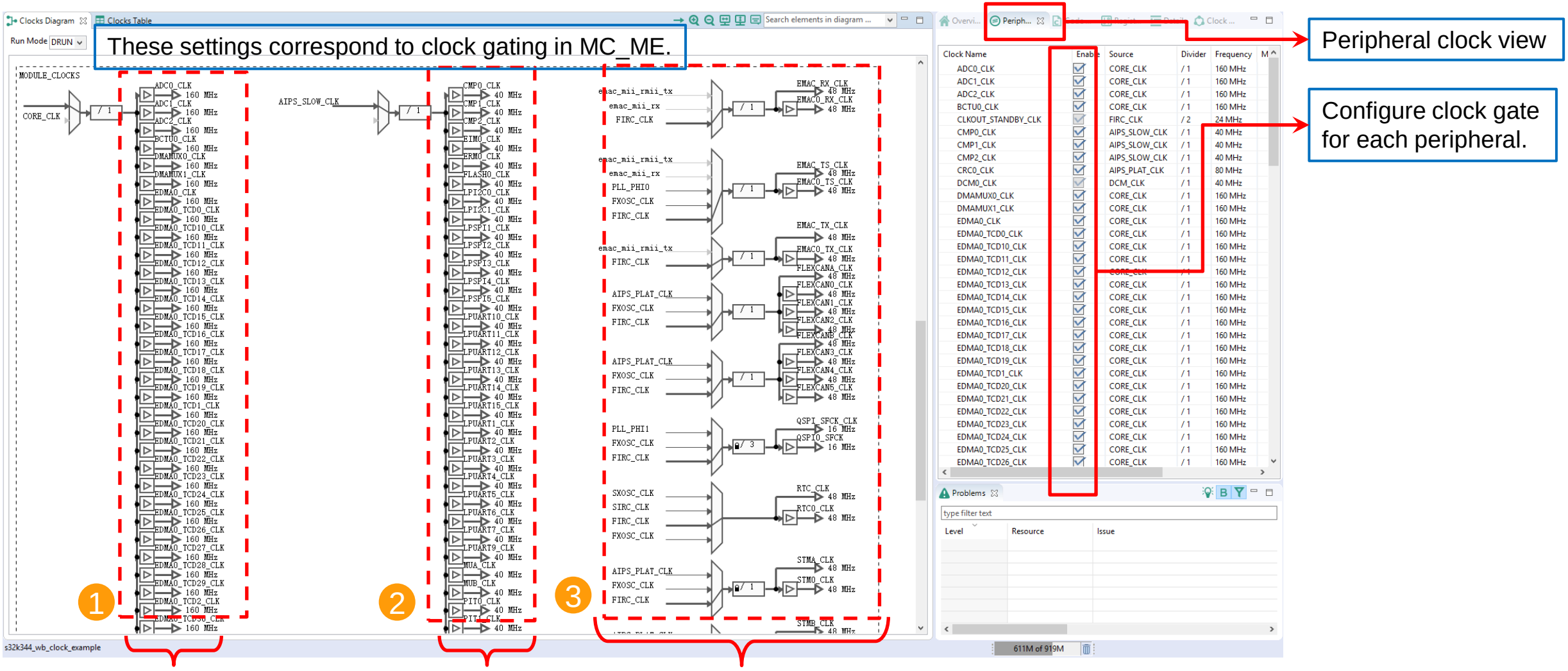


Double click a clock to see its configuration details.

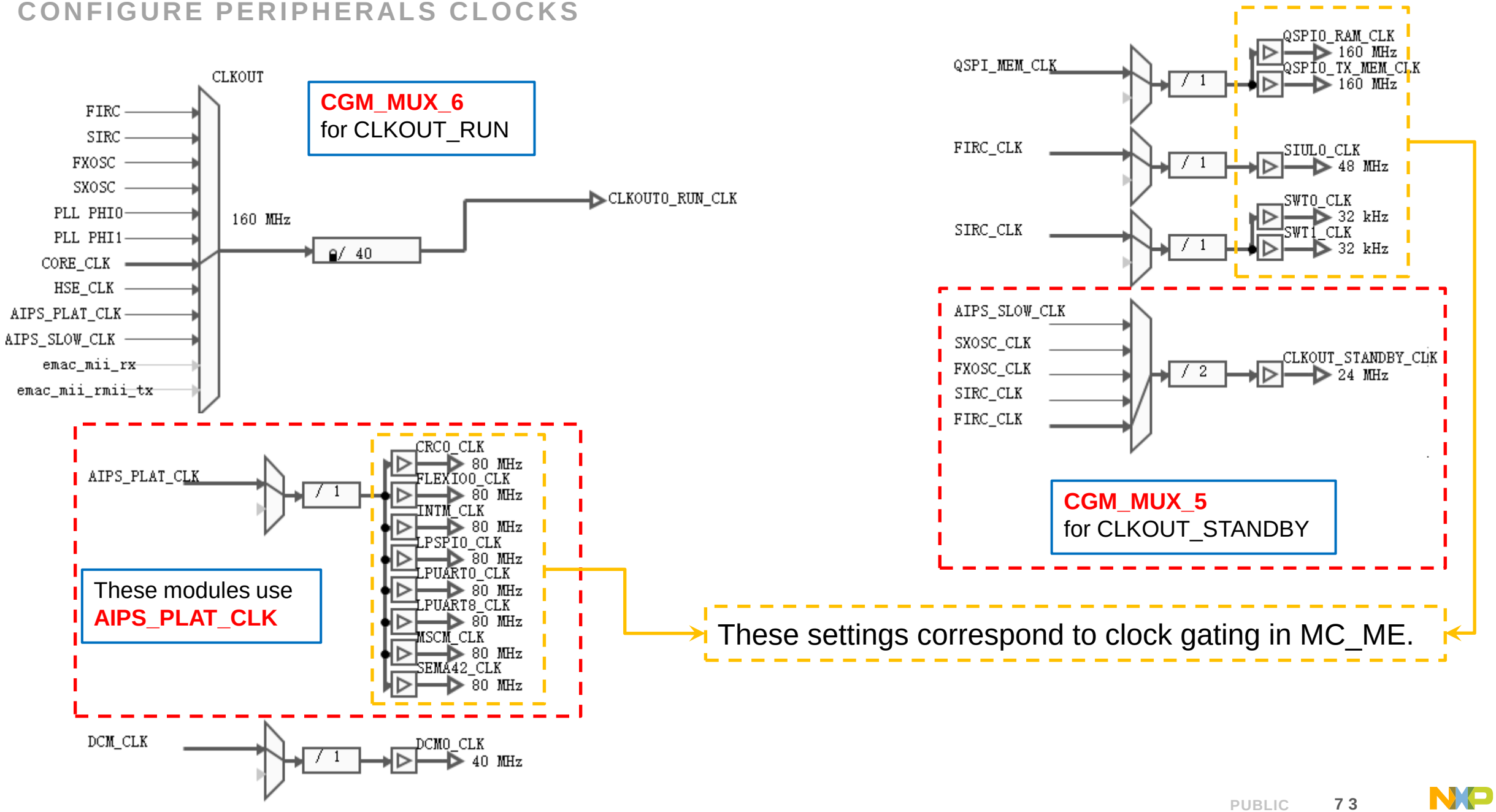


Clock details are shown in this view

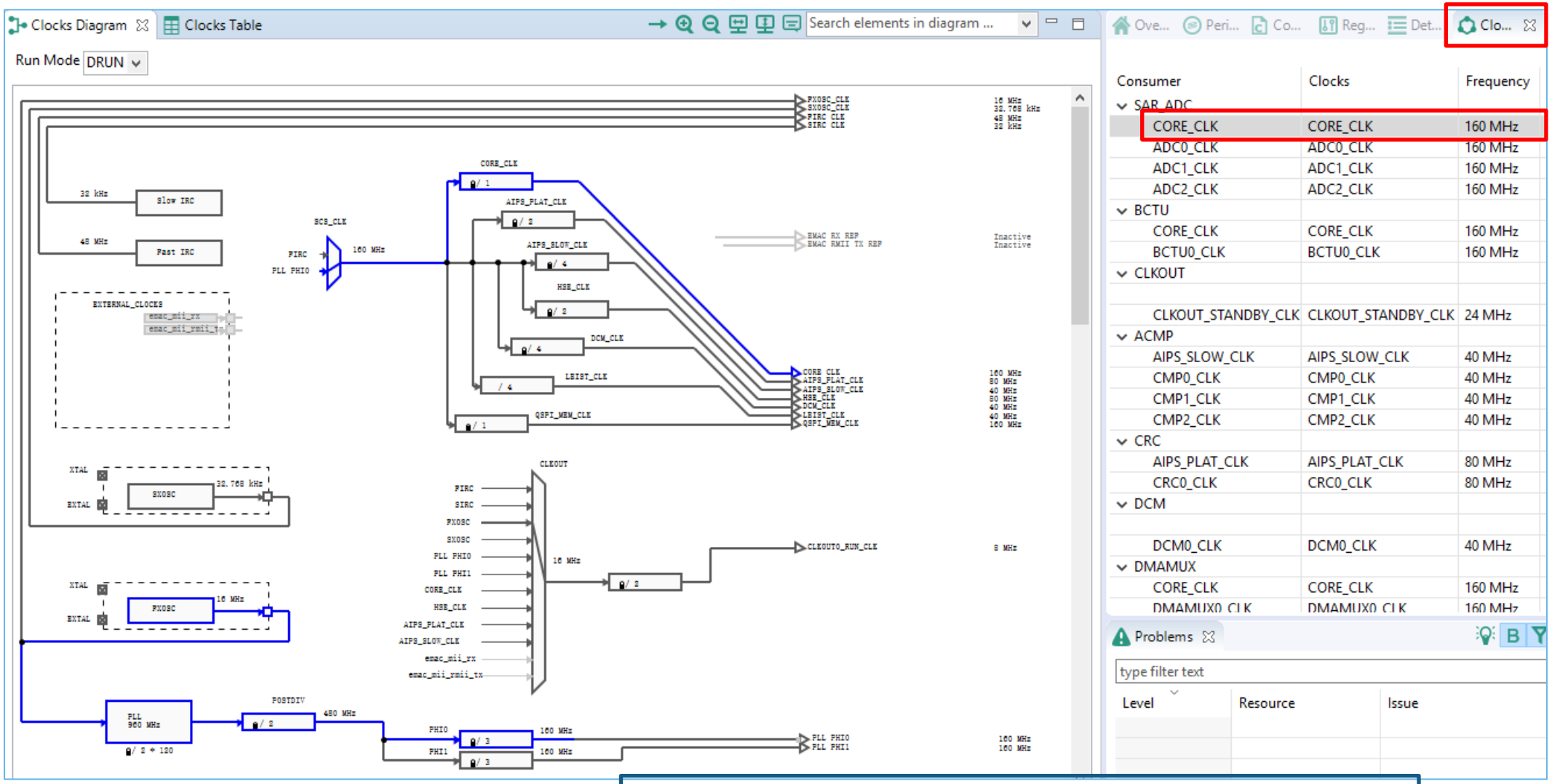
CONFIGURE PERIPHERALS CLOCKS



CONFIGURE PERIPHERALS CLOCKS



CLOCK CONSUMERS VIEW

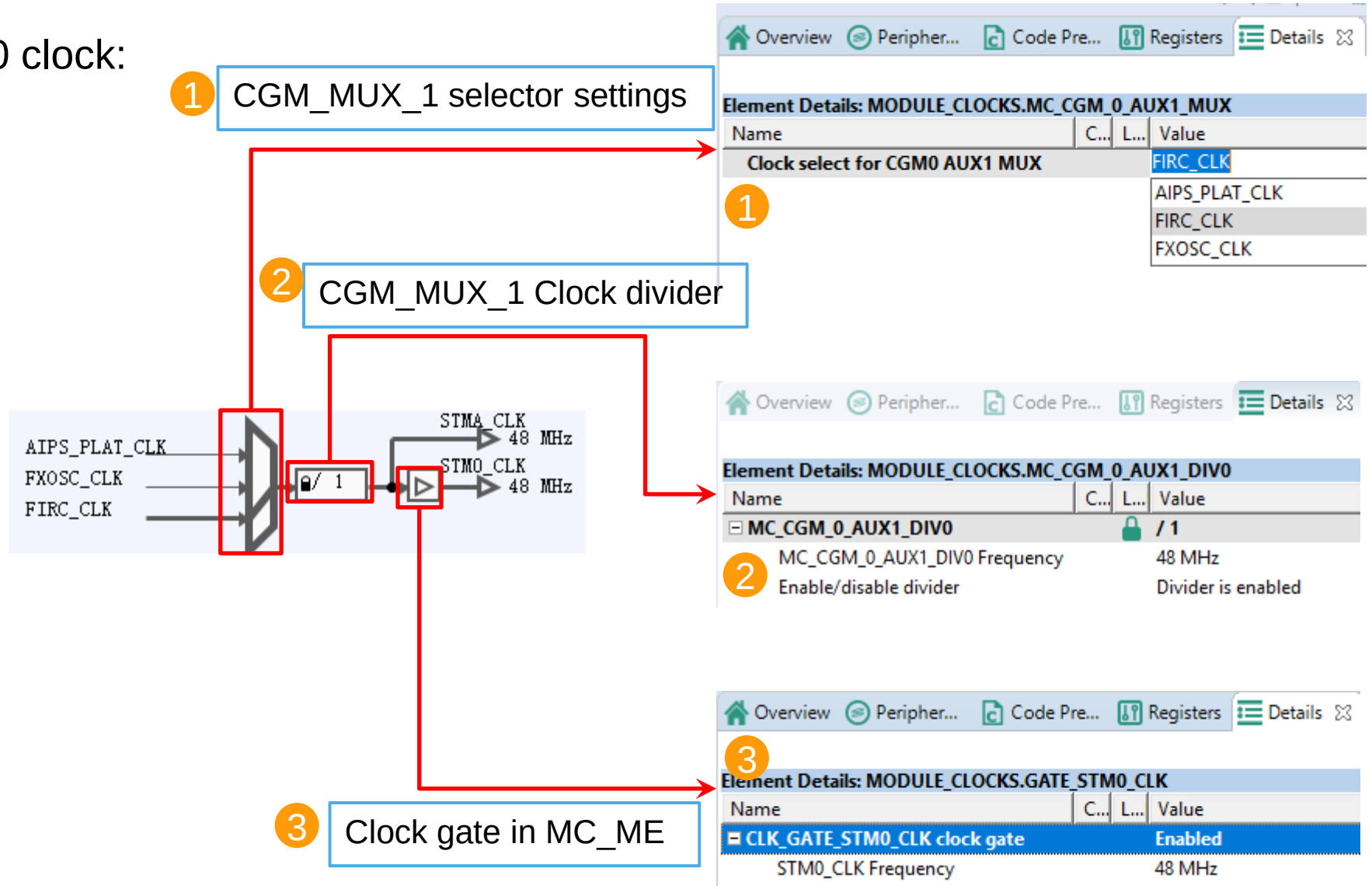


Clock Consumers

Select a clock consumer and its clock generation path will be highlighted in the clock diagram.

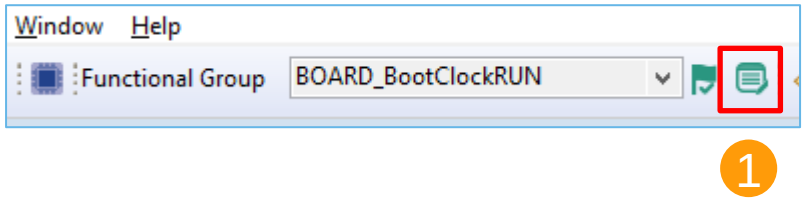
Configure STM0 clock:

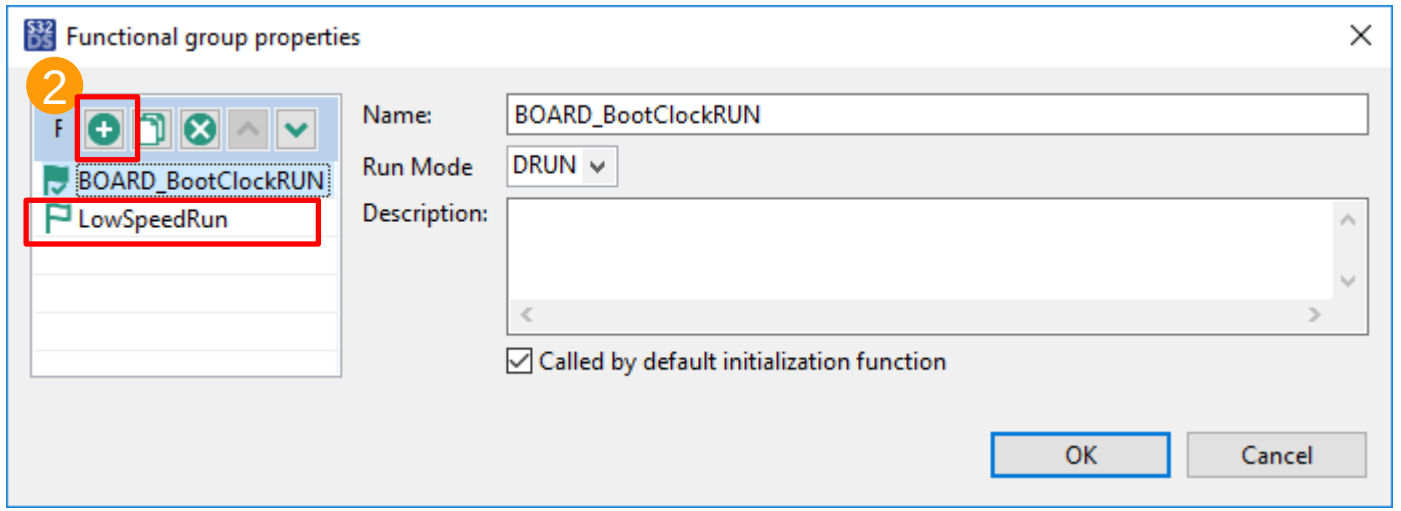
- clock source
- clock divider
- clock gate



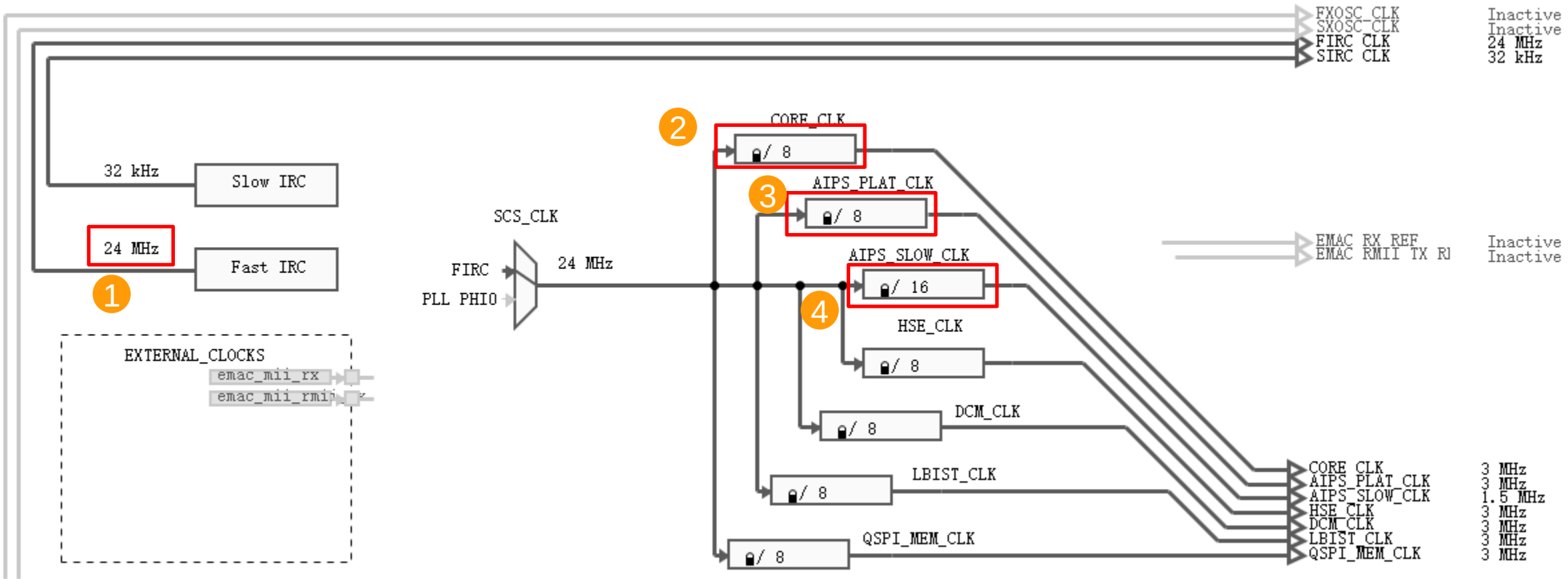
ADD NEW CLOCK SETTINGS GROUP (OPTIONAL)

- If there are more than one set of clocks settings are required, use the process described below to add one more group of clocks settings.
  - For example, one group for high performance operation and the other for low-speed operation.





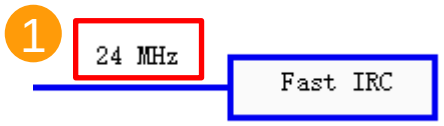
CONFIGURE CLOCK FOR LOW-SPEED-RUN MODE (OPTIONAL)



External crystal and PLL is disabled by default in new clock settings.
Manually change the settings in above diagram to get the desired clock frequency.

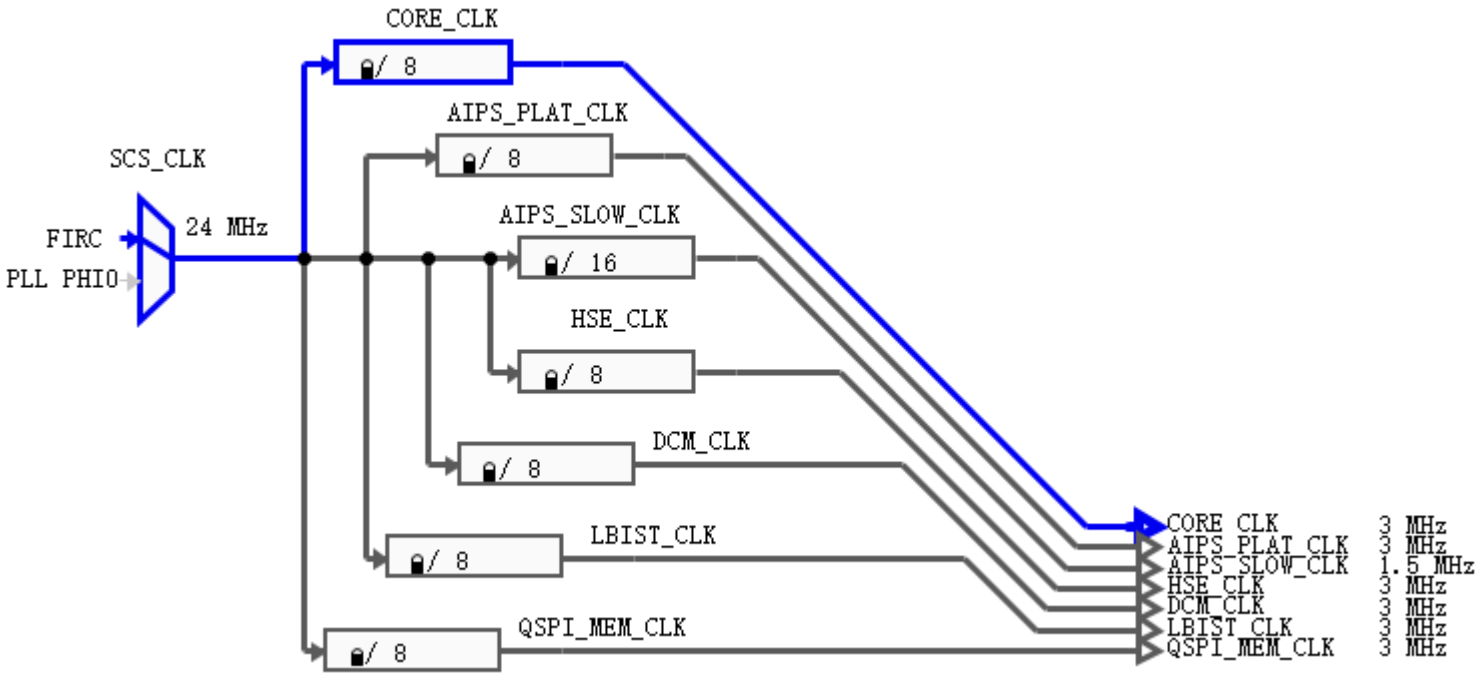
1. Set FIRC divided by 2 as clock source.
2. CORE_CLK = 3MHz
3. AIPS_PLAT_CLK = 3MHz
4. AIPS_SLOW_CLK = 1.5MHz

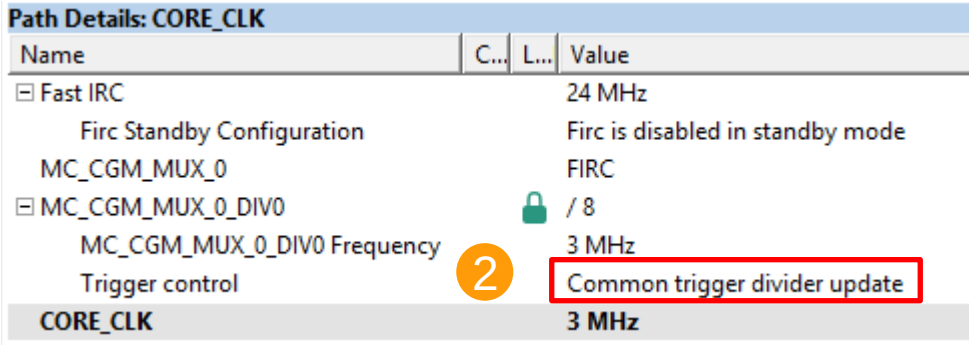
CONFIGURE CLOCK FOR LOW-SPEED-RUN MODE (OPTIONAL)



1 Note: This parameter is determined by register “CONFIG_REG_GPR”.
This is not included in Clock_Ip driver.
Manual code in the application is needed if FIRC clock divider is to be used:

```
CONFIGURATION_GPR->CONFIG_REG_GPR = 0xA0000000;
```

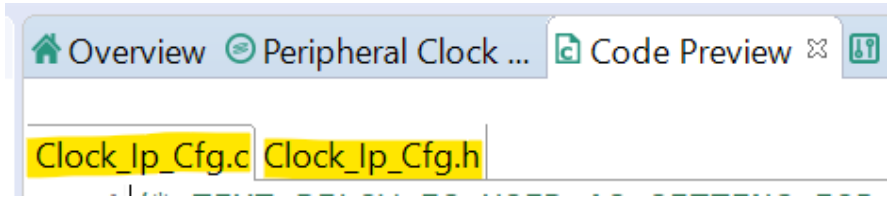


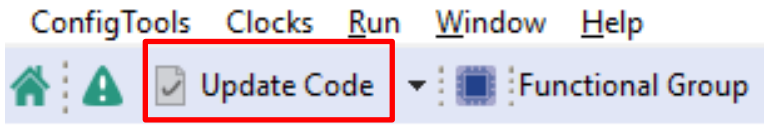


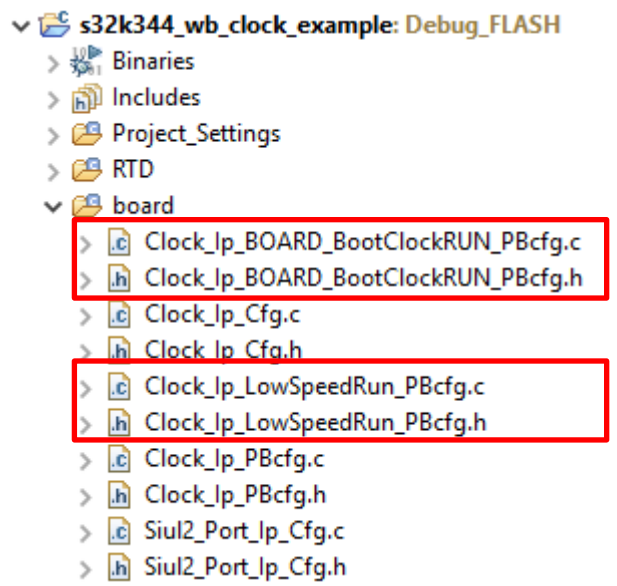
2 “Common trigger divider update” must be enabled.

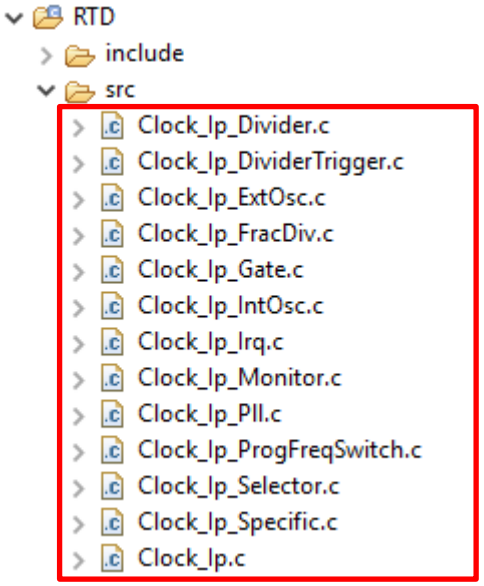
GENERATE CODE FOR CLOCK CONFIGURATION

- In the **Code Preview** tab, you can see the files which are to be generated.
- To add them into your application project, click the **Update Code** button.
- Come back to the **C/C++** perspective.
- Generated code will be available under the **board** folder.
- Clock driver source files will be added into the project **RTD** folder.









Clock
API Functions

SECURE CONNECTIONS
FOR A SMARTER WORLD

CLOCK API FUNCTIONS

Clock API functions details are described in source code comments and in the user manual under the RTD installation folder: “<RTD_installation_path> \Mcu_TS_T40D34M9I0R0\doc”

The header file “Clock_Ip.h” should be included in your source code. Available API Functions for Pins Configure can be found in “Clock_Ip.h”. Some are explained below:

```
Clock_Ip_StatusType Clock_Ip_Init(Clock_Ip_ClockConfigType const * config);
```

This function configures all clocks according to a clock configuration structure.

1) Ramp down to safe configuration (reset PLL, FXOSC, selectors, dividers)
2) Load the configuration in “config” to enables the FXOSC and PLL, configures clock dividers and selectors, enable/disable clocks to peripherals.

```
void Clock_Ip_DisableModuleClock(Clock_Ip_NameType clockName)
void Clock_Ip_EnableModuleClock(Clock_Ip_NameType clockName)
```

Disable or enable the clock to a peripheral.

```
uint32 Clock_Ip_GetClockFrequency(Clock_Ip_NameType clockName)
```

Returns the frequency of a given clock.

```
void Clock_Ip_InitClock(Clock_Ip_ClockConfigType const * config);
```

This function configures all clocks according to a clock configuration.

```
Clock_Ip_PllStatusType Clock_Ip_GetPllStatus(void);
```

Returns the PLL status – either locked or unlocked.

```
void Clock_Ip_DistributePll(void);
```

Configures the PLL and then activates the PLL clock to MCU

```
Clock_Ip_CmuStatusType Clock_Ip_GetClockMonitorStatus(Clock_Ip_NameType clockName)
```

Returns the clock monitor status of a given clock.

The customer can use the functions included within the Clocks driver. Some examples are shown on the next slide.

CLOCK EXAMPLE CODE

```
#include "Mcal.h"
#include "Clock_Ip.h"
```

Include necessary header files.

```
Clock_Ip_Init(&Mcu_aClockConfigPB_BOARD_BootClockRUN[0]);
```

Configure MCU with full performance clock settings.

```
Clock_Ip_Init(&Mcu_aClockConfigPB_LowSpeedRun[0]);
```

Configure MCU with Low-Speed-Run clock settings.

Initialize MCU clocks first before configuring any peripherals since the peripheral's clocks might be off by default after reset.

CLOCK INITIALIZATION PROCESS IN CLOCK_IP_INIT() FUNCTION

1. Switch all CGM clock mux to safe clock
2. Turn off PLL, FXOSC, SXOSC
3. Initialize FIRC, SIRC,FXOSC, SXOSC
4. Configure PCFS (present only on CGM_MUX_0) if needed
5. Configure clock divider trigger (present only on CGM_MUX_0)
6. Configure all clock dividers - PLL dividers and CGM_MUX_n dividers
7. Trigger the clock update with common trigger feature
8. Configure PLL - Write to PLLDV, PLLFD, PLLFM, PLLCR
9. Configure CGM MUX0~MUX11 clock selector (for clocks that are not using PLL)
10. Configure the clock gate in MC_ME for each peripheral – 105 peripherals gates
11. Distribute PLL – Configure CGM MUX_0~MUX_11 that are using PLL as clock source

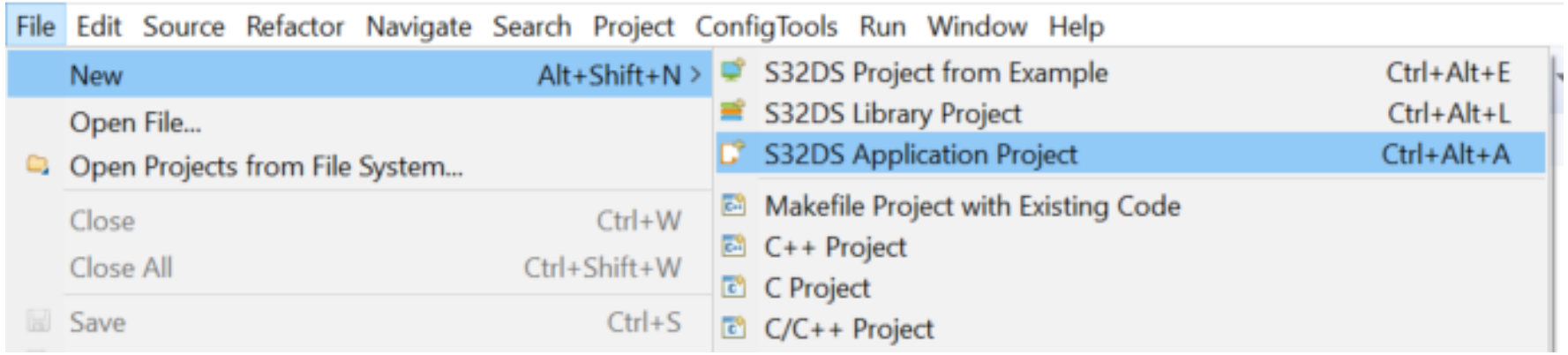
- SXOSC (32KHz external crystal) startup takes a lot of time.
- It's recommended to disable the SXOSC for clock initialization process and enable it later when you need it.
- The SXOSC disable bit should be modified manually in "Clock_Ip_PBcfg.c". See below code.

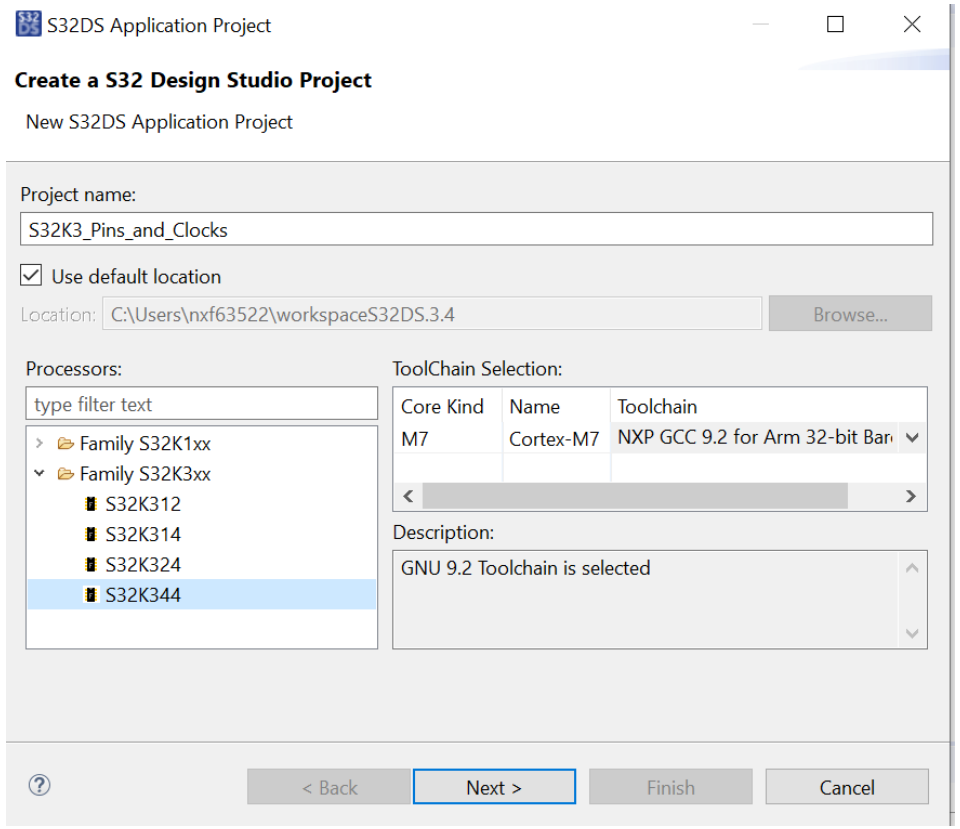
```
#if CLOCK_XOSCS_NO > 1U
{
    SXOSC_CLK,              /* name */
    32768U,                 /* frequency */
    0U,                     /* enable */
    125U,                   /* startupDelay */
    0U,                     /* bypassOption */
    0U,                     /* Comparator is not enabled */
    0U,                     /* Crystal overdrive protection */
},
#endif
```

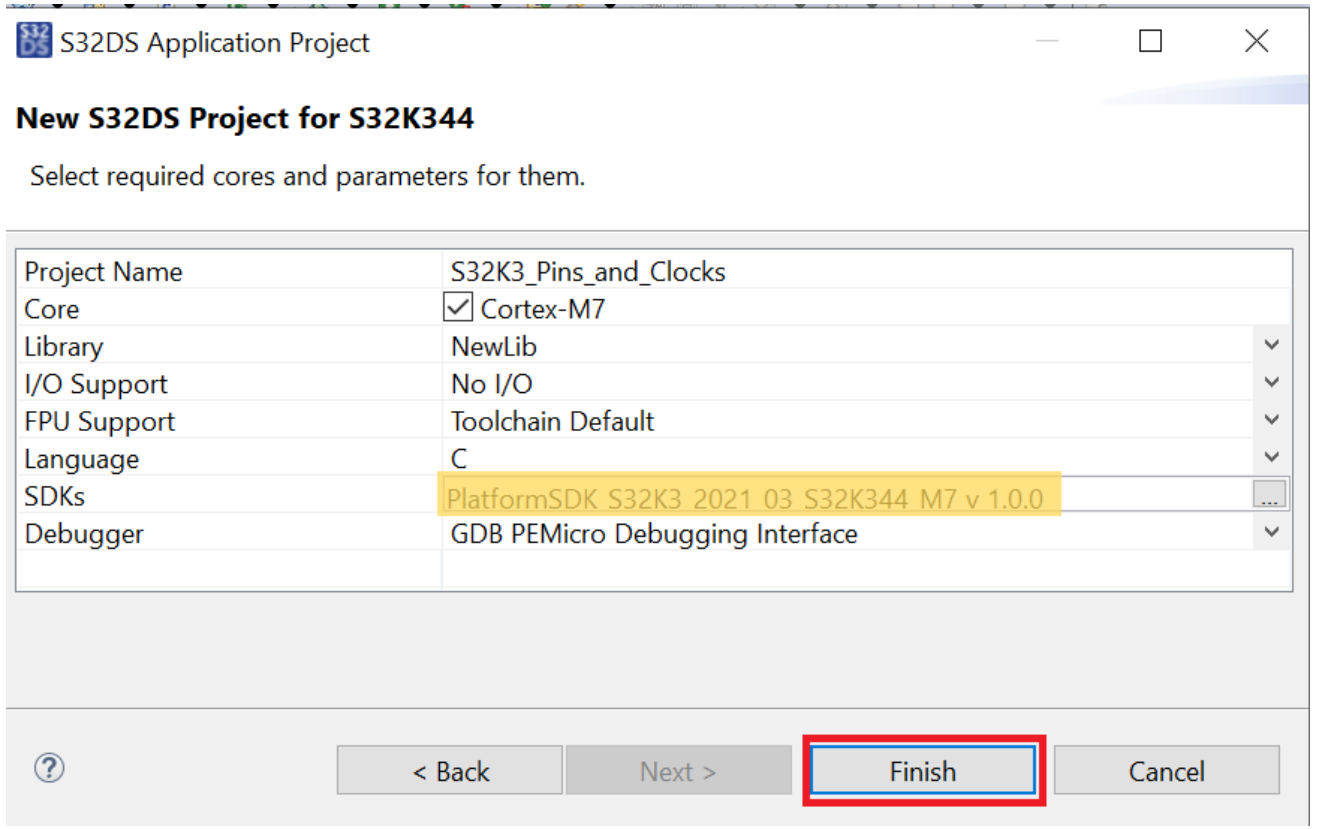
RTD Clocks and Pins Lab

SECURE CONNECTIONS
FOR A SMARTER WORLD

- Start **S32DS** with desired workspace.
- File -> New -> **S32DS Application Project.**
- Enter a **Project name**, select the **Target Processor** (S32K344). Click **Next**.
- Select the **Debugger** to use (PEMicro) and latest **SDK**.
- Click **Finish**.

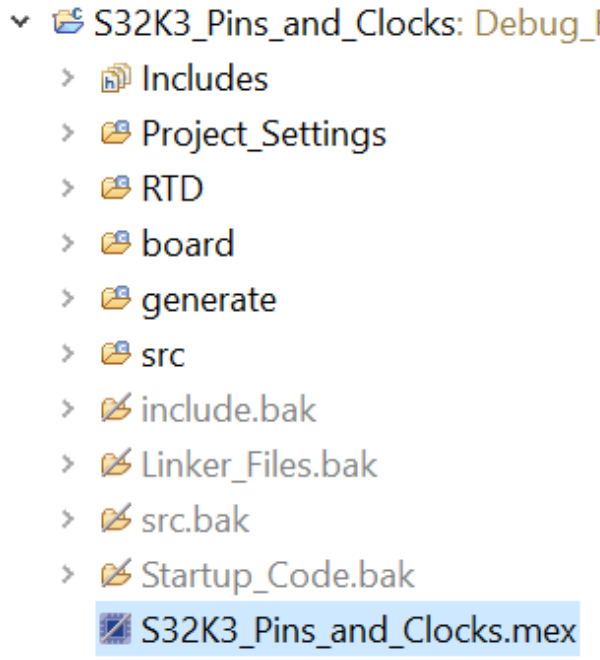


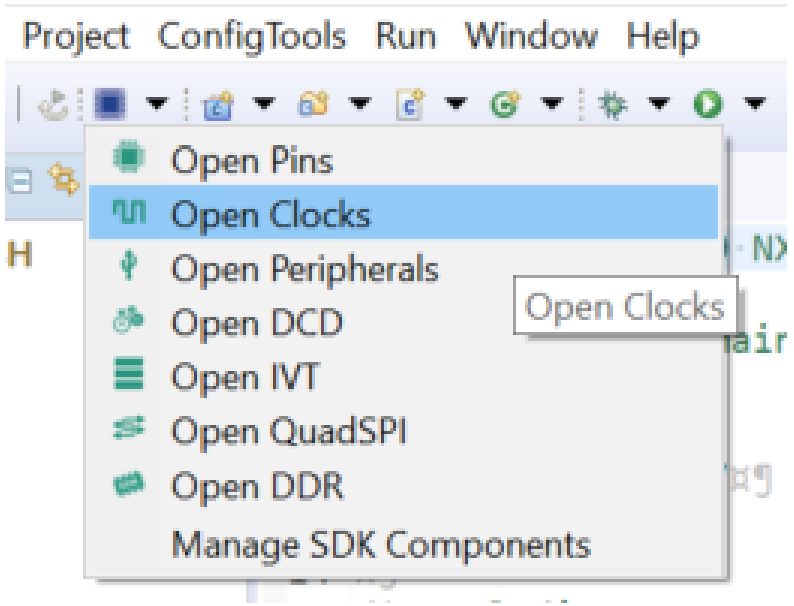




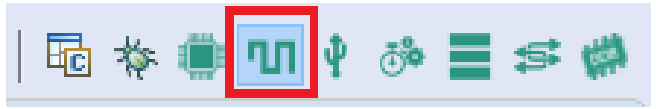
Notice that the SDK could change depending on the RTD version. Select the latest release.

- Expand your project and double click on the *<Project name>.mex* file.
- **Or** click on the “Open S32 Configuration” on the top bar and select “**Open Clocks**”.

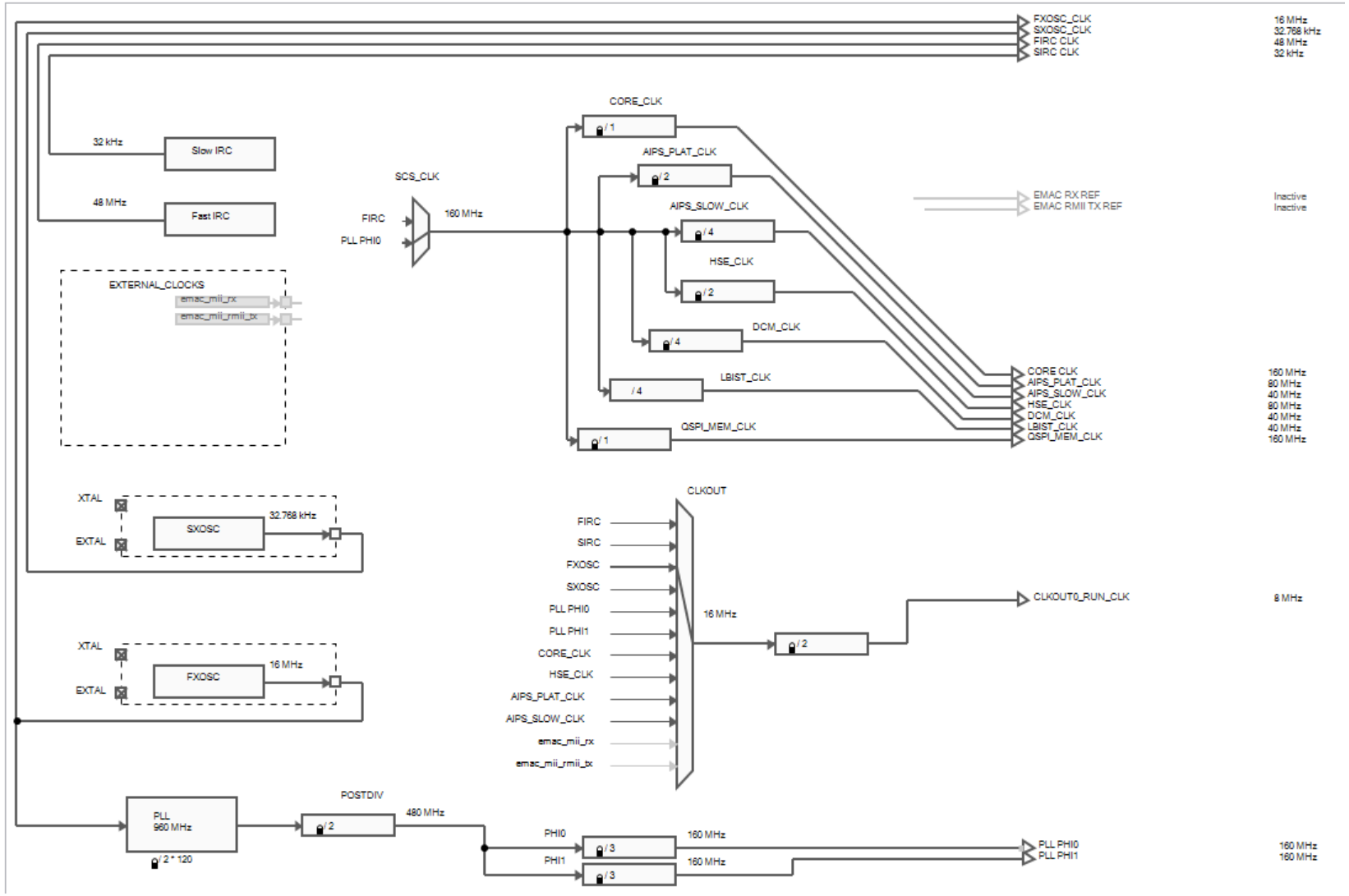




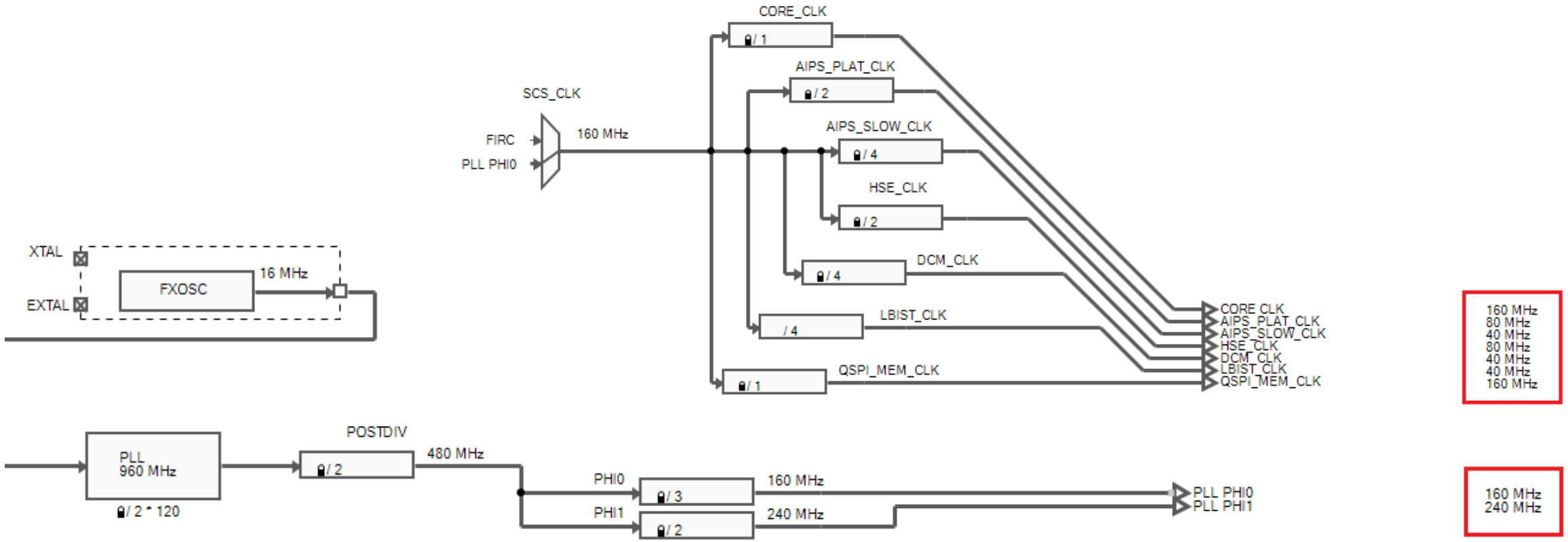
- Make sure, you are in the **Clocks tab** (top right bar).



- In the **Clocks Diagram** view, you will see the complete Clocking tree of your device.



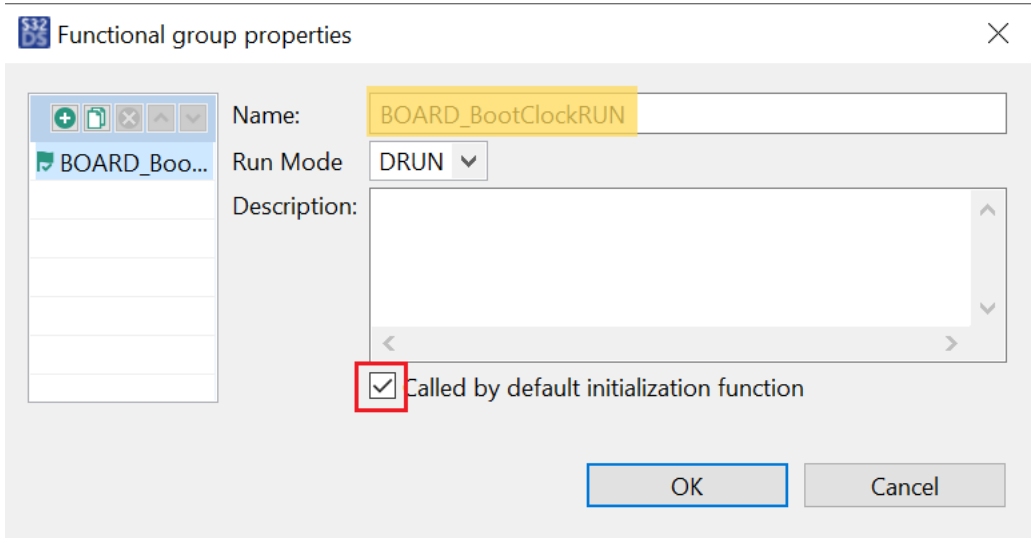
Let's configure the Option A (High Performance mode) as in the Reference Manual (RM), using the 16 MHz crystal from the EVB.



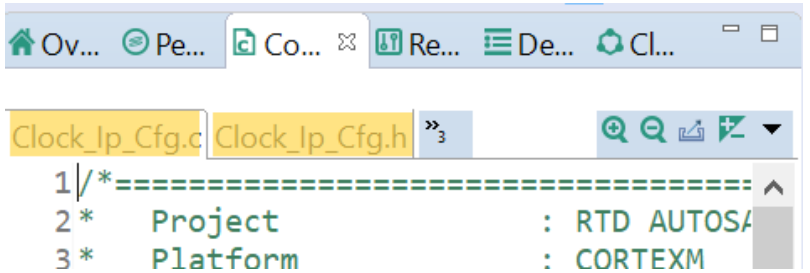
- Open the **Functional Group Properties**

Functional Group Properties

- And check the **Called by default…** Option.



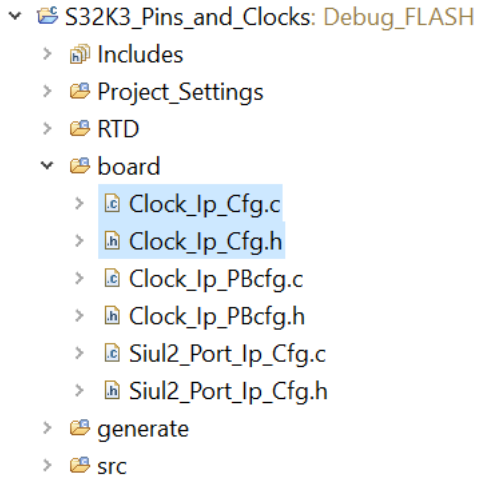
- In the **Code Preview** tab, you can see these files



- To add them into your application project, click the **Update Code** button.

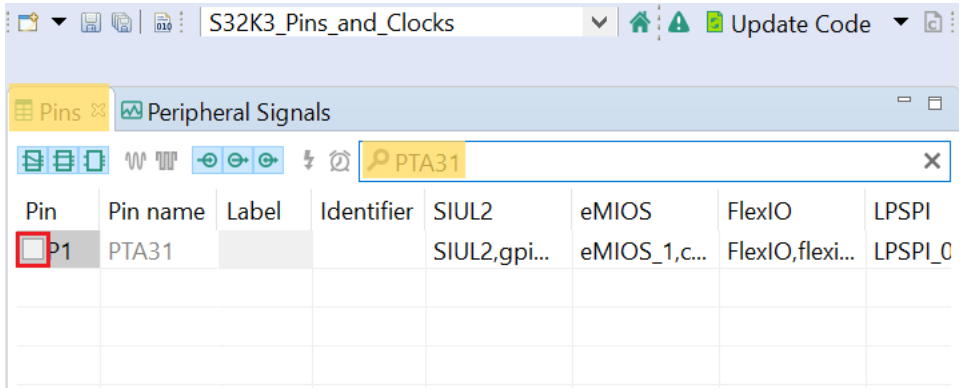
- Come back to the **C/C++** perspective.

- Code will be available under the **board** folder.

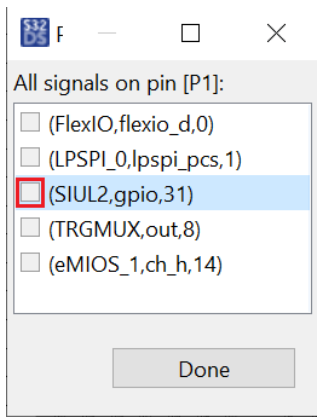


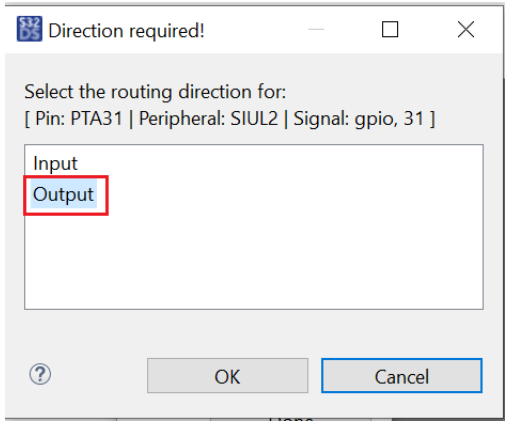
- Open the **Pins** Configuration Tool.

- In the **Pins** tab, type **PTA31** and click the corresponding pin name.



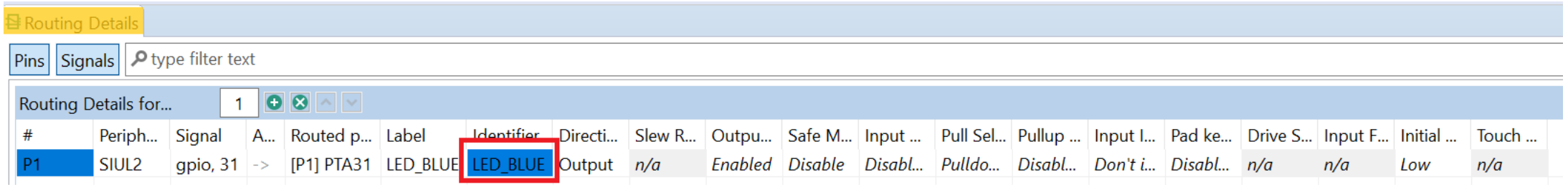
- Select the pin alternative (**SIUL2**) and direction (**Output**). Then click OK, Done.



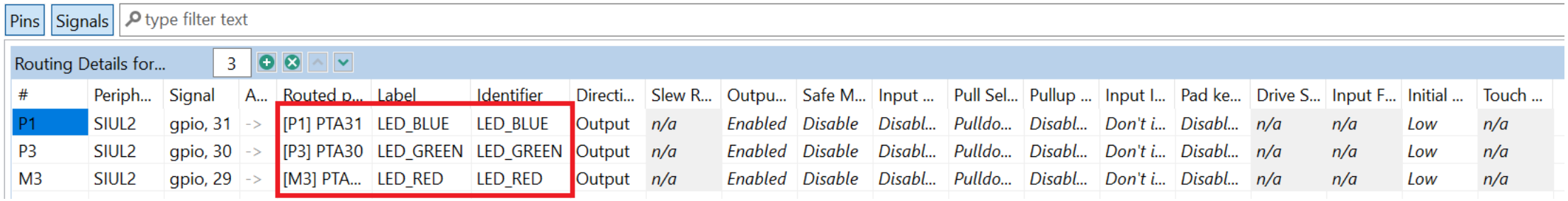


- In the **Routed Pins** tab, you can continue configuring the electrical characteristics of your pin, such as *pull resistors, drive strength or pad keeping*.

- It is recommended to use the **Identifier** field to give a meaningful ID to each pin.

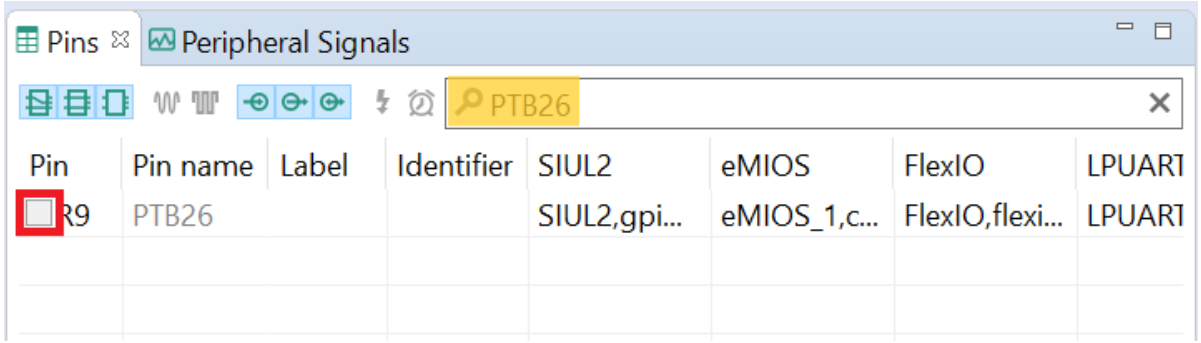


- Do the same procedure for PTA30 (LED_GREEN) and PTA29 (LED_RED)

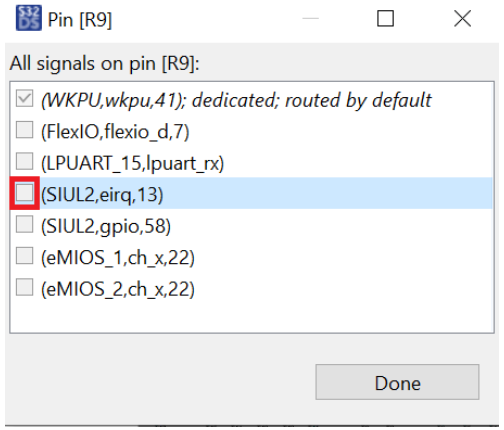


- The outputs (LEDs) were already configured. Now, configure the inputs.

- In the **Pins** tab, type **PTB26** and click the corresponding pin name.

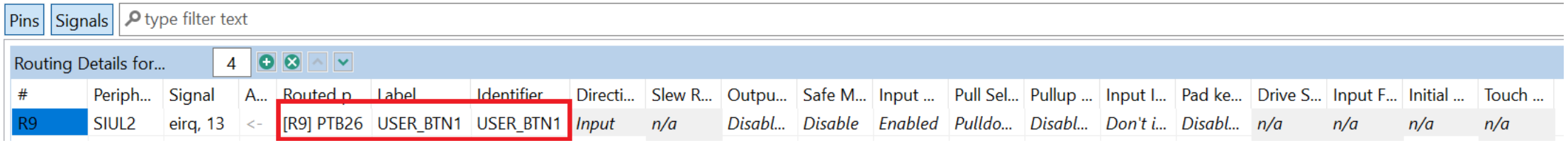


- Select the pin alternative (**SIUL2, eirq**). Then click OK, Done.

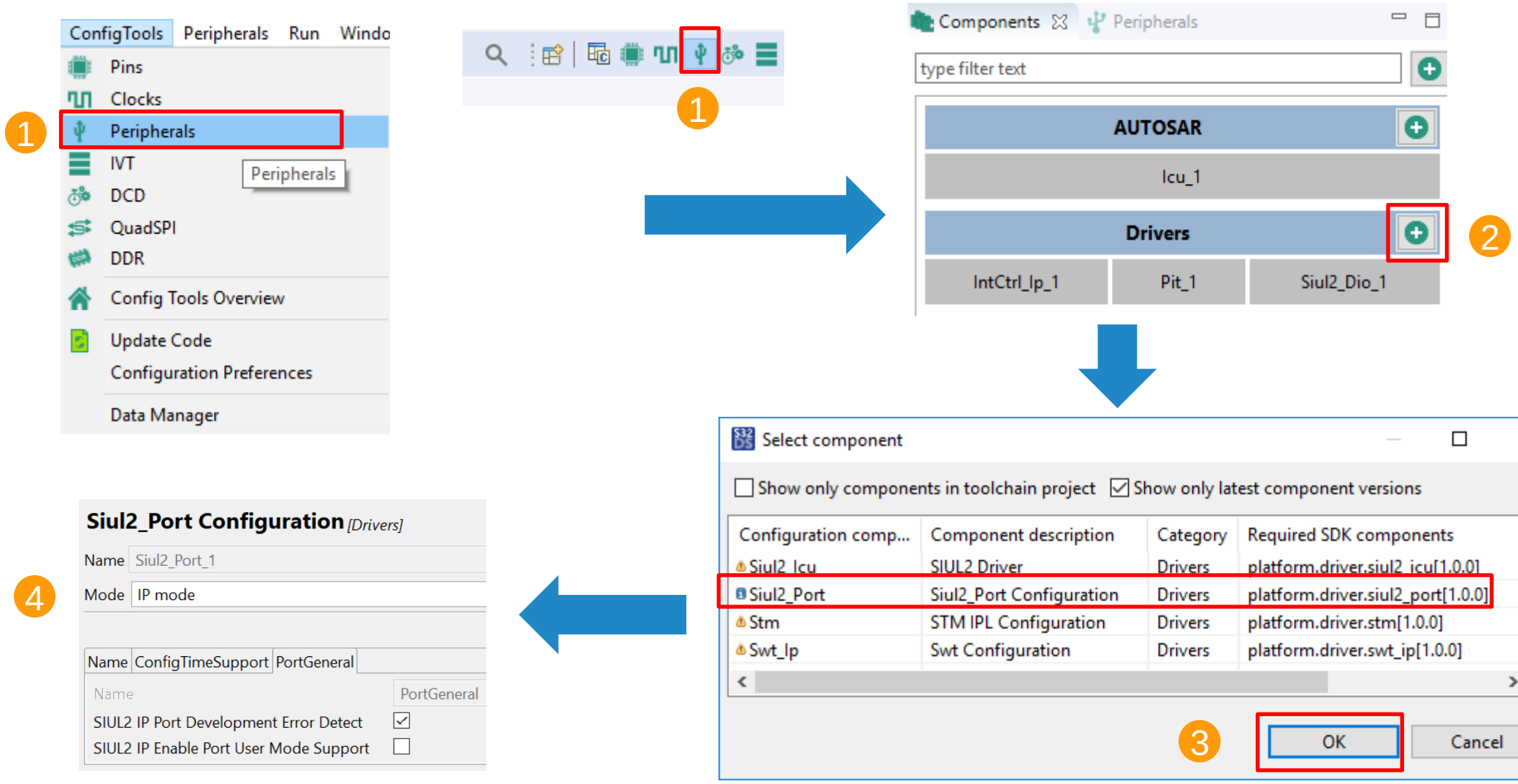


- In the **Routed Pins** tab, you can continue configuring the electrical characteristics of your pin, such as *pull resistors, drive strength or pad keeping*.

- It is recommended to use the **Identifier** field to give a meaningful ID to each pin.

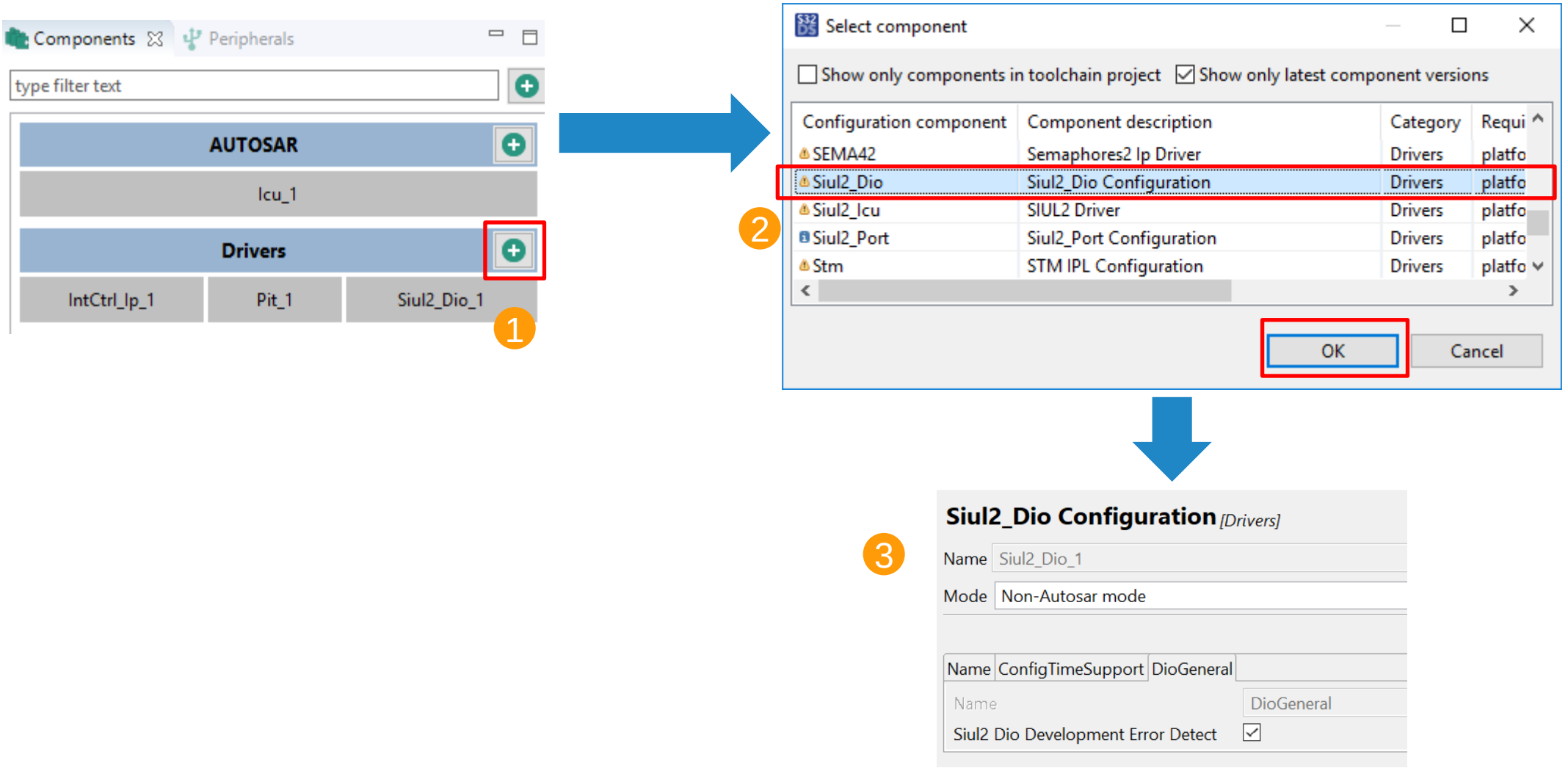


- After configuring all the desired pins. Go to the Peripherals view and add the Siul2_Port driver.

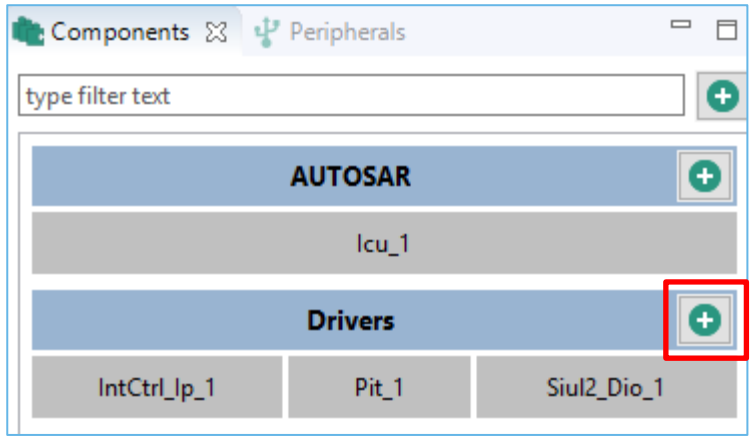


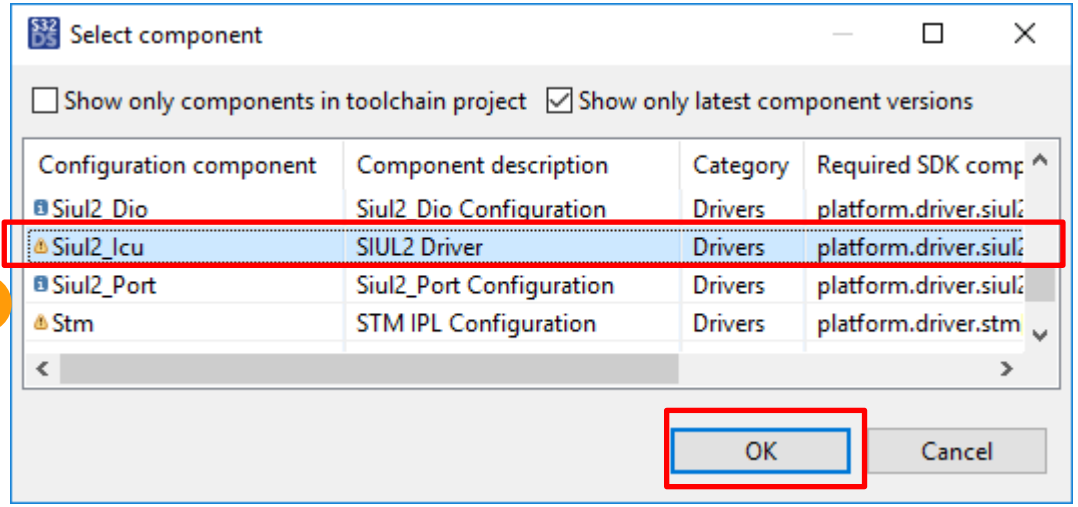
ADD SIUL2_DIO MODULE

- The Siul2_Dio driver has the necessary functions for digital input/output pins

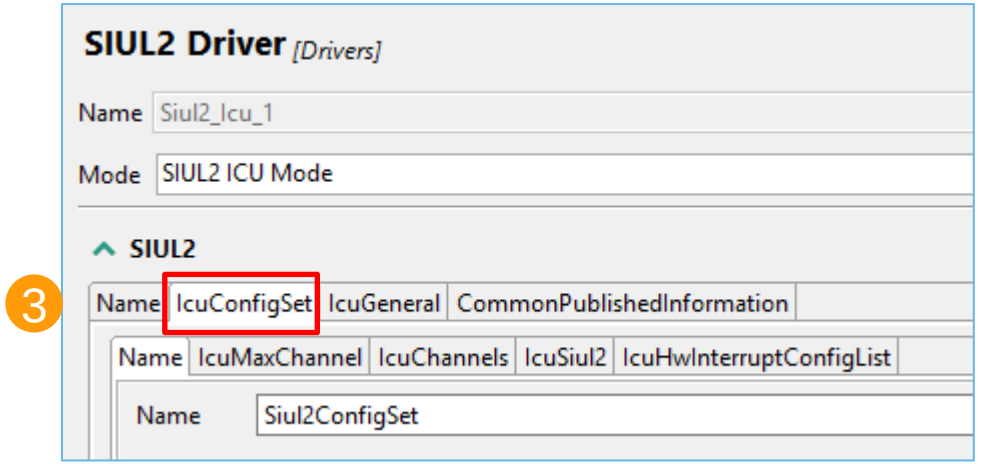


ADD SIUL2_ICU MODULE

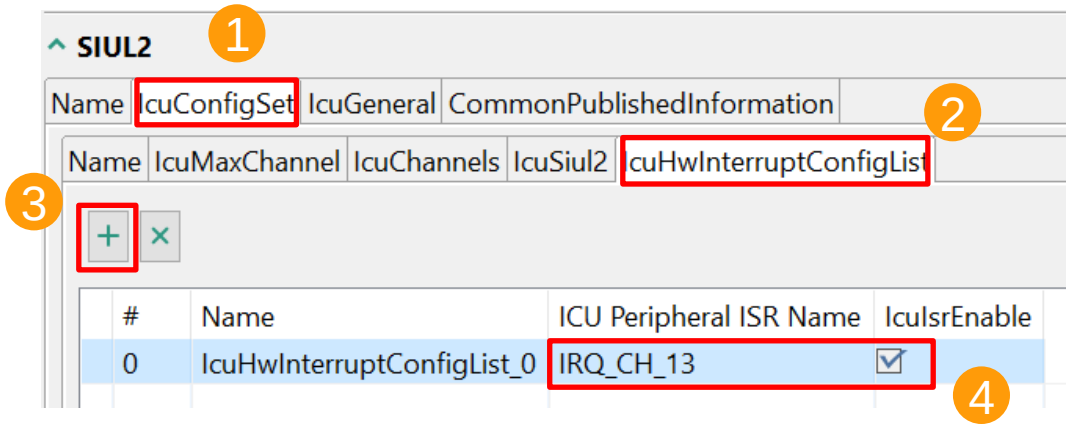




- The Siul2_Icu driver has the necessary functions to capture rising/falling edges and configure their interrupts.



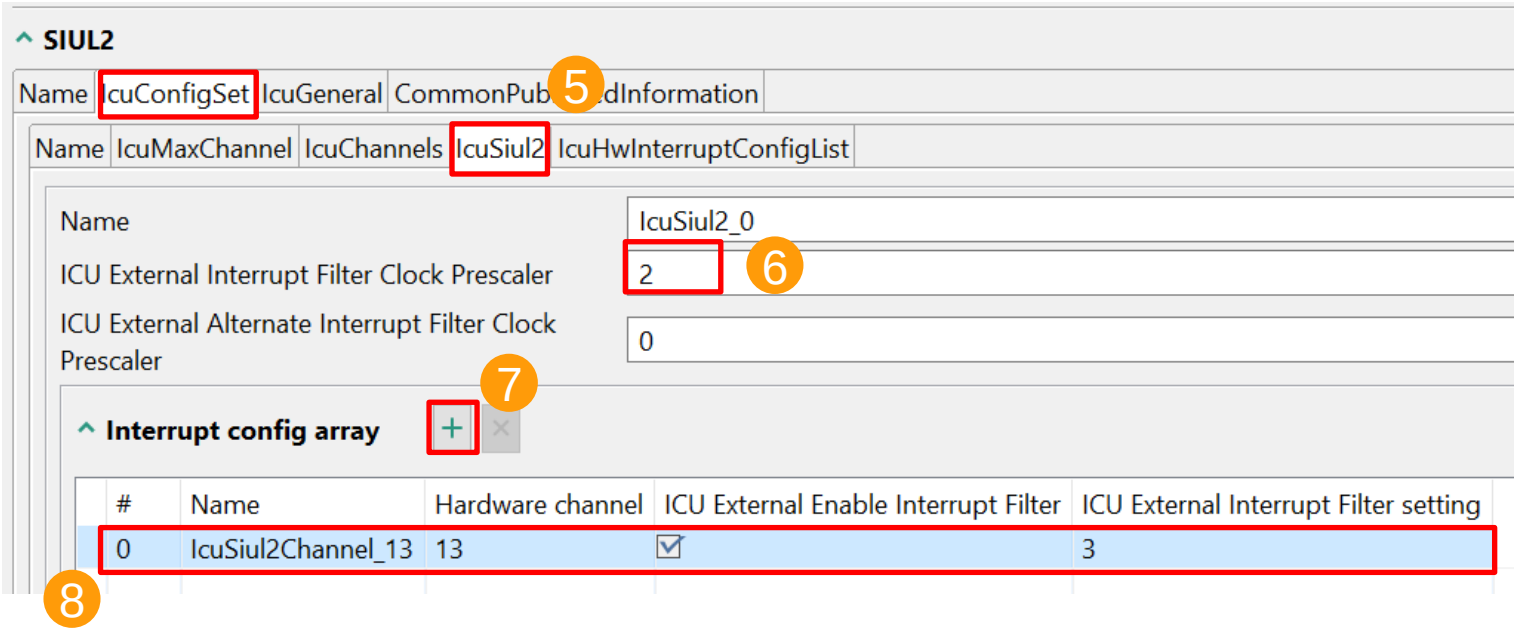
Add the desired Siul2 IRQ channel for detecting rising/falling edges.
Enable this IRQ channel interrupt



Configure each IRQ channel glitch filter:

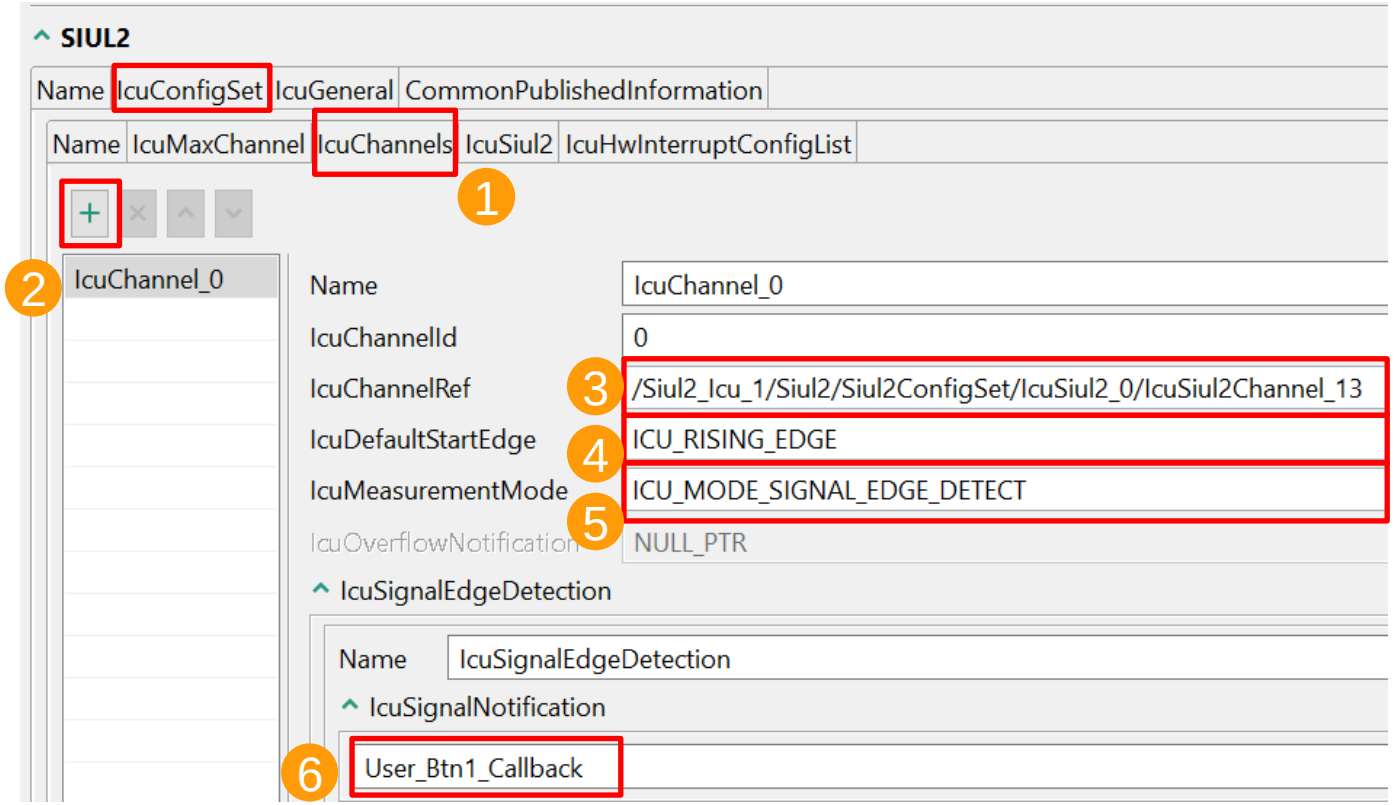
- Set the filter clock prescaler
- Enable the glitch filter
- Set the filter counter

Interrupt Filter setting => IFMCRn registers
Interrupt Filter Clock Prescaler => IFCPR register
(Filter clock source is FIRC 48MHz)

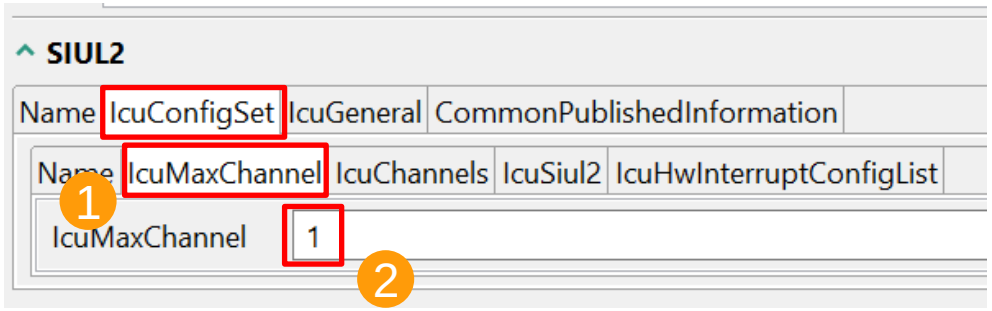


Add ICU channel and configure the property:

1. Select the tab “IcuChannels”
2. Clock “plus” button to add an icu channel
3. IcuChannelRef – must select from the existing enabled IRQ channels.
4. Select rising edge, falling edge, or both edges.
5. IcuMeasurementMode - must be “signal edge detect” mode.
6. Icu SignalNotification – set the callback function name which will be called when desired edge detected.



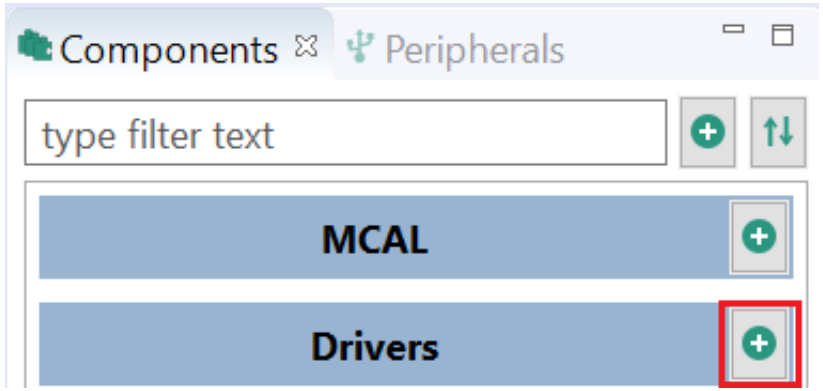
Set the number of used Siul2_Icu channels.

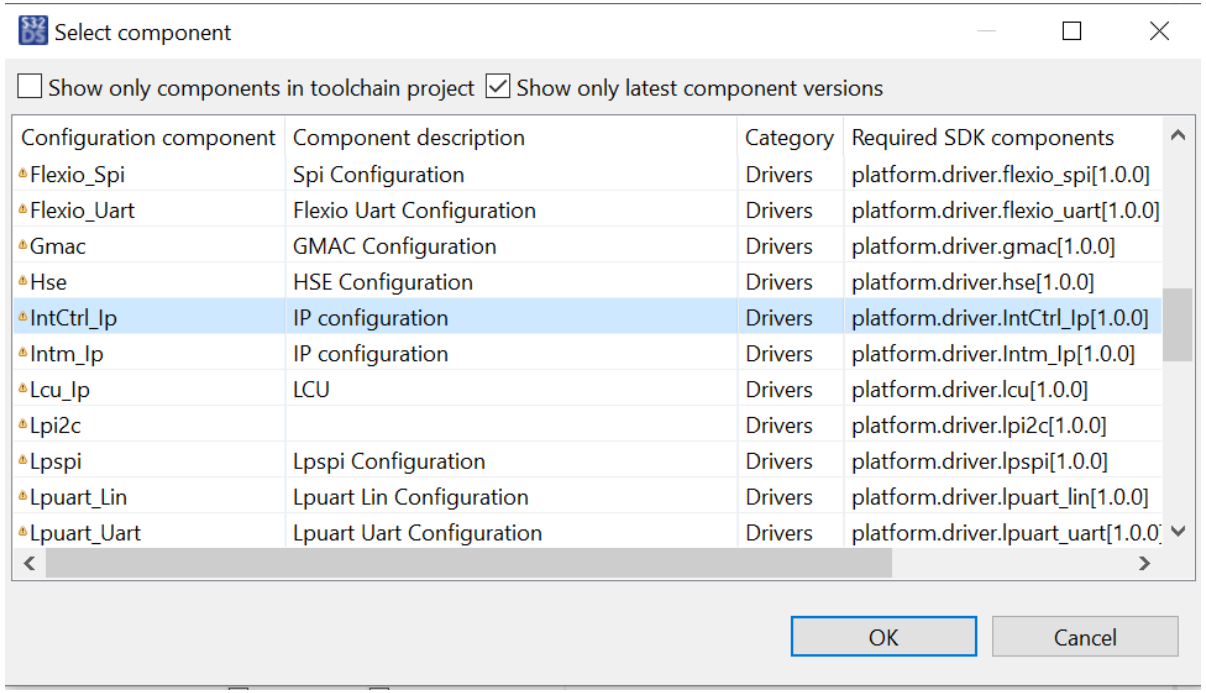


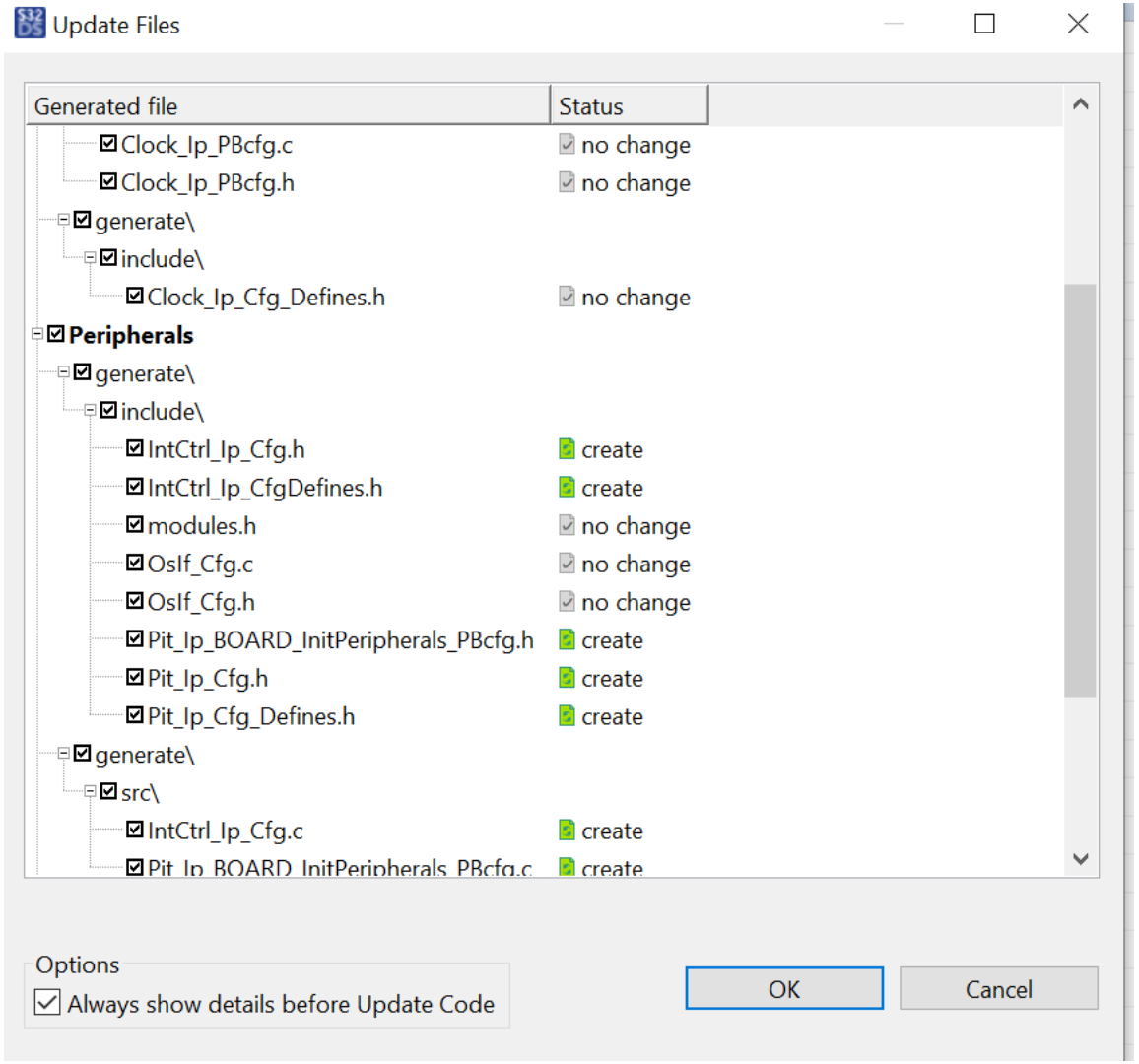
Now, the pins configuration is ready. The User LEDs and the User Push Button (USER_SW0) have been configured:

- LED BLUE: PTA31
- LED GREEN: PTA30
- LED RED: PTA29
- Push Button (USER_SW0): PTB26

- A similar procedure is required for the IntCtrl_Ip config.
- Open Peripherals ConfigTool and add a new Component.
- Scroll down and select the IntCtrl driver.







- Again, we can look at the Code Preview, and add it to the project by clicking the “Update Code” button.

In the **main.c** include the SIUL2 header and add the following functions

```
#include "Mcal.h"
#include "Siul2_Port_Ip.h"
#include "Siul2_Dio_Ip.h"
#include "Clock_Ip.h"
#include "Intctrl_Ip.h"
#include "Siul2_Icu_Ip.h"
```

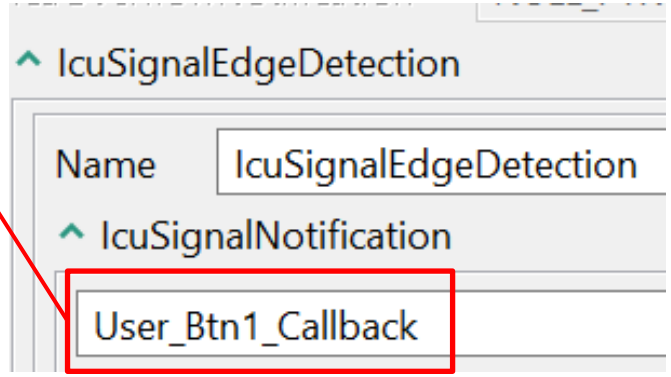
Include the necessary header files.

```
/*******************************************************************************
 * External Variables and Functions
 ******************************************************************************/
extern ISR(SIUL2_EXT_IRQ_8_15_ISR);
```

SIUL2_EXT_IRQ_8_15_ISR is the interrupt handler defined in RTD driver.
It clears the SIUL2 interrupts flags and calls the callback functions for each channel

```
/* EVB SIUL2 IRQ13 notification - PTB26 rising edge */
void User_Btn1_Callback(void)
{
    Siul2_Dio_Ip_TogglePins (LED_BLUE_PORT, (1<<LED_BLUE_PIN));
    return;
}
```

Notification function for each Siul2 IRQ channel should be defined as the same function name in Siul2_Icu configuration.



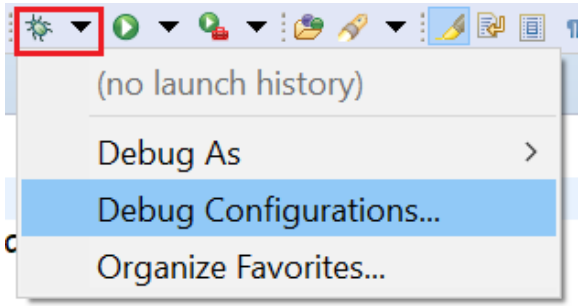
NOTE: To avoid errors, write the same name for the callback in the ICU controller and the application.

```
volatile int exit_code = 0;

int main (void)
{
    /* Clock initialization */
    Clock_Ip_Init(&Mcu_aClockConfigPB[0]);

    /* SIUL2 pin configuration */
    Siul2_Port_Ip_Init(NUM_OF_CONFIGURED_PINS0, g_pin_mux_InitConfigArr0);

    /* Install IRQ handler for SIUL2 EIRQ8~15 */
    IntCtrl_Ip_InstallHandler(SIUL_1_IRQn, &SIUL2_EXT_IRQ_8_15_ISR, NULL_PTR);

    /* Enable SIUL2 EIRQ8~15 in NVIC */
    IntCtrl_Ip_EnableIrq(SIUL_1_IRQn);

    /* SIUL2 IRQ configuration */
    Siul2_Icu_Ip_Init(SIUL2_ICU_IP_INSTANCE,
    &Siul2_Icu_Ip_0_Config_PB_BOARD_InitPeripherals);
```

```
    /* Enable SIUL2_EIRQ13 */
    Siul2_Icu_Ip_EnableInterrupt (SIUL2_ICU_IP_INSTANCE,
    (*Siul2_Icu_Ip_0_Config_PB_BOARD_InitPeripherals.pChannelsConfig)[0].hwChannel);

    /* Enable Notification */
    Siul2_Icu_Ip_EnableNotification (SIUL2_ICU_IP_INSTANCE,
    (*Siul2_Icu_Ip_0_Config_PB_BOARD_InitPeripherals.pChannelsConfig)[0].hwChannel);

    for(;;)
    {
        if(exit_code != 0)
        {
            break;
        }
    }

    return exit_code;
}
```

- Build the application (click the hammer icon).

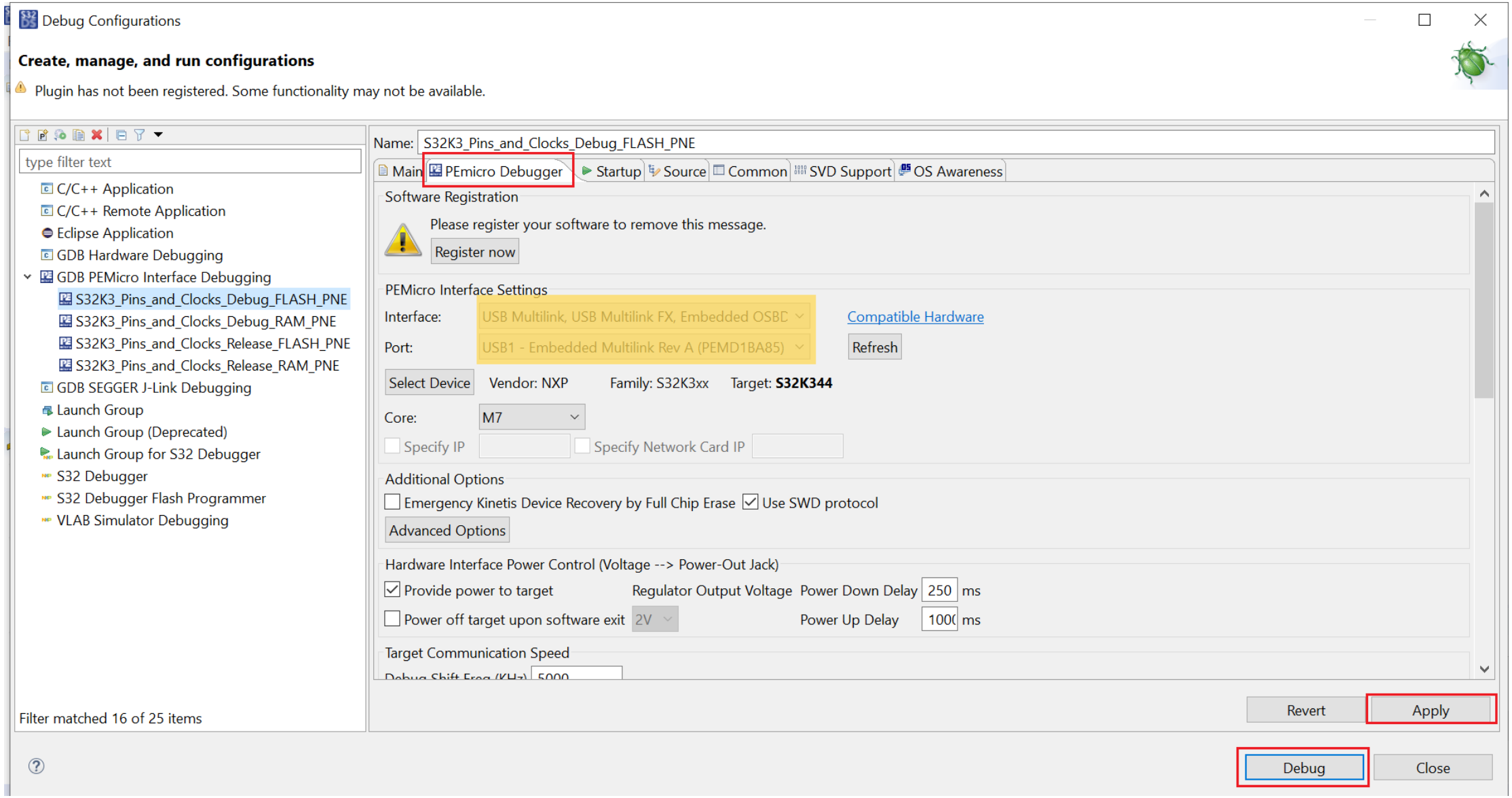
- Debug the application (click the bug icon).



- Leave the standard debug configuration as it is. (Check the next slide to be sure)
- Click on **Apply** and then **Debug**

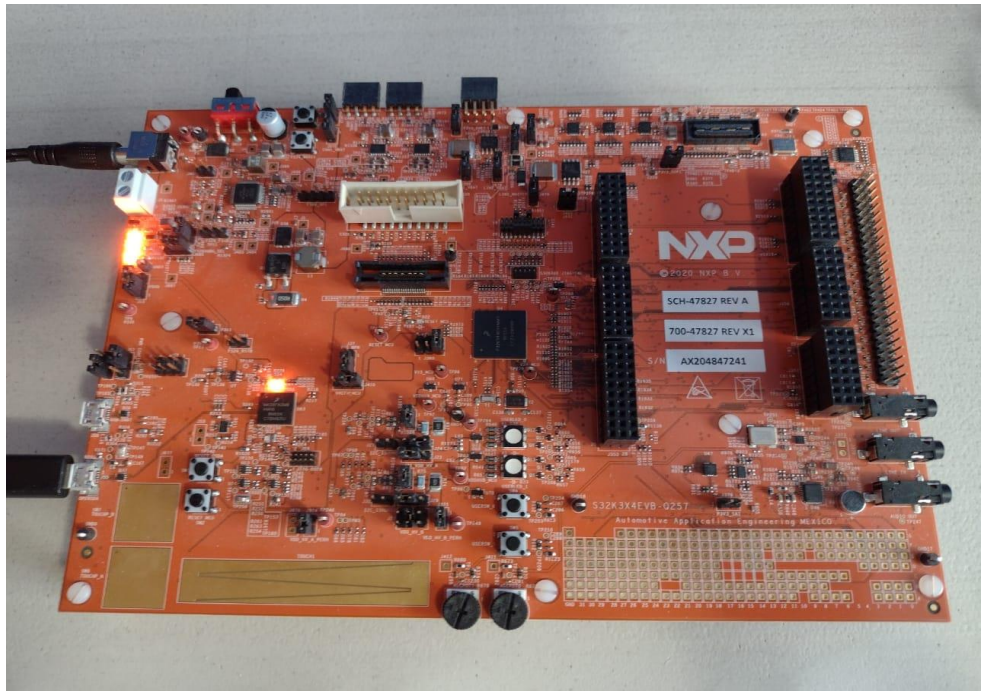
Before debugging your application, remember to have the board (EVB) connected

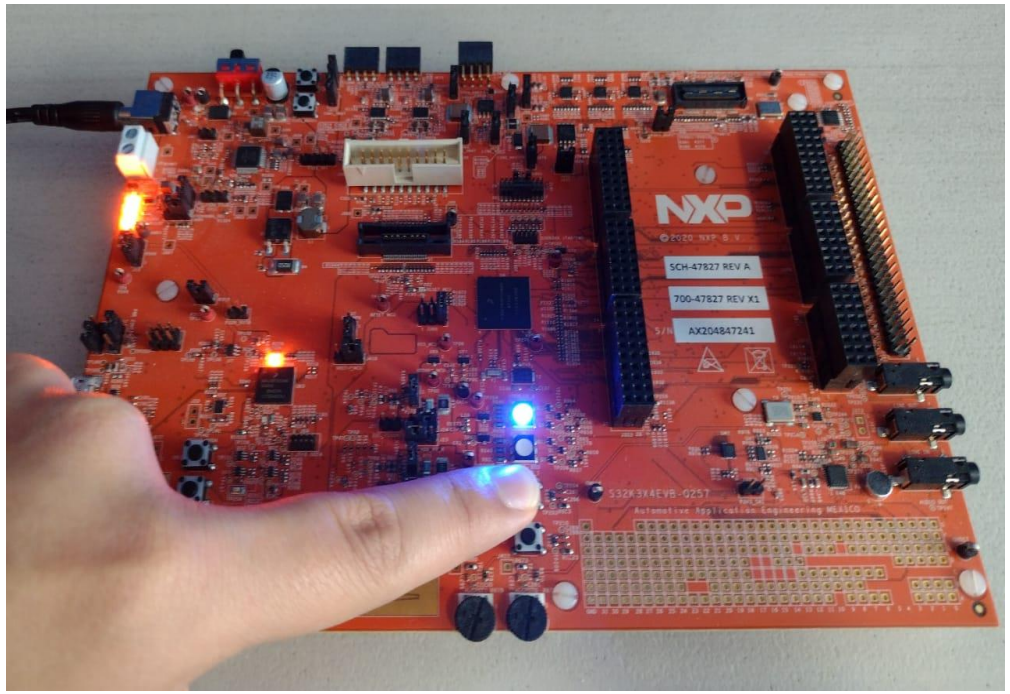
DEBUG THE APPLICATION



- Run de application (click the play icon). 

Observe the **BLUE LED** toggling every time the User Push Button is pressed (USER_SW0) on the board.





S32K3 MICROCONTROLLERS FOR AUTOMOTIVE AND INDUSTRIAL

Tackling cost and complexity of automotive software development

S32K MCU family page

S32K3 MCU page

S32DS for S32 platform page

Online engineering support:

- S32K MCUs community
- S32DS community
- S32 Configuration Tool

SECURE CONNECTIONS FOR A SMARTER WORLD

SHOWROOM.NXP.COM